Master of Science in Software Engineering
October 2026



Domain-Aware Refactoring Prioritization in Telecom Software

Srikar Katakam
Kandoti Venkata Anirudh Sankarshan

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Master of Science in Software Engineering. The thesis is equivalent to 20 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:
Author(s):
Srikar Katakam
E-mail: srka24@student.bth.se

Kandoti Venkata Anirudh Sankarshan
E-mail: vekn24@student.bth.se

University advisor:
Dr. Eriks Klotins
Department of Software Engineering

Faculty of Computing
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background. Large industrial software systems can contain more technically relevant refactoring candidates than engineering teams can address within a limited planning period. Maintainability and development-history measures can identify technically relevant files, but they do not directly represent the operational importance of the functionality to which those files belong. This motivates examining prioritization that combines technical and operational evidence transparently.

Objectives. This thesis aims to design and evaluate a deterministic, domain-aware method for the prioritization of file-level refactoring targets under limited engineering capacity in an industrial telecom software context. The study investigates which prioritization factors are relevant and feasible, and whether incorporating domain criticality provides useful planning support compared with domain-agnostic alternatives.

Methods. The study follows the Design Science Research Methodology (DSRM) and was conducted in a selected Ericsson User Plane Control (UPC) uplink software scope. The artifact combines file-level Code Health and non-merge development activity into a technical Base score and applies researcher-created module-level domain criticality as a proportional adjustment. It produces Refactor-Now and Stabilize-First planning outputs. DomainAware was compared with HealthOnly, ActivityOnly, and HealthActivity using quantitative metrics, a January–April endpoint comparison, evaluation by four Ericsson practitioners, and a two-expert assessment of the domain-criticality construct.

Results. Code Health, development activity, and module-level domain criticality were retained as relevant and feasible factors. At the April endpoint, DomainAware and HealthActivity shared eight of ten selected files; the two strategy-specific differences occurred in the MODERATE Code Health band, where the DomainAware-only files were mapped to HIGH-criticality modules and the HealthActivity-only files to MEDIUM-criticality modules. DomainAware achieved the most favourable practitioner mean rank (1.50), although HealthOnly received the same number of first-place votes. DomainAware retained identical Top-10 membership across the January and April endpoints. The two experts agreed on 28 of 31 represented modules (90.32%), indicating strong inter-expert consistency while retaining classification uncertainty.

Conclusions. The study provides context-specific evidence that domain-aware prioritization can support transparent pre-refactoring decision-making in the studied context. The artifact makes operational context explicit while distinguishing immediate refactoring candidates from those requiring additional preparation. The evidence supports contextual usefulness and auditability, but does not establish universal superiority, general temporal stability, optimal parameter settings, refactoring safety, or post-refactoring improvement.

Keywords: refactoring prioritization, Code Health, development activity, domain criticality, telecom software.

Acknowledgments

We would like to express our sincere gratitude to our university advisor, Eriks Klotins, for his guidance, feedback, and support throughout this thesis work.

We are also grateful to Sasank Sai Sujan Adapa, our Ericsson contact and manager, for providing industrial support, practical direction, and valuable inputs during the project.

We would also like to thank the domain practitioners and experts for sharing their knowledge, reviewing the domain-related aspects of the work, and helping us understand the industrial context of the selected software scope.

Their support and feedback were important for shaping the thesis and for ensuring that the proposed method remained relevant to the practical software engineering context studied in this work.

Contents

Abstract i

Acknowledgments iii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement and Research Gap . . . 2
- 1.3 Aim and Research Questions . . . 3
- 1.4 Scope and Contribution . . . 3
- 1.5 Thesis Outline . . . 4

2 Related Work 5
- 2.1 Chapter Overview . . . 5
- 2.2 Technical Debt Management and Prioritization . . . 5
- 2.3 Refactoring Target Selection Under Limited Engineering Capacity . . . 7
- 2.4 Maintainability, Code Smells, and Code Health . . . 7
- 2.5 Development Activity, Change Metrics, and Behavioural Code Analysis 8
- 2.6 Context, Business, and Architecture-Aware Technical Debt Decisions 9
- 2.7 Telecom Context and Operational Criticality . . . 10
- 2.8 Automated Prioritization Alternatives . . . 11
- 2.9 Positioning of This Thesis and Research Gap . . . 12

3 Method 15
- 3.1 Chapter Overview . . . 15
- 3.2 Research-Question Traceability . . . 15
- 3.3 Research Design and Method Selection . . . 15
  - 3.3.1 Design Science Research Methodology . . . 15
  - 3.3.2 Rigor, Relevance, and Iterative Artifact Development . . . 16
  - 3.3.3 Alternative Research Designs Considered . . . 21
- 3.4 Industrial Context, Scope, and Unit of Analysis . . . 21
  - 3.4.1 Industrial Context and Scope . . . 21
  - 3.4.2 Units of Analysis . . . 22
- 3.5 Data Sources and Candidate Construction . . . 22
  - 3.5.1 Data Sources . . . 22
  - 3.5.2 Candidate Construction . . . 23
  - 3.5.3 Analysis Windows . . . 24
- 3.6 Artifact Design and Development . . . 25
  - 3.6.1 Factor Feasibility . . . 25

3.6.2 Technical Signal Operationalization . . . 25
3.6.3 Domain Criticality Construction and Consistency Review . . . 27
3.6.4 Technical Base Score . . . 29
3.6.5 Domain-Aware Score . . . 30
3.6.6 From Ranking to Planning . . . 32
3.7 Demonstration and Quantitative Evaluation . . . 36
3.7.1 Baseline Strategies . . . 36
3.7.2 Evaluation Metrics . . . 37
3.8 Temporal Evaluation . . . 38
3.9 Practitioner Evaluation . . . 39
3.9.1 Participants and Evaluation Material . . . 39
3.9.2 Procedure and Analysis . . . 40
3.10 Independent Domain-Criticality Validation . . . 41
3.11 Reproducibility, Traceability, and Confidentiality . . . 43
3.12 Validity and Reliability . . . 44
3.12.1 Construct Validity . . . 44
3.12.2 Internal Validity . . . 45
3.12.3 Reliability . . . 45
3.12.4 External Validity and Transferability . . . 46
3.12.5 Human-Evaluation and Temporal Limitations . . . 46

4 Results and Analysis 47
4.1 Chapter Overview . . . 47
4.2 Study Population and Artifact-Development Results . . . 47
4.2.1 Population and Factor-Feasibility Results . . . 47
4.2.2 Domain-Classification Consistency-Review Results . . . 48
4.3 April Endpoint Prioritization Results . . . 49
4.3.1 Final Refactor-Now Selections . . . 49
4.3.2 Quantitative Planning Metrics . . . 49
4.3.3 Selection Overlap and DomainAware-HealthActivity Differences 50
4.4 Stabilize-First Results . . . 51
4.5 January Endpoint Results . . . 52
4.6 Temporal Sensitivity Across the January and April Endpoints . . . 53
4.7 Practitioner Evaluation Results . . . 54
4.7.1 Strategy Ranking . . . 54
4.7.2 Strategy Ratings . . . 54
4.7.3 Qualitative Observations and Stabilize-First Review . . . 55
4.8 Independent Domain-Criticality Validation . . . 56
4.8.1 Expert Completion and Label Distributions . . . 57
4.8.2 Agreement Between the Two Experts . . . 57
4.8.3 Agreement With the Frozen Researcher Mapping . . . 58
4.8.4 Disagreement Patterns and Module-Level Observations . . . 60
4.9 Summary of Results . . . 61

5 Discussion **62**
5.1 Chapter Overview . . . 62
5.2 RQ1: Relevant and Feasible Prioritization Factors . . . 62
5.3 RQ2: Usefulness of the Domain-Aware Artifact . . . 64
5.3.1 Quantitative Comparison with the Baselines . . . 65
5.3.2 Practitioner Assessment of Planning Usefulness . . . 66
5.4 Interpreting the Stabilize-First Mechanism . . . 67
5.5 Temporal Sensitivity of the Prioritization . . . 68
5.6 Domain-Criticality Construct and Independent Expert Evidence . . . 68
5.7 Relation to Prior Work and Research Contribution . . . 70
5.8 Alternative Explanations and Limits of Interpretation . . . 71
5.9 Practical Implications and Transferability . . . 73
5.10 Ethical, Societal, and Sustainability Considerations . . . 74
5.11 Summary of Discussion . . . 75

6 Conclusions and Future Work **76**
6.1 Conclusions from the Research Questions . . . 76
6.1.1 Relevant and Feasible Prioritization Factors . . . 76
6.1.2 Usefulness of the Domain-Aware Prioritization Artifact . . . 76
6.2 Research and Practical Contribution . . . 77
6.3 Future Work . . . 78
6.3.1 Replication Across Software Contexts and Time . . . 78
6.3.2 Strengthening the Domain-Criticality Construct . . . 78
6.3.3 Parameter Sensitivity and Artifact Extension . . . 79
6.3.4 Longitudinal Post-Refactoring Evaluation . . . 79
6.4 Final Conclusion . . . 80

References **81**

A Supporting Evidence and Reproducibility **85**
A.1 Evidence, Anonymization, and Study Identifiers . . . 85
A.2 Architecture Scope and Anonymized Module Context . . . 86
A.3 Frozen Researcher Domain-Criticality Mapping . . . 93
A.4 Frozen Artifact Configuration . . . 94
A.5 Endpoint Population and Eligibility Results . . . 96
A.6 Prioritization Output Evidence . . . 98
A.7 Quantitative Evaluation Evidence . . . 100
A.8 Practitioner Evaluation Evidence . . . 102
A.9 Independent Expert Validation Evidence . . . 103

B Generative AI Disclosure **105**
B.1 Example: Academic Language and Clarity Revision . . . 105

Chapter 1

Introduction

1.1 Background and Motivation

Modern telecommunications software must evolve continuously while supporting communication infrastructure on which users and services depend. New functionality, defect corrections, performance improvements, security updates, and operational adaptations must be introduced while preserving the ability of the software to be understood, modified, tested, and maintained over time. Maintainability is therefore an important software-quality characteristic [17], and software maintenance represents a substantial part of the lifecycle effort of long-lived systems [11].

This thesis was conducted with Ericsson in a selected software scope within the 5G Radio Access Network (RAN) baseband environment. The empirical study focuses on a proprietary User Plane Control (UPC) uplink software scope. At a high level, the user plane is concerned with the handling of user data, while the uplink refers to transmission from user equipment toward the network. The selected UPC uplink scope represents one bounded part of Ericsson's broader 5G NR software environment.

The industrial environment also provides the main sources of evidence used in the study. CodeScene is used to obtain file-level Code Health, a maintainability-oriented measure reported on a 1-10 scale [7, 8]. Git repository history provides evidence about recent file-level development activity, while Ericsson architecture and high-level design (HLD) material provides contextual information about module responsibilities and operational behaviour. The collection and operationalization of these evidence sources are described in Chapter 3.

One way of describing accumulated maintainability problems is through technical debt. Technical debt captures conditions or design choices that can make subsequent software evolution more difficult [6, 20]. Refactoring provides a way to improve the internal structure of software while preserving its externally observable behaviour [3, 14, 23].

Identifying technically problematic code does not, however, determine what should be addressed first. Prior research shows that technical-debt prioritization involves choosing among competing maintenance concerns using more than technical condition alone [19]. Repository-derived maintainability and development-history information can provide useful evidence for such decisions, but these signals represent different aspects of the software and do not by themselves capture the operational importance of the functionality associated with a file.

This distinction is particularly relevant in the industrial context studied. Files

with similar maintainability and development-activity characteristics may belong to modules with different operational consequences or available fallback behaviour. This motivates examining whether domain criticality can complement repository-derived evidence when prioritizing limited file-level refactoring attention.

1.2 Problem Statement and Research Gap

The practical problem addressed in this thesis is how to select a limited set of refactoring candidates when more technically relevant files exist than can be addressed within a planning period. Maintainability information can identify files with structural concerns, while development history can indicate where developers have recently interacted with the software. However, neither type of evidence directly represents the operational importance of the functionality to which a file belongs. On the other hand, operational importance alone does not establish whether a file is an appropriate candidate for immediate structural change.

The planning problem therefore motivates combining technical evidence with operational context while keeping the resulting decision process transparent and bounded. It also motivates distinguishing between files that should receive near-term refactoring consideration and technically relevant, operationally important files for which additional preparation or engineering review may be appropriate before structural change.

Prior research establishes that technical-debt prioritization can involve technical and stakeholder considerations [19], business and value considerations [5, 27], and architectural considerations [22]. Development history can also provide evidence that complements static code properties when identifying areas of engineering interest [24, 29].

However, a gap remains in how these different forms of evidence can be operationalized together for transparent and bounded file-level pre-refactoring planning in an industrial telecom context. In particular, the prior approaches reviewed in this thesis do not directly address how file-level technical evidence can be combined with module-level operational context while also distinguishing prioritization from the immediacy of structural change.

A further issue is that operational criticality is not directly observed from repository data. It depends on interpretation of architecture and domain evidence. Any method using such information should therefore make the classification procedure explicit and allow the resulting domain judgments to be assessed separately rather than treating them as objective ground truth.

Accordingly, the research problem addressed in this thesis is how to construct and evaluate a deterministic and transparent file-level refactoring-prioritization method that combines maintainability condition, recent development interaction, and operational context under limited engineering capacity, while distinguishing near-term refactoring candidates from candidates requiring additional preparation or review.

1.3 Aim and Research Questions

The aim of this thesis is:

To design and evaluate a deterministic, domain-aware method for prioritizing file-level refactoring targets under limited engineering capacity in an industrial telecom software context.

To achieve this aim, the thesis pursues two objectives: (i) identify prioritization factors that are both relevant and feasible in the selected software scope, and (ii) construct and evaluate a deterministic domain-aware planning artifact against domain-agnostic baselines using quantitative, temporal, and practitioner evidence.

The study addresses this aim through two research questions:

> **RQ1:** Which prioritization factors are relevant and feasible for file-level refactoring-target selection in the selected telecom software context?
>
> **RQ2:** How useful is the resulting domain-aware refactoring-prioritization method compared with domain-agnostic baselines from quantitative and practitioner perspectives?

RQ1 concerns the evidence from which the prioritization artifact is constructed. A factor must be conceptually relevant to the decision problem while also having sufficiently complete and traceable evidence to support systematic operationalization in the studied scope. The study therefore does not assume in advance that every theoretically useful signal can be incorporated.

RQ2 concerns the usefulness of the finalized planning artifact rather than the empirical optimality of one formula or parameter setting. In this thesis, usefulness is operationalized through two complementary perspectives: the quantitative characteristics and temporal sensitivity of the selected candidate sets, and practitioner assessment of candidate reasonableness, avoidance of inappropriate immediate refactoring, actionability, scheduling realism, and operational alignment. The domain-aware method is therefore compared with simpler domain-agnostic strategies using these quantitative and practitioner perspectives. Because domain criticality is constructed by the researchers from architecture and high-level design evidence, the construct is examined separately through an independent two-expert assessment. This assessment evaluates agreement and disagreement in the module-level classifications rather than inferring domain-label validity from practitioner preference for the generated prioritization outputs.

The resulting artifact is intended to support software engineers and technical decision-makers who must allocate limited refactoring capacity while retaining visibility of both technical evidence and operational context.

1.4 Scope and Contribution

The empirical study is bounded to the selected Ericsson UPC uplink software scope. The source-code file is the primary unit of analysis and the unit returned by the prioritization artifact. Operational context is assessed at module level using architecture and HLD evidence and is mapped to files through the existing module-to-file relationship.

The study focuses on pre-refactoring planning. It prioritizes files for engineering consideration but does not implement the selected refactorings or evaluate their downstream effects. The thesis therefore does not claim that following the recommendations will reduce defects, improve post-refactoring maintainability, increase engineering productivity, improve network reliability, or make a particular refactoring objectively safe.

The research contribution is the design, operationalization, and industrial evaluation of a deterministic and auditable file-level prioritization artifact that combines maintainability evidence, development-history evidence, and module-level operational context. The artifact provides two planning outputs, Refactor-Now and Stabilize-First, to distinguish near-term refactoring consideration from candidates requiring additional preparation or review. The detailed artifact design and evaluation procedures are presented in Chapter 3.

The artifact is intended as decision support rather than automated authority. Engineers remain responsible for reviewing the evidence and deciding whether and how a recommended candidate should be acted upon. The empirical scope is intentionally bounded. Numerical findings from the selected UPC uplink scope are not statistically generalized to the complete Ericsson codebase or to telecom software as a whole. The artifact structure may be examined in other software contexts where comparable technical, repository, module-mapping, and domain evidence are available, but its parameters and classifications require context-specific evaluation.

Confidentiality constrains public reporting. Proprietary file paths, architecture descriptions, and implementation-specific evidence remain within the Ericsson-controlled research material. The public thesis uses anonymized file and module identifiers and aggregated results where required.

Use of AI-assisted and language-support tools. AI-assisted and language-support tools were used during the preparation of this thesis for literature-search support, literature-review assistance, language editing, document preparation, and visual presentation. These tools were not treated as scientific sources or as substitutes for the empirical evidence, methodological decisions, or interpretation performed by the authors. A complete description of the tools used, their respective roles, and an example of AI-assisted language revision is provided in Appendix B.

1.5 Thesis Outline

Chapter 2 reviews the literature on technical-debt prioritization, refactoring-target selection, maintainability, development-history analysis, and context-aware engineering decisions, and establishes the research gap.

Chapter 3 presents the DSRM-based research design, artifact construction, evaluation procedure, and validity considerations.

Chapter 4 reports the April and January endpoint results, baseline comparisons, practitioner evaluation, and expert domain assessment.

Chapter 5 interprets these findings in relation to the research questions, prior work, practical transferability, and study limitations.

Chapter 6 answers the research questions, summarizes the contribution, and outlines future work.

Chapter 2

Related Work

2.1 Chapter Overview

This chapter reviews the research relevant to prioritizing refactoring attention under limited engineering capacity. The purpose is not to provide a general survey of all refactoring or technical-debt research, but to establish the research foundations for the specific decision problem addressed in this thesis and to position the proposed artifact against related approaches.

The literature is organized around several complementary perspectives. Technical-debt research explains why software-maintenance problems require explicit prioritization rather than indiscriminate repayment. Refactoring research establishes target selection as a decision that occurs before structural changes are implemented. Maintainability and Code Health provide evidence about technical condition, while software-evolution and Behavioural Code Analysis research motivates the use of development history. Research on architectural and business-aware technical-debt decisions demonstrates that contextual consequences and stakeholder priorities also influence what should be addressed. Finally, recent automated prioritization research provides an alternative to deterministic rule-based approaches.

These streams are directly relevant to the artifact developed in this thesis. However, they represent different kinds of evidence rather than interchangeable measurements of one concept. Code Health represents a maintainability concern, development activity represents recent engineering interaction, and domain criticality represents operational context. The thesis combines these signals while retaining their distinct meanings.

The chapter concludes by synthesizing these research areas and identifying the integration gap addressed by this thesis.

2.2 Technical Debt Management and Prioritization

Technical debt describes technical decisions or conditions that may enable short-term objectives while creating future costs or constraints on software evolution [6, 20]. These costs may appear as increased maintenance effort, slower delivery, additional rework, reduced evolvability, or difficulty modifying the system safely.

Technical-debt management therefore involves more than identifying technical problems. Li, Avgeriou, and Liang identify management activities including identification, measurement, prioritization, monitoring, and repayment [20]. Similarly,

Alves et al. show that managing technical-debt requires organizations not only to identify debt but also to decide which debt should be addressed and when [6].

Prioritization becomes necessary because organizations rarely have the capacity to pay back each debt item. Lenarduzzi et al.'s systematic literature review of technical-debt prioritization found a broad range of prioritization strategies, factors, processes, and tools [19]. Various considerations such as severity of the technical debt, cost, value, risk, architectural significance, change rate, and judgment of stakeholders have been reported. Therefore the review presents technical-debt prioritization as a multi-dimensional decision problem rather than a ranking determined by one universally accepted metric [19].

Further empirical evidence supports this conclusion. Alfayez et al. investigated prioritization of SonarQube-identified technical-debt items in four companies and a master's-level course, involving 89 participants working under a resource constraint [5]. Most participants considered both the value and cost of debt items, and the study found that perceived value was subjective and context-dependent. The authors concluded that no single technical-debt prioritization strategy satisfies all developer objectives.

This is particularly relevant to the present thesis because the prioritization problem also begins from limited engineering capacity. The issue is not simply whether a file contains a maintainability concern, but whether that file should receive attention before competing candidates.

Another industrial perspective is provided by de Almeida et al., who evaluated a business-driven technical-debt prioritization approach in a five-month industrial case study involving both technical and business stakeholders [27]. The findings from their work indicate that incorporating business-oriented criteria helped align prioritization with high-value business assets and revealed multiple business factors affecting technical-debt decisions.

Aldaeej and Seaman further illustrate that technical debt decisions may be explicitly represented using decision rules. Their 2024 multiple-case study examined technical-debt decisions in five web and mobile application startups and developed a rule-based decision model from the observed decisions [4]. The work is relevant because it treats technical-debt management as contextual decision-making that can nevertheless be expressed through explicit, inspectable rules.

Together, these studies provide three foundations for this thesis. First, limited resources make prioritization necessary. Second, technically detected debt does not by itself determine priority. Third, contextual information and explicit decision logic can contribute to more transparent prioritization.

The present study differs in the unit and context of analysis. It does not prioritize general technical-debt items or business processes. Instead, it focuses on file-level pre-refactoring planning in a bounded telecom software scope and investigates how technical and operational evidence can be combined deterministically.

2.3 Refactoring Target Selection Under Limited Engineering Capacity

Refactoring is commonly understood as improving the internal structure of software while preserving its externally observable behaviour [14, 23]. Fowler describes refactoring as a disciplined approach to improving design without altering intended functionality [14], while Mens and Tourwé survey refactoring techniques, tool support, and research challenges [23].

The refactoring problem can be separated into two decisions: where refactoring attention should be directed and what transformation should subsequently be performed. This distinction matters for this thesis because it addresses the first decision.

Al Dallal's systematic literature review identifies the detection of refactoring opportunities as a distinct stage that precedes the actual refactoring process and reviews techniques for identifying such opportunities [3]. This supports treating target identification as a research problem in its own right.

In a large industrial system, identifying a maintainability problem does not automatically establish that it should be addressed immediately. Engineering capacity is limited, multiple technically relevant candidates can compete for attention, and different files can have different development or operational contexts. Consequently, target selection is a planning problem as well as a code-quality problem.

This distinction is particularly important for the artifact developed in this thesis. The thesis does not propose a new refactoring transformation and does not evaluate whether a particular code transformation preserves behaviour. Instead, it produces a bounded set of files that deserve engineering consideration before a refactoring is selected and implemented.

Recent code-smell prioritization research provides another perspective on this resource-allocation problem. Liu et al. argue that code-smell instances can differ in severity and that limited refactoring resources make prioritization necessary [21]. They compare 49 learning-to-rank configurations across four code-smell severity datasets and propose ranking-oriented evaluation metrics. Their work demonstrates that target prioritization can be formulated as a learned ranking problem rather than only as manual or rule-based selection.

The present thesis investigates a different design point. A supervised ranking approach requires appropriate labelled examples from which ranking behaviour can be learned. The studied Ericsson context did not provide a labelled ground-truth dataset of historically correct refactoring priorities. The thesis therefore uses a deterministic artifact whose inputs, rules and effects can be inspected directly. This choice does not imply that learned ranking is generally unsuitable. It reflects the evidence and transparency requirements of the present industrial case.

2.4 Maintainability, Code Smells, and Code Health

Maintainability is an established software-quality characteristic concerned with how effectively and efficiently software can be modified [17]. Refactoring is closely associated with maintainability because structural improvement is intended to make

future evolution easier without changing externally observable behaviour [14, 23].

Code smells provide one way of identifying structural problems that may affect maintainability. Palomba et al. studied code smells across multiple releases of open-source systems and reported associations between smells and maintenance-related outcomes [25]. Such findings support the general idea that internal structural characteristics can provide useful evidence for identifying areas that may require maintenance attention.

The present thesis does not prioritize individual code smells directly. Instead, it uses Code Health, provided by CodeScene, as an aggregated file-level maintainability signal. CodeScene reports Code Health on a 1-10 scale as an aggregated metric based on more than 25 factors. The exact set varies with programming language and includes code properties such as complexity, cohesion, duplication, and code size, together with organizational factors considered by CodeScene [7, 8].

Empirical work using CodeScene also provides evidence that maintainability condition is associated with engineering outcomes. Tornhill and Borg analyzed 39 proprietary production codebases and reported that lower-quality code was associated with more defects and longer issue-resolution times [30]. This does not establish that refactoring any low-health file will automatically improve those outcomes, but it supports treating structural code quality as a meaningful source of maintenance evidence.

This distinction is important for the thesis. Code Health represents maintainability concern, not a direct measurement of business importance, operational risk, or the correct order in which files should be refactored. A file with poor Code Health may deserve attention, but competing candidates may differ in recent development interaction or operational context.

This is also why a Code-Health-only strategy provides an important baseline in the study. It represents a simple and understandable prioritization strategy grounded in maintainability conditions. If the proposed artifact is useful, its contribution must be demonstrated relative to such a meaningful alternative rather than against an artificially weak baseline.

2.5 Development Activity, Change Metrics, and Behavioural Code Analysis

Static code properties provide only one view of a software system. Version-control history offers additional evidence about how the software evolves and where developers repeatedly interact with it.

Moser, Pedrycz, and Succi compared change metrics with static code attributes for defect prediction and found that development-history measures can provide useful information beyond static structural attributes [24]. Although the present thesis does not use activity for defect prediction, this work demonstrates a broader principle: repository history contains information that is not fully represented by the current source-code structure.

Behavioural Code Analysis develops this principle further by analysing source code together with its development history [29]. Tornhill argues that areas receiv-

ing repeated developer attention are especially relevant because quality problems in frequently modified code can impose recurring maintenance costs [29].

CodeScene operationalizes this reasoning through hotspots, where development activity is used to identify files or components receiving substantial engineering attention [9]. When combined with maintainability information, hotspot analysis can highlight code that is both structurally problematic and repeatedly modified [9, 29].

The present thesis uses recent file-level development activity as a complementary signal to maintainability. Activity is represented by the number of non-merge commits that changed a file during the corresponding analysis window. Merge commits are excluded so that branch-integration events are not counted as additional development interactions when the underlying file-changing commits are already present in the repository history. The detailed transformation and normalization of this signal are described in Section 3.6.2. Activity is therefore interpreted as recent development interaction rather than as a direct measure of effort, defect probability, or operational importance.

This distinction is central to the research problem. A highly active file receives repeated development interaction, but this does not mean that it is the most operationally important file or necessarily the best immediate refactoring target. Conversely, a relatively inactive file may implement mature but operationally important functionality. Development activity therefore complements maintainability rather than replacing it.

This reasoning also motivates two of the study's baselines. ActivityOnly represents prioritization driven exclusively by recent development interaction, while HealthActivity combines the maintainability and activity signals without using domain information. HealthActivity is particularly important because it provides the most direct comparison for determining what changes when operational context is added.

2.6 Context, Business, and Architecture-Aware Technical Debt Decisions

Technical-debt prioritization literature increasingly recognizes that debt decisions cannot be separated from system context.

Martini and Bosch's AnaConDebt approach is particularly relevant. Their empirically developed method supports decisions concerning architectural technical debt by considering aspects such as consequences, refactoring decisions, timing, and stakeholder reasoning [22]. This work demonstrates that technical-debt management is not solely an assessment of structural deficiency, as decisions also depend on the consequences of the debt and the context in which repayment would occur.

The present thesis shares this decision-oriented perspective but differs in granularity. AnaConDebt addresses architectural technical-debt cases, whereas the present artifact ultimately produces file-level planning outputs. Architecture is therefore not the final unit of prioritization. Instead, architectural knowledge provides contextual evidence that is mapped to the files being ranked.

Business-aware technical-debt research provides a complementary perspective.

De Almeida et al. showed that business criteria can materially affect which technical-debt items organizations consider most important and identified eight business factors influencing prioritization decisions [27]. Their work supports the broader argument that purely technical measures do not necessarily represent the importance of a debt item to the organization.

Alfayez et al. similarly found substantial variation in how participants valued SonarQube-identified technical-debt items. Most participants balanced value and cost, but the perceived value of an item depended on the participant's context and perspective [5]. This is directly relevant to the practitioner evidence in the present thesis: a quantitative ranking can organize evidence, but practitioners may still interpret the suitability of individual candidates differently.

Aldaeej and Seaman's rule-based model provides another relevant connection. Their work shows that contextual technical-debt decisions can be expressed as explicit decision rules derived from empirical cases [4]. The present thesis adopts a similarly explicit philosophy, although the rules and target context are different: the artifact exposes its scoring, eligibility, and planning rules rather than learning them implicitly.

Taken together, these studies motivate the introduction of domain criticality in the present artifact. Domain criticality is not intended to replace Code Health or Activity. It represents a separate contextual question: what are the operational consequences associated with the module to which a technically relevant file belongs?

The present study represents operational context at module level through three dimensions: *Service Impact*, referring to the likely consequence if the module fails or degrades; *Operational Criticality*, referring to the importance of the module's functionality to live system operation; and *Workaround Availability*, referring to whether a practical fallback, degraded mode, or workaround exists. These dimensions were selected during the factor-feasibility assessment because the available Ericsson architecture and HLD material provided comparable evidence for them across the represented modules. The detailed construction procedure is described in Section 3.6.3.

However, contextual knowledge introduces a construct-validity challenge that repository-derived measures do not have to the same degree. Operational importance requires engineering interpretation, and different assessors may assign different criticality levels even when considering the same architectural context. The present thesis therefore treats domain criticality as an explicitly reviewable construct rather than as objective ground truth. The researcher-created module classifications are examined separately through independent expert assessment, allowing agreement and disagreement in the domain construct to be evaluated without conflating domain-label validity with practitioner preference for the generated prioritization outputs.

2.7 Telecom Context and Operational Criticality

The empirical setting of the thesis is a selected Ericsson User Plane Control (UPC) uplink software scope. Telecom software provides a relevant setting for studying contextual prioritization because it operates within communication infrastructure where security and privacy concerns are well documented [12, 13, 18].

At the broader 5G New Radio (NR) physical-layer level, the Third Generation Partnership Project (3GPP) TS 38.211 defines uplink physical channels including

the Physical Uplink Shared Channel (PUSCH), Physical Uplink Control Channel (PUCCH), and Physical Random Access Channel (PRACH) [2]. This public standard provides broader context for the software environment. It does not imply that the selected proprietary UPC scope implements every function described by the physical-layer specification.

Security-focused 5G literature and guidance further illustrate the operational sensitivity of the wider telecom environment. The European Union Agency for Cybersecurity (ENISA) discusses security concerns across 5G specifications [12], while Fang et al. and Khan et al. survey security and privacy challenges in 5G networks [13, 18]. At the standards level, 3GPP TS 33.501 specifies the security architecture and procedures for the 5G System [1].

These sources are not refactoring-prioritization studies, and they should not be used to claim that a particular file is safety or security-critical. Their relevance is narrower: they establish that the software exists in an operational environment where functionality can differ in consequence and where engineering changes must be considered in context.

The domain-criticality construct used in this thesis therefore comes primarily from the architecture and HLD evidence of the selected industrial scope, not from general 5G security literature.

This distinction also matters for the Stabilize-First mechanism. Architecture-aware technical-debt research supports considering consequences and timing when deciding whether debt should be addressed [22]. Business- and rule-based studies further show that contextual value and explicit decision logic can influence technical-debt decisions [4, 27]. The prior literature reviewed for this thesis, however, did not provide an equivalent deterministic file-level rule that separates HIGH-criticality, relatively low-activity candidates into a distinct preparation queue.

Stabilize-First should therefore be understood as an artifact design contribution inspired by the broader decision-oriented literature, not as a previously validated refactoring-safety metric. Its purpose is to make the distinction between *importance* and *immediacy of action* explicit.

2.8 Automated Prioritization Alternatives

Refactoring and technical-debt prioritization can also be approached through automated ranking.

Liu et al. investigate Code Smell Severity Prioritization using learning-to-rank techniques [21]. Their study compares 41 pointwise, four pairwise, and four listwise learning-to-rank algorithms across four code-smell severity datasets. They argue that prioritization becomes particularly important when refactoring resources are limited and that evaluation should focus on the quality of the ranking rather than only on generic prediction accuracy.

This work is relevant because it represents an alternative answer to the same broad resource-allocation problem: when many code instances could be improved, which should receive attention first?

The present study intentionally takes a different approach. Rather than training a model to predict severity ranking from labelled examples, it constructs a deter-

ministic score from explicitly defined evidence. This design was appropriate because the industrial case did not provide a validated dataset of historically correct file-level refactoring priorities from which a supervised ranking model could be trained.

The difference also affects interpretability. In the present artifact, a practitioner can trace a recommendation from the underlying maintainability, development-history, and module-level contextual evidence through the explicit scoring and planning rules to the final prioritization result. This traceability is important because the thesis evaluates the method as a transparent decision-support artifact rather than only as a predictive model. The detailed scoring and planning procedure is described in Chapter 3.

The two approaches should therefore not be treated as competitors where one is universally preferable. Learned prioritization is promising when suitable labelled datasets and evaluation targets exist. Deterministic prioritization is useful when the evidence needs to remain directly auditable and when appropriate training labels are unavailable.

Future work could eventually compare these alternatives if a sufficiently large and trustworthy history of practitioner prioritization or refactoring outcomes becomes available.

2.9 Positioning of This Thesis and Research Gap

The reviewed literature establishes several foundations for the thesis.

Technical-debt research establishes that debt must be prioritized when repayment resources are limited [19, 20]. Prioritization can additionally involve business and value considerations [5, 27], alongside technical, architectural, and stakeholder considerations [19].

Refactoring research distinguishes structural transformation mechanics [14, 23] from the preceding task of identifying refactoring opportunities [3], while recent code-smell research shows that prioritization can also be formulated as a learned ranking problem [21].

Maintainability research supports structural quality as a meaningful source of maintenance evidence [25, 30], while CodeScene documentation provides the Code Health concepts used as the maintainability signal in this thesis [7, 8].

Software-evolution research and Behavioural Code Analysis support the use of development history as complementary evidence [9, 24, 29].

Architecture-aware technical-debt research emphasizes consequences, timing, architecture, and stakeholder reasoning [22], while business-aware studies show that contextual value can affect prioritization [5, 27]. Rule-based research further shows that explicit decision logic can represent technical-debt decisions [4].

The closest reviewed approaches address different parts of this integration problem. De Almeida et al. incorporate business criteria into technical-debt prioritization [27], while Martini and Bosch's AnaConDebt considers consequences, timing, architecture, and stakeholder reasoning in architectural technical-debt decisions [22]. Liu et al. show that code-level prioritization can instead be automated using labelled code-smell severity data [21]. These approaches establish the relevance of contextual criteria and explicit prioritization, but they operate on different decision units or

require evidence that was not available in the present industrial case. The gap addressed here is therefore not the absence of contextual or automated prioritization in general. Among the approaches reviewed, none directly combines file-level maintainability and development-history evidence with module-level operational context in one deterministic, bounded pre-refactoring procedure that also separates near-term Refactor-Now candidates from a preparation-oriented Stabilize-First queue. The thesis addresses this specific integration gap while keeping the resulting assumptions, classifications, and planning decisions inspectable.

Table 2.1 summarizes this positioning.

Table 2.1: Positioning of the thesis relative to related work

Related-work area	**What prior work establishes**	**Relationship to this thesis**
Technical-debt prioritization	Limited resources require prioritization; multiple criteria may matter [5, 19, 20].	Provides the multi-dimensional prioritization foundation.
Business-aware TD decisions	Contextual value can affect debt-management choices [27]; explicit decision rules can represent such choices [4].	Motivates explicit operational/domain context rather than technical scoring alone.
Refactoring research	Refactoring research addresses transformation mechanics [14, 23] and the preceding identification of refactoring opportunities [3].	Thesis focuses specifically on pre-refactoring file selection rather than transformation mechanics.
Maintainability and Code Health	Structural quality is relevant to maintenance outcomes [25, 30]; CodeScene provides the Code Health concepts used as the maintainability signal [7, 8].	Code Health supplies the maintainability signal and HealthOnly baseline.
Change metrics and Behavioural Code Analysis	Development history provides evidence complementary to static code properties [9, 24, 29].	Activity supplies the development-interaction signal and contributes to HealthActivity.

Continued on next page

Table 2.1 – continued from previous page

Related-work area	What prior work establishes	Relationship to this thesis
Architecture and context-aware debt	Architectural debt decisions can involve consequences, timing, architecture, and stakeholder reasoning [22]; business context can also influence prioritization [27].	Motivates the use of module-level operational context in file-level planning and the explicit treatment of contextual judgments as reviewable rather than objective measurements.
Learned code-smell prioritization	Ranking can be automated using labelled severity data and learning-to-rank methods [21].	Provides an alternative prioritization paradigm; the present artifact instead uses deterministic rules because labelled ground truth was unavailable.
Telecom/5G context	3GPP defines the broader 5G NR technical context [2]. Security and privacy concerns are documented in ENISA and peer-reviewed survey literature [12, 13, 18], while 3GPP TS 33.501 specifies the security architecture and procedures for the 5G System [1].	Motivates evaluating operational context, without treating general 5G literature as file-level criticality evidence.

The study therefore occupies the intersection between technical-debt prioritization, behavioural software analytics, context-aware engineering decisions, and industrial refactoring planning.

The next chapter operationalizes this gap through the Design Science Research process used to construct and evaluate the artifact.

Chapter 3

Method

3.1 Chapter Overview

This chapter presents the research design used to construct and evaluate the domain-aware refactoring-prioritization artifact. The study follows the Design Science Research Methodology (DSRM) proposed by Peffers et al. [26], together with broader Design Science Research principles [15, 16] and software-engineering-oriented design science guidance [31].

The empirical study was conducted in a selected Ericsson UPC uplink software scope, with the source-code file as the primary unit of analysis. The Method describes how candidate prioritization factors were assessed, how the retained evidence was operationalized into a deterministic scoring and planning artifact, and how module-level operational context was connected to file-level prioritization.

The chapter further defines candidate construction, technical transformations, artifact parameters, Refactor-Now and Stabilize-First planning procedures, domain-agnostic baseline strategies, quantitative evaluation metrics, and the two-endpoint temporal comparison. It also describes the practitioner evaluation and the separate expert assessment of the domain-criticality construct.

The chapter concludes with the procedures used to support reproducibility, traceability, confidentiality, validity, and reliability.

3.2 Research-Question Traceability

The research aim and research questions are defined in Section 1.3. Table 3.1 maps each research question to the main evidence and analysis used to address it and to the corresponding Results sections. For RQ2, the expert assessment is treated as construct-validity evidence for the domain-criticality construct rather than as direct evidence of planning usefulness.

3.3 Research Design and Method Selection

3.3.1 Design Science Research Methodology

We use Design Science Research Methodology (DSRM) as the primary research approach because the central research output is an artifact, which is derived from an explicit and executable procedure that transforms technical and domain evidence

Table 3.1: Traceability between research questions, evidence, and results

Research question	Focus	Main evidence and analysis	Results
RQ1	Relevant and feasible prioritization factors	Prior research and industrial relevance, followed by population-wide feasibility and traceability assessment.	Section 4.2
RQ2	Usefulness of the finalized artifact	Baseline comparison, quantitative planning metrics, temporal sensitivity, and practitioner evaluation; independent domain assessment as a construct-validity check.	Sections 4.3–4.8

into bounded refactoring-planning recommendations. Peffers et al. describe DSRM as a sequence of problem motivation and identification, definition of solution objectives, design and development, demonstration, evaluation, and communication [26]. This methodological structure fits the thesis because the work progresses from an industrial prioritization problem to the construction, demonstration, and evaluation of a deterministic decision-support artifact.

Moreover, the broader Design Science Research principles described by Hevner et al. support this choice [15]. Design science studies create and evaluate artifacts meant to address relevant problems while contributing knowledge about their design and use. In this thesis, the artifact is not intended as a production-ready refactoring tool, but a proof-of-concept method whose input tables, parameters, eligibility rules, transformations, and outputs are preserved, so that the computational analysis can be inspected and re-executed.

The study also takes place within a bounded industrial case. We therefore use the software-engineering case-study guidance of Runeson and Höst to structure the description of context, unit of analysis, data sources, human participants, confidentiality, and validity [28]. DSRM provides the comprehensive research process, whereas case-study guidance supports disciplined execution and reporting in the Ericsson setting.

The study follows the six DSRM activities as one coherent research process. The prioritization problem and the limitations of purely technical indicators motivate the work. The solution objective is a transparent method that combines technically observable refactoring relevance with operational context. Design and development generate the scoring and planning artifact. Demonstration applies the artifact to the selected software population. Evaluation compares its output with baseline strategies, applies it to the January and April software endpoints, obtains practitioner feedback, and independently assesses the domain-criticality construct. The thesis communicates the resulting method, evidence, limitations, and implications.

3.3.2 Rigor, Relevance, and Iterative Artifact Development

We use Hevner's three-cycle view of Design Science Research to distinguish the main sources of design decisions [16].

The relevance cycle connects the artifact to the industrial environment. The main

constraints include limited refactoring capacity, the operational sensitivity of telecom software, the availability of CodeScene and Git data, architecture and high-level design (HLD) documentation, and the requirement that recommendations remain accessible to engineers.

The rigor cycle connects the design to established knowledge. Research on maintainability, technical debt, refactoring, software evolution, and behavioural code analysis motivates the technical signals considered in the artifact. Architecture and technical-debt literature motivates the inclusion of contextual information when focusing on engineering work. DSRM and empirical software-engineering research inform how we construct and evaluate the artifact.

The design cycle consists of repeated build-evaluate-refine activities. The final method did not emerge in one step. Figure 3.1 summarizes how the six DSRM activities, the relevance and rigor cycles, and the iterative artifact development relate to the study. We identify five principal artifact design and evaluation iterations within one DSRM study.

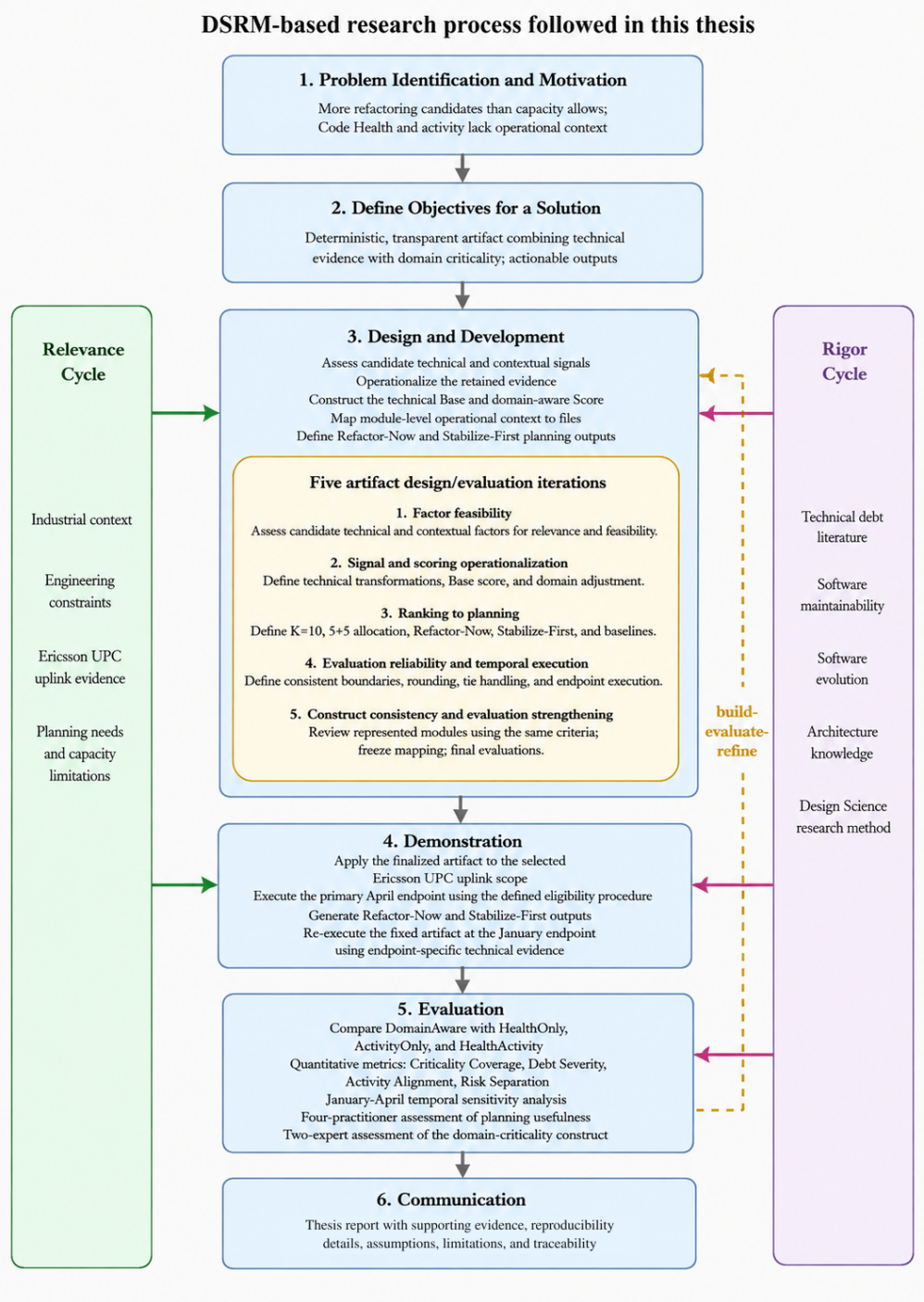


Figure 3.1: DSRM-based research process followed in this thesis

Table 3.2: Main artifact design and evaluation iterations

Iteration	Objective	Activity performed	Evidence used
1. Factor feasibility	Determine which candidate prioritization factors could be supported in the industrial study.	Identify candidate technical and contextual factors and assess each for conceptual relevance, data availability, traceability, and consistent operationalization across the study scope.	Prior research, CodeScene availability, Git history, architecture/HLD material, and available industrial engineering evidence.
2. Signal and scoring operationalization	Define comparable and interpretable representations of the retained evidence.	Specify transformations for the technical signals, construct the technical Base, define the representation of domain criticality, and select a transparent method for combining technical and contextual evidence.	Prior research, properties of the available measures, architecture/HLD evidence, and inspection of artifact behaviour during development.
3. Ranking to planning	Convert file ranking into a bounded engineering-planning artifact.	Define the planning budget, Code Health band allocation, Refactor-Now and Stabilize-First outputs, low-activity threshold procedure, baseline strategies, and evaluation metrics.	Industrial planning requirements, engineering-capacity considerations, and artifact-design objectives.

Continued on next page

4. Evaluation reliability and temporal execution	Ensure deterministic execution and support evaluation using two software endpoints.	Define and apply consistent Code Health boundaries, numerical calculation order, tie handling, endpoint-specific technical measurements, and the procedure for comparing January and April selections.	Repository states, CodeScene measurements, Git history, and reproducibility requirements identified during artifact execution.
5. Construct consistency and evaluation strengthening	Strengthen consistency of the researcher-created domain construct and separate artifact evaluation from construct assessment.	Review the represented-module classifications using the same domain criteria, freeze the researcher mapping before the final artifact analyses and human evaluations, then conduct the practitioner assessment of planning usefulness and the separate expert assessment of domain criticality.	Architecture/HLD evidence and additional Ericsson-internal operational information concerning release status and fallback behaviour for the consistency review; practitioner-evaluation and expert-assessment material only after the mapping freeze.

The five iterations in Table 3.2 should not be interpreted as five complete repetitions of the six DSRM activities. They are the principal build-evaluate-refine iterations within the design cycle of one DSRM study. The relevance and rigor cycles repeatedly informed those iterations. Industrial constraints and available Ericsson evidence supplied relevance-cycle input, while prior research on maintainability, technical debt, software evolution, architecture, and Design Science supplied rigor-cycle input. The design cycle then translated those inputs into artifact changes and evaluated the resulting behaviour.

Iteration 5 also established an important sequencing rule. Additional Ericsson-internal operational information concerning module release status and fallback behaviour prompted a consistency review. We applied that review across the complete represented-module set using the same Service Impact, Operational Criticality, and Workaround Availability considerations. The resulting researcher mapping was frozen before the final artifact analyses and human evaluations. Subsequent evaluation evidence was therefore not used to modify the frozen labels or other artifact parameters.

3.3.3 Alternative Research Designs Considered

We considered alternative research designs before selecting DSRM.

A purely descriptive case study or interview study could have provided rich information about how engineers currently reason about technical debt and refactoring. Such a design would have been appropriate for studying existing decision processes, but it would not by itself construct and demonstrate the deterministic prioritization artifact required by the thesis aim.

A controlled or longitudinal refactoring experiment would provide stronger causal evidence. For example, files selected by different strategies could be refactored and compared using subsequent Code Health, defect, productivity, or maintenance outcomes. This approach was not feasible within the study period because it would require comparable interventions, behavioural verification, integration into normal development work, and a sufficiently long observation period. More importantly, such a study would answer a different question: what happens after refactoring rather than how targets should be prioritized in advance.

Incident- or defect-based validation was also considered. Such a design could compare prioritization outputs against historical operational failures. However, sufficiently complete and reproducible incident-to-file mappings were not available for the full candidate population.

Another alternative was a learned ranking model. Training such a model would require a sufficiently large historical set of labelled prioritization or refactoring decisions. No such ground-truth dataset existed for the selected scope. The deterministic approach also allowed the study to specify and inspect each decision rule directly.

The selected design prioritizes transparent artifact construction and bounded pre-refactoring evaluation rather than causal assessment of post-refactoring outcomes. This better matches the research objective, the available data, and the requirement for inspectable engineering reasoning.

3.4 Industrial Context, Scope, and Unit of Analysis

3.4.1 Industrial Context and Scope

We conducted the study with Ericsson in the selected User Plane Control (UPC) uplink software scope introduced in Section 1.1. The scope forms part of Ericsson's broader 5G Radio Access Network (RAN) baseband software environment and evolves through feature development, maintenance, and operational adaptation.

The broader public 5G NR uplink context is discussed in Section 2.7. The specific UPC uplink scope examined in this thesis is proprietary and represents only a bounded part of Ericsson's wider software environment. The description provided in the public thesis is therefore limited to the context necessary to define the empirical study and does not disclose internal UPC architecture or imply that the selected scope implements every function defined by the broader 3GPP specifications.

The study intentionally examines one bounded software scope rather than the complete Ericsson codebase. Candidate files are identified through the scope, change-window, and artifact-type rules defined in Section 3.5.2. The architecture catalogue

provides the corresponding module context, while only modules associated with files entering the candidate population participate in the file-level prioritization analysis.

Modules for which no file satisfies the candidate-construction procedure remain part of the architecture context but do not contribute files to the prioritization population. Their absence from the prioritization population is therefore determined by the candidate-construction procedure rather than by their domain-criticality classification or by researcher preference. The resulting file and module populations are reported in Section 4.2.1.

3.4.2 Units of Analysis

The source-code file is the primary unit of analysis and the unit returned by the prioritization artifact. Code Health and development activity are available at file level, and the practical planning question concerns which files should receive near-term refactoring consideration.

Domain criticality is classified at module level. Architecture and high-level design (HLD) documentation primarily describe module responsibilities, runtime roles, failure consequences, and fallback behaviour rather than the operational importance of individual files. We therefore use the deterministic relationship:

$$\text{module-level domain assessment} \rightarrow \text{module-to-file mapping} \rightarrow \text{file-level prioritization}$$

The module structure and file-to-module relationships were part of the existing Ericsson software and architecture context. Each candidate file therefore retained its established module association. After a domain-criticality label was assigned to a module, all candidate files associated with that module inherited the same label for the purposes of the artifact.

Only files satisfying the Code Health eligibility procedure use the inherited domain label in the scoring and planning calculations. The artifact does not apply general file-specific domain-criticality overrides. This abstraction improves consistency because it avoids unsupported file-by-file judgments. It can, however, conceal internal variation within heterogeneous modules. We treat that limitation explicitly in the validity analysis and examine it further through the independent expert assessment.

3.5 Data Sources and Candidate Construction

3.5.1 Data Sources

The artifact combines three principal evidence sources: version-control history, CodeScene Code Health, and architecture/high-level design (HLD) information.

Git history provides the candidate change history and the development-activity signal. CodeScene provides the file-level maintainability indicator used in the artifact. Architecture and HLD documents provide information about module responsibilities, runtime roles, possible service consequences, and workaround or fallback behaviour.

The evaluation introduces two additional human evidence sources with different purposes. Four practitioners assess the usefulness of the finalized planning outputs. Two domain experts independently assess the researcher-created domain-criticality construct. We separate these evaluation tasks so that preference for a generated strategy is not treated as evidence that the underlying domain labels are objectively correct.

Table 3.3: Data sources and roles

Source	**Unit**	**Role**
Git repository history	File	Candidate construction and non-merge development activity.
CodeScene Code Health	File	Maintainability signal and eligibility bands.
Architecture/HLD documentation	Module	Domain-criticality construction and module-to-file interpretation.
Practitioner workbook	Final planning outputs	Perceived usefulness, actionability, scheduling realism, operational alignment, and Stabilize-First review.
Independent expert workbooks	Represented modules	Independent assessment of the researcher-created domain-criticality construct.

The empirical material analyzed in this thesis consists primarily of software-engineering and architecture data of the modules rather than live radio-traffic measurements. The computational dataset contains source-file identities from the selected UPC uplink scope, file-level CodeScene Code Health values, non-merge Git change counts, module-to-file mappings, and module-level responsibility, runtime, failure-consequence, and fallback information derived from architecture and HLD documentation.

The human-evaluation material consists of practitioner rankings, ratings, and qualitative comments on the finalized planning outputs, together with two independent experts' module-level domain-criticality assessments. The prioritization analysis does not use packet captures, live radio samples, or customer-specific traffic measurements. Proprietary identifiers and detailed architecture information remain within the Ericsson-controlled research material, while the thesis reports the supporting evidence using anonymized identifiers and sanitized descriptions where required.

3.5.2 Candidate Construction

We constructed the candidate population through explicit scope, activity-window, and artifact-type rules rather than by manually selecting files based on their later prioritization characteristics.

Candidate construction first identified target source files within the selected UPC uplink scope that were changed during the primary analysis window. Generated code,

vendor or externally maintained artifacts, documentation, and other unsupported artifact types were excluded. Candidate files that were no longer present at the primary April endpoint were removed from the endpoint population. The remaining file identities formed the master population used for module mapping, supporting data preparation, and cross-endpoint traceability.

Code Health determines eligibility for automatic scoring. For candidate construction, files are treated as scoreable when their numeric Code Health falls in either the LOW band ($CH < 4$) or the MODERATE band ($4 \leq CH < 9$). Files with $CH \geq 9$ are classified as `OUT_OF_SCOPE` for the prioritization artifact because they are treated as sufficiently healthy under the study's eligibility definition and therefore do not enter automatic scoring. The detailed Code Health transformation and the rationale for these boundaries are described in Section 3.6.2.

Files without a valid numeric Code Health value are retained in the master population with the data-quality status `NEEDS_REVIEW`. Because the automatic prioritization procedure requires a numeric Code Health value, these files do not enter the scoreable population or the subsequent automatic scoring and planning steps. They remain in the study inventory for traceability. The detailed scoring and planning procedure is described in Section 3.6.

The Method therefore distinguishes between the master population used for traceability and the endpoint-specific scoreable population used for automatic prioritization. The resulting population sizes and eligibility distributions are reported in Section 4.2.1.

3.5.3 Analysis Windows

The primary analysis covers:

21 October 2025 to 21 April 2026

For this period, development activity represents the accumulated non-merge file-changing commits. Code Health represents the maintainability state of the analyzed software at the April endpoint.

The temporal evaluation uses the earlier January endpoint, covering:

21 October 2025 to 21 January 2026

For this endpoint, we computed point-in-time Code Health for the January repository state using the CodeScene command-line interface (CLI) and calculated non-merge development activity over the corresponding October-to-January interval. Code Health and Activity were therefore measured for the software state and observation window associated with each endpoint, while the prioritization rules remained fixed.

The temporal comparison therefore uses endpoint-specific Code Health and Activity data at both January and April rather than reusing April measurements and shortening only the Activity window. For cross-endpoint comparability, the January analysis uses the same master identity set established for the primary April analysis. In other words, the January analysis asks how the same April-defined file cohort

would have been prioritized using January-specific Code Health, Activity, and file availability while keeping the artifact definitions and frozen domain mapping unchanged; it is not a separately constructed January candidate population. Files from the master cohort that were not present at the January endpoint were treated as unavailable and did not enter January scoring. This design maintains consistent file identities across the two evaluated endpoints while allowing the technical inputs to reflect each endpoint.

3.6 Artifact Design and Development

3.6.1 Factor Feasibility

The first artifact-design iteration assessed candidate prioritization factors against both conceptual relevance and industrial feasibility.

Candidate factors were identified from the literature reviewed in Chapter 2 and from the evidence available in the industrial environment. The assessed factors included maintainability condition, development activity, domain or architectural context, change or temporal coupling, incident information, and file-level test-coverage information.

For each candidate factor, we considered two questions. First, the factor had to represent information that could reasonably contribute to pre-refactoring target selection. Second, the required evidence had to be available with sufficient coverage, traceability, and consistency to support systematic use across the intended study population.

Feasibility was assessed using these criteria and qualitative judgement, rather than a fixed numerical threshold for data completeness. We checked whether the available evidence could be linked consistently to candidate files or their modules and used with the same definition across the study population.

A factor was retained for artifact construction only when both conditions were satisfied. Conceptually relevant factors for which the available evidence was incomplete, inconsistently available, or insufficiently traceable were excluded from population-wide scoring rather than being estimated or selectively applied to only part of the candidate population.

The resulting feasibility outcomes and the factors retained for artifact construction are reported in Section 4.2.1.

3.6.2 Technical Signal Operationalization

Code Health Bands

We use Code Health both as a continuous maintainability-related signal and as an eligibility mechanism. Let CH_i denote the numeric Code Health value of file i. The Code Health eligibility bands follow the CodeScene Code Health categories used in the Ericsson study environment [7, 8]. CodeScene reports Code Health on a 1–10 scale, with lower values indicating stronger maintainability concern [7, 8]. We therefore use the following boundaries in the artifact:

$$\text{CHBand}_i = \begin{cases} \text{LOW}, & \text{CH}_i < 4, \\ \text{MODERATE}, & 4 \leq CH_i < 9, \\ \text{OUT_OF_SCOPE}, & \text{CH}_i \geq 9. \end{cases} \tag{3.1}$$

Files without a valid numeric Code Health value are classified as `NEEDS_REVIEW`. Only LOW and MODERATE files enter automatic scoring. Files with Code Health greater than or equal to 9 are classified as `OUT_OF_SCOPE` because they are already considered healthy according to the Code Health classification used in the study and therefore do not represent immediate refactoring candidates for this prioritization artifact. `OUT_OF_SCOPE` and `NEEDS_REVIEW` files are excluded according to the eligibility rules defined in Section 3.5.2.

The purpose of this constraint is to prevent activity or domain importance alone from causing technically healthy files to displace files with observable maintainability concern.

The Code Health boundaries are treated as eligibility definitions rather than tunable ranking parameters in this study. If the artifact were transferred to a context using different maintainability categories or a different maintainability measure, the eligibility boundaries would need to be defined for that context before applying the scoring procedure.

HealthRisk

Code Health and prioritization concern have opposite directions: higher Code Health means healthier code, whereas a larger maintainability-concern value should represent stronger maintainability concern. We therefore define:

$$\text{HealthRisk}_i = \frac{10 - CH_i}{9}. \tag{3.2}$$

The constants in this transformation follow directly from the 1-10 Code Health scale. Subtracting from 10 reverses the direction of the measure, while division by the scale width of 9 maps CH = 1 to HealthRisk = 1 and CH = 10 to HealthRisk = 0. These constants therefore arise from the source scale and are not tunable artifact parameters.

This transformation reverses the direction of Code Health and maps the 1–10 Code Health scale to the interval $[0, 1]$, where larger values represent stronger maintainability concern. It is a linear proxy for maintainability concern and does not imply that equal differences in Code Health correspond to equal differences in actual maintenance difficulty, refactoring benefit, or engineering effort.

Development Activity

For file i, we define:

$$\text{Activity}_i = \text{NonMergeCommitCount}_i. \tag{3.3}$$

Activity therefore represents the number of non-merge commits that changed the file during the corresponding analysis window. The intention is to count the original file-changing development commits rather than treating later branch-integration

events as additional development activity. In a standard Git merge, commits made on a development branch remain part of the repository history after that branch is integrated; counting both those commits and the subsequent merge commit can therefore represent the same development episode more than once. Merge commits can still contain integration-specific changes, for example conflict resolutions, so their exclusion does not assume that merge commits never modify code. Instead, it is an operational choice to make Activity represent direct file-level development interactions while reducing workflow-related inflation from branch integration. The Activity values obtained for each analysis endpoint are preserved as study inputs and used consistently in the subsequent transformation, scoring, and ranking procedures.

Raw development activity is strongly skewed. We therefore transform it as:

$$\text{ActivityLog}_i = \ln\left(1 + \text{Activity}_i\right). \tag{3.4}$$

The addition of 1 keeps the logarithmic transformation defined when $\text{Activity}_i = 0$ and maps zero observed activity to $\text{ActivityLog}_i = 0$. It is therefore part of the transformation rather than a tunable weighting parameter.

We then normalize it using the maximum transformed activity across all files in the master population:

$$\text{ActivityNorm}_i = \frac{\text{ActivityLog}_i}{\max\limits_{j \in \mathcal{M}} \ln\left(1 + \text{Activity}_j\right)}. \tag{3.5}$$

where $\mathcal{M}$ denotes the master population.

ActivityNorm lies between 0 and 1. A value closer to 1 indicates that a file has relatively high recent development activity compared with the maximum observed activity in the master population for that analysis window, while a value closer to 0 indicates relatively little recent development interaction. ActivityNorm therefore measures relative development activity, not business importance, operational criticality, developer effort, defect probability, or refactoring need.

The logarithmic transformation reduces the influence of extremely active files while preserving their ordering. The normalization denominator is calculated using the complete master population to provide one consistent activity reference for the software scope. Only files in the endpoint-specific scoreable population subsequently use ActivityNorm in Base and ranking calculations.

3.6.3 Domain Criticality Construction and Consistency Review

Repository-derived measures do not directly express the operational importance of a software module. We therefore created a qualitative domain-criticality construct using Ericsson architecture and HLD evidence.

Because the artifact uses categorical labels for two different constructs, *LOW Code Health* and *MODERATE Code Health* refer to maintainability-related eligibility bands, whereas *HIGH*, *MEDIUM*, and *LOW domain criticality* refer to the operational classification assigned at module level.

The module structure and the corresponding file-to-module relationships already existed within the Ericsson software and architecture context. We did not create

the represented modules or determine a new file-to-module mapping for the study. Instead, each candidate file retained its established module association.

During the factor-feasibility assessment, we first proposed candidate dimensions for representing operational context and discussed their evidence availability with Ericsson. Ericsson reviewed the proposed dimensions against the information available in the relevant architecture and HLD material. Service Impact, Operational Criticality, and Workaround Availability were the three proposed dimensions for which the required industrial evidence was available across the represented modules. We therefore fixed these three dimensions as the evidence basis for the domain-criticality assessment and did not include proposed dimensions for which comparable evidence was unavailable. We then interpreted these three dimensions jointly to assign an initial HIGH, MEDIUM, or LOW classification to each represented module.

We assessed each represented module using three considerations:

1. **Service Impact**: the likely consequence for service behaviour if the module fails or degrades;
2. **Operational Criticality**: the importance of the module's functionality to live system operation; and
3. **Workaround Availability**: whether a practical fallback, degraded mode, or workaround exists.

We interpreted these considerations jointly. They do not form an automatic Boolean decision tree. The assigned module label is an ordinal judgment based on the combined operational evidence.

The available labels are:

- **HIGH**: strong operational consequence or importance;
- **MEDIUM**: significant but more localized, degraded, mixed, or partly recoverable consequence; and
- **LOW**: limited direct service consequence or primarily supporting functionality.

We reviewed the available Ericsson architecture and HLD information together for each represented module. We considered the three domain dimensions jointly and agreed on one HIGH, MEDIUM, or LOW classification for each module. The resulting domain-criticality labels were therefore researcher-created labels. Ericsson provided the underlying module structure, file associations, and architecture/HLD evidence, but did not assign the HIGH, MEDIUM, or LOW labels used in the artifact.

For use in the scoring artifact, we encode the labels as:

$$\text{DomainWeight}_i = \begin{cases} 1.0, & \text{DomainLabel}_i = \text{HIGH}, \\ 0.5, & \text{DomainLabel}_i = \text{MEDIUM}, \\ 0.2, & \text{DomainLabel}_i = \text{LOW}. \end{cases} \tag{3.6}$$

The values provide a transparent ordinal encoding of the three domain-criticality levels. HIGH is used as the reference level at 1.0, MEDIUM receives an intermediate

weight of 0.5, and LOW receives a smaller but non-zero weight of 0.2. The non-zero LOW value allows low-criticality modules to retain a limited domain adjustment rather than treating LOW criticality as the absence of operational relevance. The weights preserve the intended ordering HIGH > MEDIUM > LOW and were fixed before the final evaluation.

The relative spacing between the three weights controls how strongly the domain levels are differentiated in the scoring artifact. Increasing the numerical distance between HIGH, MEDIUM, and LOW increases the score difference attributable to domain classification for files with similar technical Base values. Reducing the distance makes the three domain levels behave more similarly. If the same ordinal amplification interpretation is retained in another application, the weights should remain non-negative and preserve the ordering HIGH > MEDIUM > LOW.

These numerical values are artifact-design parameters. They do not represent probabilities, measured service risk, or Ericsson-prescribed severity values.

After a module received a domain-criticality classification, candidate files associated with that module inherited the same label through the established Ericsson module-to-file relationship. The inherited label was used in the scoring artifact only for files satisfying the automatic-scoring eligibility procedure. Domain labels associated with `OUT_OF_SCOPE` or `NEEDS_REVIEW` files therefore did not influence *Refactor-Now* or *Stabilize-First* selection.

Classification consistency review: During Iteration 5, before the final evaluations, we reviewed the researcher-created domain classifications for consistency. At this stage, additional Ericsson-internal operational information concerning the release status and fallback behaviour of some modules became available. This information was relevant because release status and fallback behaviour can affect the interpretation of Service Impact, Operational Criticality, and Workaround Availability. We therefore checked whether the existing module classifications remained consistent with the same three domain-criticality criteria.

To avoid reconsidering only the modules that initially prompted the review, we applied the consistency check across the complete represented-module set. Where the available evidence supported reconsideration, we jointly reassessed the affected classification and documented the resulting decision. After completing the review, we froze the researcher-created domain mapping.

The frozen mapping was then used in the final January and April artifact analyses and the associated quantitative and temporal evaluations. The practitioner evaluation reviewed planning outputs generated using this frozen mapping, while the independent experts assessed the modules without seeing the researcher labels; their classifications were subsequently compared with the frozen researcher mapping. Evidence obtained from these evaluations was not used to modify the domain labels or other artifact parameters.

The classification outcomes of the consistency review are reported in Section 4.2.2.

3.6.4 Technical Base Score

After transforming Code Health and activity into comparable directions and ranges, we combine them into a technical composite:

$$\text{Base}_i = 0.5\,\text{HealthRisk}_i + 0.5\,\text{ActivityNorm}_i. \tag{3.7}$$

We assign equal coefficients to HealthRisk and ActivityNorm in the technical Base. The available study evidence did not justify assigning a larger coefficient to either signal, so equal weighting provides a simple and transparent starting configuration.

The two coefficients control the relative influence of maintainability concern and recent development interaction in the technical Base. For alternative configurations, the weights should remain non-negative and sum to one, $w_{\text{H}} + w_{\text{A}} = 1$, if Base is to retain its interpretation as a weighted average on the interval $[0, 1]$. Increasing w_{H} while reducing w_{A} places greater emphasis on HealthRisk, whereas increasing w_{A} gives recent development activity greater influence on candidate ordering. Alternative weightings may change ranking order and selection membership and should therefore be evaluated through sensitivity analysis rather than assumed to be preferable.

The Base score deliberately contains no domain information. This separation serves two purposes. First, it makes the technical part of the artifact independently inspectable. Second, it allows the HealthActivity strategy to act as a direct domain-agnostic baseline when evaluating the effect of introducing domain context.

3.6.5 Domain-Aware Score

We introduce domain criticality through a multiplicative adjustment:

$$\text{DomainMultiplier}_i = 1 + \beta\,\text{DomainWeight}_i. \tag{3.8}$$

The constant 1 makes the technical Base the multiplier's reference point. When $\beta = 0$, the multiplier equals 1, and the score therefore reduces to the technical Base. With non-negative β and non-negative domain weights, the domain term can preserve or amplify the technical score rather than reduce it.

The amplification parameter is fixed as:

$$\beta = 0.5. \tag{3.9}$$

The resulting multipliers are therefore:

$$\text{DomainMultiplier}_i = \begin{cases} 1.50, & \text{DomainLabel}_i = \text{HIGH}, \\ 1.25, & \text{DomainLabel}_i = \text{MEDIUM}, \\ 1.10, & \text{DomainLabel}_i = \text{LOW}. \end{cases} \tag{3.10}$$

The final domain-aware score for file i is:

$$\text{DomainAwareScore}_i = \text{Base}_i \times \text{DomainMultiplier}_i. \tag{3.11}$$

DomainWeight and DomainMultiplier serve different purposes. DomainWeight numerically encodes the ordinal domain label, while DomainMultiplier determines how that label modifies the technical Base. With $\beta = 0.5$, the resulting multipliers are 1.50 for HIGH, 1.25 for MEDIUM, and 1.10 for LOW, corresponding to proportional uplifts of 50%, 25%, and 10%, respectively. Thus, for two files with the same

technical Base, domain criticality changes their relative priority while the technical Base remains the foundation of the score.

A multiplicative formulation was selected instead of an additive one because the design objective was to let domain context strengthen an existing technical refactoring case, not to create a separate source of priority. In an additive formulation, a term such as $\lambda\,\mathrm{DomainWeight}_i$ would contribute a fixed amount independently of HealthRisk and ActivityNorm. A file with a weak technical Base could therefore move ahead primarily because of its domain label.

With the multiplicative formulation:

$$\mathrm{DomainAwareScore}_i = \mathrm{Base}_i\,(1 + \beta\,\mathrm{DomainWeight}_i)\,. \tag{3.12}$$

The multiplicative formulation was selected because it matches the artifact's design principle that technical evidence should establish the refactoring case and domain evidence should modify its priority.

Interpretation of the amplification strength: The value $\beta = 0.5$ was selected during exploratory artifact development as a moderate domain-adjustment setting and was fixed before the final evaluations. The design intention was to give domain criticality enough influence to differentiate technically similar candidates near ranking boundaries while retaining the technical Base as the primary foundation of the score. The parameter therefore controls the strength of this contextual adjustment. A value of $\beta = 0$ would remove domain amplification and reduce the ranking component to the technical Base, whereas progressively larger values would give domain criticality greater influence over candidate ordering.

Conversely, decreasing β moves the DomainAwareScore closer to the technical Base. The Stabilize-First exclusion remains unchanged. To preserve the intended interpretation of domain criticality as an amplification rather than a penalty, β should be non-negative. The study does not define an empirically justified upper bound; larger values would simply make differences in domain classification increasingly influential relative to differences in the technical Base.

For illustration, $\beta = 0.25$ produces multipliers of 1.25, 1.125, and 1.05 for HIGH, MEDIUM, and LOW, respectively, whereas $\beta = 0.75$ produces 1.75, 1.375, and 1.15. For a HIGH-criticality file with $\mathit{Base} = 0.40$, the resulting DomainAwareScore would therefore be 0.50 for $\beta = 0.25$, 0.60 for the evaluated $\beta = 0.5$, and 0.70 for $\beta = 0.75$; with $\beta = 0$, the score would remain 0.40. Thus, increasing β progressively increases the influence of domain criticality while leaving the underlying technical Base unchanged. We use $\beta = 0.5$ as the fixed configuration for the evaluated artifact. Systematic sensitivity analysis of this parameter is considered in future work.

Consequently, if $\mathrm{Base}_i = 0$, then $\mathrm{DomainAwareScore}_i = 0$ regardless of the domain label. The domain factor can amplify a technical case but cannot independently create one. This property also preserves interpretability because the HealthActivity Base remains directly observable and DomainAware can be understood as a controlled contextual adjustment to that Base.

Numerical Calculation Convention

The implementation derives HealthRisk and ActivityLog directly from their respective raw inputs. ActivityNorm is then calculated from ActivityLog, and Base combines HealthRisk and ActivityNorm. DomainMultiplier is calculated from the domain label and β, after which DomainAwareScore combines Base and DomainMultiplier.

To reproduce the preserved values, HealthRisk and ActivityLog are rounded to four decimal places using Python's `round()` behaviour. ActivityNorm uses the rounded ActivityLog numerator and the full-precision maximum logarithmic activity as its denominator. The resulting ActivityNorm is then rounded to four decimal places. Base, DomainMultiplier, and DomainAwareScore are likewise rounded before subsequent use. Evaluation metrics use the stored rounded values. This convention defines numerical execution rather than the relative importance of the factors.

3.6.6 From Ranking to Planning

A complete ranking of all scoreable files does not directly represent a bounded engineering plan. The third artifact iteration therefore converts the ranking into two planning queues.

Within the final planning procedure, HIGH domain criticality can have two effects: it increases the priority of technically relevant files that remain eligible for Refactor-Now, while HIGH-criticality files with relatively low recent activity are instead routed to Stabilize-First for additional preparation and review.

Selection Budget and Band Allocation

The planning budget and band allocation are defined as:

$$K = K_{\text{LOW}} + K_{\text{MODERATE}} = 5 + 5 = 10. \tag{3.13}$$

The planning budget was set to $K = 10$ to keep the recommendation small enough for focused engineering review and practitioner evaluation while still representing multiple candidates from both eligible Code Health bands. The value therefore acts as an explicit planning-budget assumption for the evaluated artifact.

K controls the size of the immediate Refactor-Now planning set. A smaller value produces a more selective shortlist concentrated near the top of each ranking, whereas a larger value broadens the planning set and includes candidates located further down the ranked lists. Increasing K can therefore expose differences between strategies that are not visible within a smaller Top-K boundary. In another application, K should preferably reflect the amount of work that engineers can realistically review or consider within the relevant planning period. Where no such capacity estimate is available, the value should be treated as an explicit planning assumption.

The 5+5 split ensures that both stronger and moderate maintainability concerns remain represented in the recommendation. Without a fixed allocation, one Code Health band could dominate the final list.

In an alternative configuration, K must be a positive integer, while K_{LOW} and K_{MODERATE} must be non-negative integers satisfying $K_{\text{LOW}} + K_{\text{MODERATE}} = K$ and must not exceed the number of eligible files available in their respective bands.

K_{LOW} and K_{MODERATE} control the representation of the two eligible maintainability bands within the total budget K. Increasing K_{LOW} relative to K_{MODERATE} places greater planning emphasis on files with stronger Code Health concern. Increasing K_{MODERATE} instead gives more positions to files with moderate maintainability concern. Neither the total budget nor the band allocation is Ericsson-prescribed.

Stabilize-First

The Stabilize-First rule does not infer engineering risk from low activity. Instead, low recent activity is used as a conservative planning signal: when a technically relevant file belongs to a HIGH-criticality module but has received relatively little recent development interaction, the artifact routes it to a preparation and review queue before immediate refactoring consideration. This distinguishes these files from HIGH-criticality candidates that are already being changed more actively. Because candidate construction begins from files changed during the primary six-month analysis window, Stabilize-First identifies relatively low-activity files within that changed-file candidate population. It is not intended to identify dormant or untouched HIGH-criticality files outside the study population.

To determine what counts as relatively low activity, we use the 25th percentile of the raw Activity values among the LOW and MODERATE Code Health files that are eligible for scoring. We denote this threshold by $P25$. The number 25 refers to the 25th percentile; it does not mean 25 commits. $P25$ is the Activity value located near the lower quarter of the scoreable Activity distribution.

We use the 25th percentile as a transparent relative threshold for identifying the lower-activity portion of the scoreable population. A relative percentile avoids imposing the same absolute commit-count threshold across analysis windows with different durations and development intensities.

The percentile controls how selective the low-activity condition is. Using a lower percentile would define low activity more narrowly and would generally make fewer files eligible for the Stabilize-First activity condition. Using a higher percentile would include a larger portion of the Activity distribution and would generally make more files eligible for that condition. The final number of Stabilize-First candidates also depends on how many of those files belong to HIGH-criticality modules and on ties in the observed Activity values, so changing the percentile does not imply a fixed proportional change in queue size.

We calculate $P25$ separately at each endpoint because the number of commits observed depends on the length and development intensity of the corresponding Activity window. Using an endpoint-specific percentile therefore defines low activity relative to the software state being analyzed rather than imposing the same fixed commit threshold on both windows.

Let n be the number of scoreable LOW and MODERATE files at an endpoint. Their raw Activity counts are sorted in ascending order. We calculate the zero-based index:

$$r = \lfloor 0.25(n-1) \rfloor . \tag{3.14}$$

The percentile level (25%) is the planning parameter, whereas the floor operation is a deterministic calculation convention used to make the threshold reproducible. A replication using another percentile should likewise specify and keep one percentile calculation convention fixed before execution.

The threshold is then:

$$P25 = A_{(r)}, \tag{3.15}$$

where $A_{(r)}$ denotes the Activity value at zero-based position r in the sorted Activity sequence.

A scoreable file enters Stabilize-First only when both conditions are satisfied:

$$i \in \text{StabilizeFirst} \iff (\text{DomainLabel}_i = \text{HIGH}) \wedge (\text{Activity}_i \leq P25) . \tag{3.16}$$

The threshold is recalculated independently at each endpoint using the same percentile procedure. Consequently, the numerical Activity threshold may differ between endpoints even though the Stabilize-First rule itself remains unchanged.

For files assigned to this queue, preparation may include test strengthening, observability, ownership clarification, behavioural guardrails, or deeper engineering review before structural change. The DomainAware method removes these candidates from immediate Refactor-Now eligibility.

Refactor-Now

For the remaining eligible DomainAware population, we rank LOW files and MODERATE files separately by DomainAwareScore in descending order and select the highest five from each band.

The result is a ten-file Refactor-Now recommendation together with a separate Stabilize-First preparation queue.

The complete artifact flow is summarized in the following pipeline. Code Health and development activity form the technical Base, while module-level domain criticality enters separately through the multiplicative domain adjustment. The planning stage then routes HIGH-criticality, low-activity candidates to Stabilize-First before selecting the five highest-ranked LOW and five highest-ranked MODERATE candidates for Refactor-Now. The equations and parameter provenance are specified in Sections 3.6.2-3.6.6.

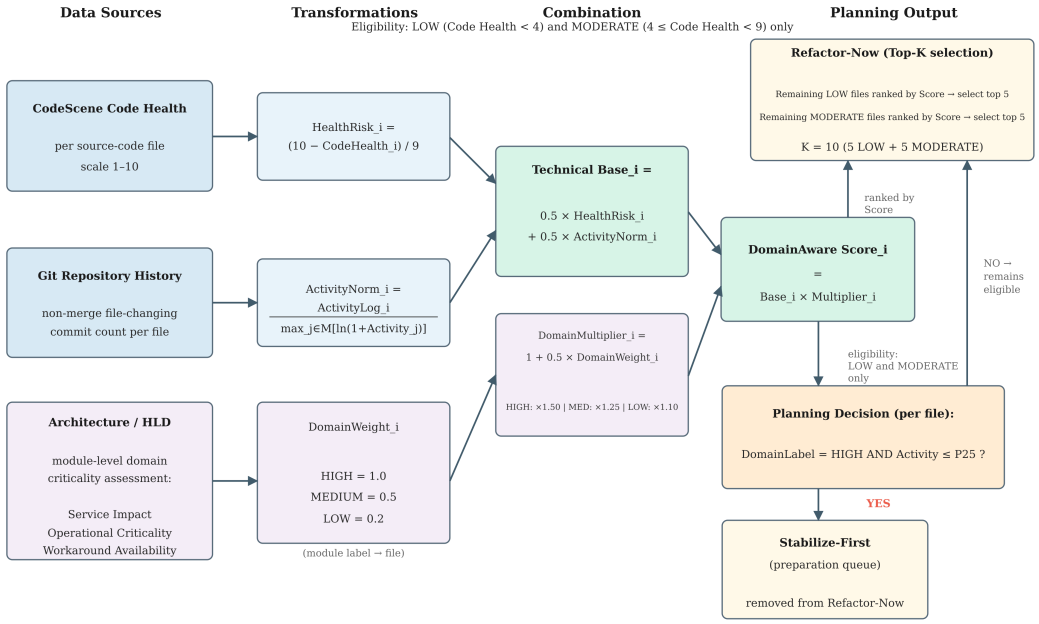


Figure 3.2: Domain-aware prioritization pipeline

Table 3.4: Final artifact parameters and provenance

Item	Final choice	Purpose	Provenance
Base weights	0.5 / 0.5	Relative emphasis on HealthRisk and ActivityNorm	Neutral researcher design
Domain weights	H = 1.0, M = 0.5, L = 0.2	Differentiate the three domain levels	Researcher design
β	0.5	Control domain-amplification strength	Researcher design
K	10	Control Refactor-Now set size	Planning design choice
Band allocation	5 LOW + 5 MODERATE	Control representation of both Code Health bands	Planning design choice
P25	Floor-based 25th percentile of scoreable Activity	Define relative low-activity condition	Planning design choice
Tie rule	File_ID ascending	Deterministic exact-tie handling	Reproducibility convention

Continued on next page

Item	Final choice	Purpose	Provenance
Rounding	Four decimals for stored derived values; full-precision activity-normalization denominator	Deterministic numerical execution	Reproducibility convention

The configuration in Table 3.4 was fixed before the final evaluations and was not adjusted in response to practitioner ratings or expert judgments. Sections 3.6.2–3.6.6 document the rationale and effect of each design choice, while systematic parameter-sensitivity analysis is identified as future work.

3.7 Demonstration and Quantitative Evaluation

3.7.1 Baseline Strategies

We compare DomainAware with three domain-agnostic strategies.

Within each eligible Code Health band, HealthOnly ranks files in descending order of HealthRisk, ActivityOnly ranks files in descending order of ActivityNorm, and HealthActivity ranks files in descending order of Base. Each strategy then selects the five highest-ranked LOW files and the five highest-ranked MODERATE files, producing the common Top-10 budget.

The three baseline strategies do not apply the Stabilize-First exclusion. This asymmetry is intentional because Stabilize-First is part of the DomainAware planning artifact rather than a property of the underlying technical ranking. It also means that any evaluation involving overlap with Stabilize-First must be interpreted as partly evaluating an explicit design feature.

The ranking signal for each strategy is always the primary ordering criterion. A secondary rule is required only when two or more files have exactly the same primary ranking value and that tie affects the boundary of a Top-5 band selection. In such cases, the tied files are ordered by ascending anonymized `File_ID`.

`File_ID` was selected deliberately because it is a stable, non-semantic identifier. Using Code Health, Activity, DomainWeight, Base, Stabilize-First status, or practitioner judgement as a secondary criterion would introduce additional decision information into a strategy that was not part of its formal definition. For example, resolving an ActivityOnly tie using Code Health would turn the ActivityOnly baseline into a partly health-aware strategy. The `File_ID` rule therefore ensures deterministic repeatability without changing the conceptual meaning of any strategy.

The rule affects only exact boundary ties and is treated as a reproducibility convention rather than a tunable prioritization parameter.

3.7.2 Evaluation Metrics

The quantitative evaluation does not use a single notion of "best" because the artifact deliberately combines several planning considerations. We therefore selected four descriptive metrics that correspond directly to the three prioritization inputs and the additional planning-separation mechanism: domain representation, maintainability concern, development activity, and Stabilize-First separation. The metrics characterize the composition of each Top-K set. Hence, they are not accuracy measures against a known ground-truth ranking.

Let S_K denote the Top-K set selected by a strategy.

Criticality Coverage

$$\text{CriticalityCoverage@}K = \frac{1}{K} \sum_{i \in S_K} \text{DomainWeight}_i. \tag{3.17}$$

CriticalityCoverage@K is the mean DomainWeight of the selected files. It describes how strongly the selected set aligns with the researcher-created domain-criticality mapping. Values closer to 1 indicate greater representation of HIGH-criticality modules.

This metric is useful for comparing whether introducing domain information changes the operational profile of the selected set. However, because the same researcher-created domain construct contributes to DomainAware scoring, Criticality Coverage is not independent evidence that the selected files are objectively more operationally important. The independent expert assessment in Section 3.10 evaluates this construct separately.

Debt Severity

$$\text{DebtSeverity@}K = \frac{1}{K} \sum_{i \in S_K} \text{HealthRisk}_i. \tag{3.18}$$

DebtSeverity@K is the mean HealthRisk of the selected files. Since HealthRisk lies between 0 and 1, larger values indicate that the selected set contains files with stronger maintainability concern according to Code Health. This metric makes it possible to determine whether a strategy gains alignment with another dimension by moving away from files with more severe maintainability concerns.

Activity Alignment

$$\text{ActivityAlignment@}K = \frac{1}{K} \sum_{i \in S_K} \text{ActivityNorm}_i. \tag{3.19}$$

ActivityAlignment@K is the mean ActivityNorm of the selected files and therefore lies in the interval $[0, 1]$. Values closer to 1 indicate that the selected set is more strongly aligned with files receiving relatively high recent development activity, whereas values closer to 0 indicate weaker alignment with recent development activity. This metric is useful because one objective of incorporating repository history is to avoid treating maintainability condition in isolation. However, a higher

ActivityAlignment value is not inherently "better" in an overall sense. It only indicates stronger alignment with the activity dimension. As with ActivityNorm itself, ActivityAlignment does not measure business importance, operational criticality, developer effort, or refactoring benefit.

Risk Separation

$$\text{RiskSeparation@}K = |S_K \cap \text{StabilizeFirst}| \,. \tag{3.20}$$

RiskSeparation@K counts how many selected files also satisfy the Stabilize-First rule. Unlike the other three metrics, this is a count rather than a normalized average. It exposes whether a strategy places HIGH-criticality, relatively low-activity files directly into its Top-K recommendation.

For DomainAware, Risk Separation is zero by construction because Stabilize-First candidates are removed before Refactor-Now selection. The metric must therefore be understood as a description of planning-policy separation, rather than independent evidence of engineering safety.

The four metrics are reported separately because they represent different planning characteristics and can conflict with one another. No empirical basis exists in this study for assigning a second set of weights to combine them into one overall performance score. Reporting them separately makes the trade-offs between strategies visible and avoids implying that one aggregate value represents universally superior prioritization.

3.8 Temporal Evaluation

The temporal evaluation examines whether the list of each strategy's Top-10 recommendations changes when the same artifact is applied to the fixed master cohort using technical evidence from two different endpoints: January and April. Endpoint-specific Code Health, file availability, and development activity are used, while the artifact definitions and master file identities remain fixed. The purpose is not to predict future behaviour, but to determine how sensitive the observed Top-10 recommendations are to changes in time-dependent evidence within the study period.

Let $S_{s,\text{Jan}}$ denote the Top-10 set produced by strategy s at the January endpoint, and let $S_{s,\text{Apr}}$ denote the corresponding Top-10 set at the April endpoint.

We first report the number of files common to both selections:

$$\text{Overlap}_s = |S_{s,\text{Jan}} \cap S_{s,\text{Apr}}| \,. \tag{3.21}$$

Raw overlap is intuitive but does not express similarity relative to the total number of unique files appearing across the two sets. We therefore also use the Jaccard similarity coefficient:

$$J_s = \frac{|S_{s,\text{Jan}} \cap S_{s,\text{Apr}}|}{|S_{s,\text{Jan}} \cup S_{s,\text{Apr}}|}. \tag{3.22}$$

Jaccard is a set-similarity measure ranging from 0 to 1. A value of 0 indicates that the two Top-10 sets contain no common files, whereas a value of 1 indicates

identical set membership. It is suitable here because the temporal question concerns whether the same files remain in the Top-10 recommendation. It does not require the files to retain the same rank positions. Jaccard therefore complements the raw overlap count with a normalized measure of set similarity. Thus, it does not measure similarity in ranking order within the Top-10 set.

For the January analysis, Code Health is measured for the January repository state and Activity is calculated from 21 October 2025 to the January endpoint. For the April analysis, the corresponding April Code Health and six-month Activity values are used.

The artifact definitions themselves remain fixed: Code Health bands, transformations, technical weights, domain mapping, domain weights, β, K, the 5+5 allocation, Stabilize-First rule, baseline definitions, tie rule, and rounding convention. Only evidence that naturally changes with the software state or observation window is allowed to vary.

We describe this procedure as a two-endpoint temporal sensitivity analysis because the artifact is executed independently at two observed software endpoints while its rules are held constant, and the resulting selections are compared. The analysis therefore answers a narrow question: did the recommendations change between these two endpoints, and by how much?

The evaluation is therefore limited to sensitivity between the two observed endpoints. The corresponding temporal-validity limitations are discussed in Section 3.12.5.

3.9 Practitioner Evaluation

3.9.1 Participants and Evaluation Material

We evaluated the finalized planning artifact with four Ericsson 5G baseband software practitioners familiar with the selected UPC uplink context. We used purposive sampling because meaningful assessment required direct knowledge of the relevant implementation and operational environment.

All four participants worked as 5G baseband software developers and reported relevant familiarity with the selected UPC uplink context and refactoring or code-review activities. Two participants explicitly reported approximately four years of UPC uplink experience. All four practitioners recorded consent for their responses to be used in the thesis analysis. Also, all four practitioners reviewed the finalized April endpoint artifact.

The four prioritization strategies were anonymized as:

Strategy A, Strategy B, Strategy C, and Strategy D.

The mapping between these labels and the underlying algorithms was hidden from the participants during evaluation. Internal file paths remained visible. This was necessary because practitioners needed to recognize the implementation being assessed. The evaluation was therefore strategy-anonymized rather than file-anonymized.

The material did not display:

- Code Health values or bands;

- Activity values;
- DomainLabel or DomainWeight;
- HealthRisk;
- ActivityNorm;
- Base;
- DomainAwareScore; or
- the mapping from A-D to the underlying strategies.

The review material contained the four finalized ten-file Refactor-Now lists and the complete April endpoint Stabilize-First list.

3.9.2 Procedure and Analysis

Each practitioner completed the same structured workbook.

First, participants ranked Strategies A-D from 1 = most suitable to 4 = least suitable.

Second, they rated each strategy on a five-point agreement scale (1 = strongly disagree; 5 = strongly agree) across five dimensions:

1. whether the strategy selected reasonable refactoring candidates;
2. whether it avoided candidates considered inappropriate for immediate refactoring;
3. whether the output was actionable for practical refactoring planning;
4. whether the selection appeared realistic with respect to scheduling; and
5. whether the strategy aligned with mission or operational importance.

Third, the workbook collected qualitative explanations for the rankings and ratings, including observations about inappropriate candidates, missing important files or modules, and operational considerations affecting the recommendations.

Finally, practitioners reviewed the complete Stabilize-First list qualitatively and could identify files they believed should instead be refactored directly or suggest movement between Refactor-Now and Stabilize-First. The Stabilize-First evaluation did not use a per-file numerical 1-5 scale. We therefore analyze this part as qualitative evidence without assigning a numerical Stabilize-First score.

For each strategy, we calculate:

- the number of first-place votes;
- the mean rank;
- the mean rating for each of the five dimensions; and

- the overall mean rating across dimensions.

Because the practitioner evaluation involved only four purposively selected participants, we use descriptive rather than inferential statistics. The participants were chosen because of their relevant knowledge of the studied software context, not to form a statistically representative sample of a larger practitioner population. Hence, we report the observed rankings and ratings using counts and mean values, without applying significance tests or making population-level statistical claims.

We group qualitative comments into recurring factual themes while preserving individual exceptions. Where a participant's responses contain internal tension between different workbook fields, we preserve the explicit qualitative statement rather than imposing a numerical interpretation not directly supported by the response. The practitioner evaluation measures perceived planning usefulness and engineering relevance. It does not establish that a selected file is objectively the correct refactoring target, validate the underlying domain labels, or demonstrate post-refactoring safety.

3.10 Independent Domain-Criticality Validation

The DomainAware score and Criticality Coverage metric depend on researcher-created domain labels. To evaluate this construct independently from strategy preference, we conducted a separate domain-criticality assessment with two Ericsson experts familiar with the selected UPC uplink software.

The validators differed in experience with the studied context. One reported approximately one year of RAN software and UPC experience with medium familiarity, whereas the other reported approximately four years of UPC experience and high familiarity with the selected scope. This contextual information is retained when interpreting the expert assessments and agreement results.

The final researcher mapping was frozen before the experts conducted their assessments. The experts independently assessed all represented modules. Their instrument contained module identifiers and contextual material describing responsibilities, runtime behaviour, failure consequences, workaround or fallback context, and relevant architecture/HLD information.

The expert-assessment instrument did not display:

- researcher-assigned domain labels or numerical weights;
- Code Health;
- Activity;
- HealthRisk;
- ActivityNorm;
- Base;
- DomainAwareScore;

- Refactor-Now selections;
- baseline outputs; or
- practitioner evaluation results.

For each module, the instrument collected assessments of:

- Service Impact;
- Operational Criticality;
- Workaround Availability;
- Final Domain Label;
- confidence;
- explanatory reasoning; and
- corrections or additional contextual information.

The permitted final responses were:

HIGH, MEDIUM, LOW, and INSUFFICIENT_INFORMATION.

We preserved both experts' raw responses. We did not convert `INSUFFICIENT_INFORMATION` into a domain label.

The analysis consists of five stages.

First, we summarize completion and the number of modules for which each expert reported `INSUFFICIENT_INFORMATION`.

Second, for modules where both experts supplied HIGH, MEDIUM, or LOW, we calculate exact raw agreement and linearly weighted Cohen's kappa [10]. Raw agreement reports the proportion of modules for which the two experts assigned the same label. Cohen's kappa is an inter-rater agreement statistic that measures how strongly two assessors agree while accounting for the level of agreement that could occur by chance. A kappa value of 1 indicates perfect agreement, whereas a value of 0 indicates agreement no better than chance.

We use the weighted form because the domain labels are ordinal. A disagreement between HIGH and MEDIUM is smaller than a disagreement between HIGH and LOW. Linear weighting reflects this ordering by assigning a smaller penalty to one-level disagreements than to two-level disagreements.

Third, we compare each expert independently with the frozen researcher classification using exact raw agreement, confusion matrices, and linearly weighted Cohen's kappa. Modules marked `INSUFFICIENT_INFORMATION` by an expert remain outside that expert's ordinal agreement calculation. `INSUFFICIENT_INFORMATION` is treated as non-ordinal and excluded from weighted-kappa calculations rather than being treated as a fourth ordered category.

Fourth, we examine three-way agreement among the frozen researcher classification and the two independent expert assessments. We report the number and

proportion of represented modules for which all three assessments assign the same HIGH, MEDIUM, or LOW label.

Because the prioritization artifact propagates module-level domain labels to files through the established Ericsson module-to-file relationship, we also derive the corresponding agreement after propagation to the master file population. For each assessment, the assigned module label is propagated to the files mapped to that module, after which agreement between the resulting file-level labels is calculated. This analysis represents the consequence of module-level agreement for the file-level artifact. It is not treated as an independent file-level expert assessment because the experts classified modules rather than individual files.

Fifth, we examine the qualitative reasoning associated with disagreements, including differences in perceived service impact, fallback behaviour, module heterogeneity, and available information.

The two expert validators were also members of the practitioner pool. The tasks, however, served different purposes. The practitioner exercise evaluated finalized strategy outputs, whereas the domain-validation exercise assessed module criticality under blinding from the researcher labels and scoring results. We account for the partial participant overlap as a validity limitation rather than treating the two samples as statistically independent populations. After the two experts completed their assessments, we did not create a new combined or consensus domain mapping from their responses. We therefore kept the researcher-created mapping that had already been frozen before the expert assessment. The expert results are used only to evaluate how much the experts agreed with each other and with the frozen researcher mapping. They were not used to change the domain labels or to rerun the primary DomainAware analysis.

3.11 Reproducibility, Traceability, and Confidentiality

We implement the prioritization artifact as a deterministic analysis pipeline and preserve the inputs, configuration, intermediate values, and outputs required to audit the reported results. The preserved evidence includes the master population, the endpoint-specific scoreable populations, Code Health and Activity inputs, module-to-file and domain mappings, scoring parameters, strategy outputs, metric traces, and tie-handling records.

The implementation fixes the Code Health bands, HealthRisk transformation, Activity definition, logarithmic transformation and normalization, technical and domain weights, β, selection budget, 5+5 band allocation, P25 procedure, Stabilize-First rule, baseline definitions, tie-breaking rule, and four-decimal calculation convention.

Given the preserved study inputs, the resulting Base scores, DomainAware scores, strategy rankings, Top-10 selections, and quantitative metrics can therefore be reproduced deterministically. The January and April analyses preserve their corresponding Code Health and Activity inputs separately while applying the same fixed artifact configuration. This allows the two endpoint selections and their overlap calculations to be reproduced from the stored state-specific inputs.

Appendix A contains the supporting evidence for the reported findings, including endpoint inputs, module mappings, artifact configuration, final selections, evaluation summaries, individual practitioner rankings and ratings, and expert classifications. Together with the documented calculation rules, these records support reconstruction of the computational results and numerical human-evaluation summaries. Proprietary file paths, detailed architecture descriptions, and other confidential source material remain within the Ericsson-controlled study environment.

3.12 Validity and Reliability

3.12.1 Construct Validity

The artifact operationalizes three concepts through imperfect indicators.

Code Health is a maintainability-related indicator, not a direct measurement of future refactoring benefit. The linear HealthRisk transformation preserves the ordering of Code Health but assumes a simple linear interpretation for scoring purposes.

Development Activity counts non-merge file-changing commits. It measures the frequency of development interaction but does not distinguish feature development, defect fixing, refactoring, or the amount of work contained in each commit. It should therefore not be interpreted as direct engineering effort, defect probability, business importance, or refactoring benefit.

Domain criticality is the most judgment-dependent construct. The researchers created the original labels from architecture and HLD evidence and propagated them from modules to files through the established module-to-file mapping. Because the same researcher-created construct contributes to both DomainAware scoring and Criticality Coverage, Criticality Coverage is not an independent validation metric for domain criticality.

We address this risk by freezing the researcher mapping before the independent two-expert assessment, allowing `INSUFFICIENT_INFORMATION` as a valid expert response rather than forcing an ordinal judgment, preserving the experts' raw assessments, and reporting both agreement and disagreement. The expert comparisons provide independent evidence about the consistency of the researcher-created domain classification, but they do not establish the frozen mapping as objective ground truth.

Module-level classification can also conceal file-level heterogeneity. Architecture and HLD evidence primarily supports judgments about module responsibilities and operational consequences, whereas individual files within a module may implement different features or responsibilities. The module-level abstraction improves consistency and avoids unsupported file-by-file classification, but it does not eliminate within-module variation. The propagated file-level agreement used in the expert-validation analysis requires the same qualification. The experts assessed modules rather than independently classifying individual files. File-level agreement is therefore derived by propagating each module classification through the established module-to-file mapping. Modules containing more mapped files consequently contribute more observations to the derived percentage. The propagated agreement therefore describes the effect of module-level classification agreement on the file-level

artifact. It is not independent evidence of file-level domain-criticality validity.

3.12.2 Internal Validity

Several scoring and planning parameters in Table 3.4 are researcher-designed rather than empirically estimated. Their rationale and expected effect are made explicit in the Method, and the complete configuration was fixed before the final evaluations. Different parameter configurations may produce different rankings, which is therefore treated as an internal-validity consideration and a target for future sensitivity analysis.

We reduce outcome-driven tuning by documenting the provenance of these decisions, applying the domain consistency review across the complete represented-module population, freezing the resulting researcher mapping before the final artifact analyses and human evaluations, and keeping the artifact definitions fixed across the January and April analyses.

The independent expert assessment was conducted after the researcher mapping had been frozen. Expert agreement or disagreement was therefore not used to retune the domain labels, numerical domain weights, β, selection budget, Stabilize-First rule, or any other artifact parameter. This separation reduces the risk that the final artifact configuration was adjusted retrospectively to match the validation results.

The baseline comparison is intentionally asymmetric because only DomainAware includes the Stabilize-First exclusion. Consequently, RiskSeparation partly evaluates the operation of an explicit artifact mechanism rather than a symmetric ranking property. We therefore interpret the metric as descriptive evidence of planning separation rather than independent evidence of safety.

The domain-classification consistency review also introduces a potential circularity threat because additional Ericsson-internal operational information concerning module release status and fallback behaviour prompted reconsideration of the researcher-created mapping before the final evaluation. To mitigate this risk, we applied the reassessment across the complete represented-module set using the same Service Impact, Operational Criticality, and Workaround Availability considerations, rather than revising only the modules that initially prompted the review. We then froze the resulting researcher mapping before the final artifact analyses and human evaluations. Evidence obtained from the subsequent quantitative, temporal, practitioner, and expert evaluations was not used to modify the frozen domain labels or other artifact parameters.

3.12.3 Reliability

Reliability is supported by preserving the endpoint-specific Code Health and Activity inputs used in the executed study and applying the same documented computational rules to those inputs. The artifact fixes the transformations, four-decimal calculation convention, selection budgets, deterministic tie handling, frozen mappings, and strategy definitions required to re-execute the analysis. The January analysis uses its own repository presence, Code Health, and Activity information rather than April measurements. Files that are absent and files without valid numeric Code Health values are handled according to Sections 3.5.2–3.5.3 rather than through imputation.

Human judgments are inherently less reproducible than deterministic scoring. Procedural variation was reduced by giving all four practitioners the same finalized workbook and both domain validators the same structured module instrument, while preserving both experts' raw responses rather than replacing them with a retrospective consensus.

3.12.4 External Validity and Transferability

The study examines one bounded Ericsson UPC uplink scope using the candidate population defined in this chapter. The resulting numerical findings are therefore context-specific and cannot be statistically generalized to all Ericsson systems, telecom software, or other domains. The artifact structure is more transferable than the observed numerical results. Applying it elsewhere would require comparable file-level maintainability information, version-control history, a defensible mapping between domain units and source files, and sufficient domain knowledge to assess operational criticality. Researcher-designed parameters should also be reconsidered for the target context rather than assumed to be universally appropriate.

3.12.5 Human-Evaluation and Temporal Limitations

The practitioner evaluation involved four purposively selected participants with relevant local knowledge. Their familiarity with the selected software enabled context-sensitive assessment of the recommendations, but their judgments may also reflect individual experience, familiarity with particular files or features, and current engineering priorities. The responses are therefore interpreted as contextual professional assessments rather than as statistically representative evidence or an objective ground-truth ranking. The exercise evaluated recommendations rather than implemented refactoring outcomes, so favourable ratings do not demonstrate subsequent maintainability or operational improvement.

The independent expert assessment involved only two validators, both of whom also participated in the practitioner pool. Blinding and separation of the two tasks reduced direct dependence between the exercises but did not make the participant samples statistically independent.

The temporal evaluation compared only the January and April endpoints using the fixed master cohort established from the primary April analysis. This supports file-identity comparability but means that the analysis measures within-cohort sensitivity between two observed endpoints rather than independently sampled populations. The observed similarities and differences therefore do not establish general temporal stability, long-term robustness, or invariance under future development conditions.

Taken together, these validity boundaries define the scope of the evaluation. The study examines the artifact's transparency, reproducibility, and feasibility; differences from the baselines and between the two endpoints; and the practitioners' assessments of planning usefulness. It does not establish causal post-refactoring improvement or universally optimal prioritization.

Chapter 4

Results and Analysis

4.1 Chapter Overview

This chapter reports the empirical results obtained from the artifact and evaluation procedures defined in Chapter 3. Section 4.2 summarizes the April analysis population and the final factor-feasibility outcome. Sections 4.3 and 4.4 present the April Refactor-Now and Stabilize-First results. Sections 4.5 and 4.6 report the January endpoint results and compare the January and April selections. Sections 4.7 and 4.8 present the practitioner evaluation and independent domain-criticality assessment, respectively. Broader interpretation of these findings and the answers to the research questions are presented in Chapter 5.

4.2 Study Population and Artifact-Development Results

4.2.1 Population and Factor-Feasibility Results

Applying the population and eligibility procedures defined in Chapter 3 produced an April master population of 200 files across 31 architecture modules. Numeric Code Health was available for 196 files (98%); 136 files (68% of the full population) fell in the LOW or MODERATE bands and were eligible for automatic prioritization. The remaining 60 numeric scores were outside those bands, while four files without numeric Code Health required review. Table 4.1 reports the endpoint population.

Applying the factor-feasibility procedure defined in Section 3.6.1 retained Code Health, development activity, and module-level domain criticality as the three artifact inputs. Code Health was supported by available file-level CodeScene measurements, development activity by traceable non-merge Git history, and domain criticality by the available architecture/HLD evidence and module-to-file relationships.

Change or temporal coupling was not retained because a sufficiently complete and reproducible population-wide source was unavailable. Incident information could not be mapped consistently to candidate files, and file-level test-coverage information was not consistently available across the intended population. These were qualitative feasibility judgments based on the evidence available in the study environment. The factors were excluded because they could not be operationalized consistently across the population, while their potential relevance to prioritization was acknowledged.

Table 4.1: April endpoint study population

Property	Result
Initial change-scan files	202
Files removed before endpoint	2
Final master population	200
Architecture catalogue modules	50
Represented modules	31
LOW Code Health	26
MODERATE Code Health	110
OUT_OF_SCOPE	60
NEEDS_REVIEW	4
Catalogue-only modules without candidate files	19
Scoreable LOW + MODERATE files	136
Scoreable HIGH-domain files	109
Scoreable MEDIUM-domain files	24
Scoreable LOW-domain files	3
Maximum six-month Activity	48

The corresponding endpoint-population and eligibility evidence is provided in Appendix A.5.

4.2.2 Domain-Classification Consistency-Review Results

The population-wide consistency review produced the frozen researcher mapping used in the final artifact analyses and evaluations. The resulting represented-module distribution was 11 HIGH, 12 MEDIUM, and 8 LOW.

During Iteration 5, before the final evaluations, additional Ericsson-internal operational information concerning module release status and fallback behaviour prompted additional scrutiny of selected classifications. To avoid reconsidering only those modules, the consistency review was applied across all represented modules using the same Service Impact, Operational Criticality, and Workaround Availability considerations.

M04 remained classified HIGH. The available evidence described released functionality with relevance to active customer throughput, supporting a HIGH classification despite the existence of fallback behaviour.

M10 was classified MEDIUM because the module contained heterogeneous responsibilities, including both operationally important paths and configuration-time or supporting functionality. The available evidence therefore did not support a uniformly HIGH consequence for the module as a whole.

M18 was classified MEDIUM. The available operational evidence indicated degraded throughput rather than service loss, continued operation without the Coordinated Multi-Point (CoMP) benefit, an available fallback, and that the feature was not released to customers during the evaluated period. The module was not classified LOW because it still participated in architecturally significant behaviour.

After completing this consistency review, the researcher mapping was frozen before the final artifact analyses and human evaluations.

4.3 April Endpoint Prioritization Results

4.3.1 Final Refactor-Now Selections

The primary analysis used the period from 21 October 2025 to 21 April 2026. Each strategy selected ten files: 5 from the LOW Code Health band and 5 from the MODERATE band.

Table 4.2 reports the final April endpoint selections using the anonymized file identifiers.

Table 4.2: Final April endpoint Top-10 selections

Strategy	LOW-band selections	MODERATE-band selections
DomainAware	F185, F174, F110, F103, F170	F032, F156, F071, F113, F096
HealthOnly	F178, F185, F103, F174, F182	F113, F076, F026, F032, F179
ActivityOnly	F185, F174, F110, F103, F127	F032, F156, F090, F096, F097
HealthActivity	F185, F174, F110, F103, F170	F032, F156, F071, F033, F034

Here, LOW and MODERATE in the selection columns refer to the Code Health eligibility bands. References to HIGH, MEDIUM, or LOW criticality instead denote the frozen researcher-assigned module-level domain-criticality labels inherited by the selected files. DomainAware selected ten files mapped to HIGH-criticality modules. ActivityOnly also selected ten HIGH-criticality files, despite not using domain criticality as a ranking input. HealthOnly selected nine HIGH and one MEDIUM file, whereas HealthActivity selected eight HIGH and two MEDIUM files.

The complete anonymized Refactor-Now selections for the evaluated endpoints are reported in Appendix A.6.

4.3.2 Quantitative Planning Metrics

Figure 4.1 summarizes the three continuous quantitative planning metrics for the four strategies. Risk Separation is reported separately in the text because it is a count rather than a normalized metric.

No strategy was highest on all three continuous metrics. HealthOnly produced the highest Debt Severity (0.7944) but the lowest Activity Alignment (0.6518), whereas ActivityOnly produced the highest Activity Alignment (0.8389) but the lowest Debt Severity (0.7103); its Criticality Coverage was nevertheless 1.0000 because all ten selected files belonged to HIGH-labelled modules. DomainAware produced Criticality Coverage 1.0000, Debt Severity 0.7583, and Activity Alignment 0.8191, compared with 0.9000, 0.7661, and 0.8146 for HealthActivity. Compared with HealthActivity, DomainAware therefore had a +0.1000 difference in Criticality Coverage, a -0.0078 difference in Debt Severity, and a +0.0045 difference in Activity Alignment.

Risk Separation was zero for DomainAware, ActivityOnly, and HealthActivity, whereas HealthOnly selected one file that also satisfied the Stabilize-First rule. DomainAware's zero value follows from the planning procedure because Stabilize-First candidates are removed before Refactor-Now selection. The complete endpoint metric and strategy-overlap evidence is provided in Appendix A.7.

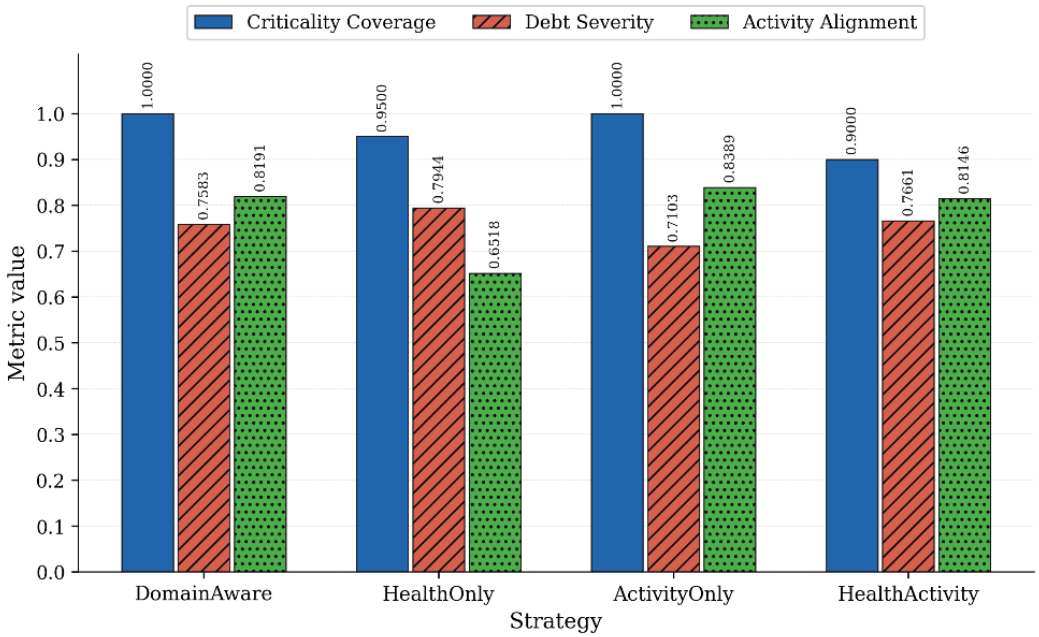


Figure 4.1: April endpoint quantitative planning-metric comparison

4.3.3 Selection Overlap and DomainAware-HealthActivity Differences

Pairwise overlap shows how much the strategies agreed on the identities of the selected files. Table 4.3 reports both the number of common Top-10 files and Jaccard similarity.

Table 4.3: Pairwise overlap among April endpoint Top-10 selections

Strategy pair	**Common files**	**Jaccard**
DomainAware – HealthActivity	8	0.6667
DomainAware – ActivityOnly	7	0.5385
DomainAware – HealthOnly	5	0.3333
HealthActivity – ActivityOnly	6	0.4286
HealthActivity – HealthOnly	4	0.2500
ActivityOnly – HealthOnly	4	0.2500

Figure 4.2 visualizes the eight selections shared by DomainAware and HealthActivity and the two strategy-specific MODERATE-band selections at the April endpoint.

The largest overlap involving DomainAware occurred with HealthActivity: 8 of the 10 selected files were common to both lists.

Within the LOW Code Health band ($CH < 4$), DomainAware and HealthActivity selected the same five files. Within the MODERATE Code Health band ($4 \leq CH < 9$), their first three selected files were also identical: F032, F156, and F071. The strategies therefore differed only at the final two MODERATE-band positions.

HealthActivity selected F033 and F034, both mapped to MEDIUM-criticality

modules. DomainAware instead selected F113 and F096, both mapped to HIGH-criticality modules. The remaining eight files were common to the two strategies.

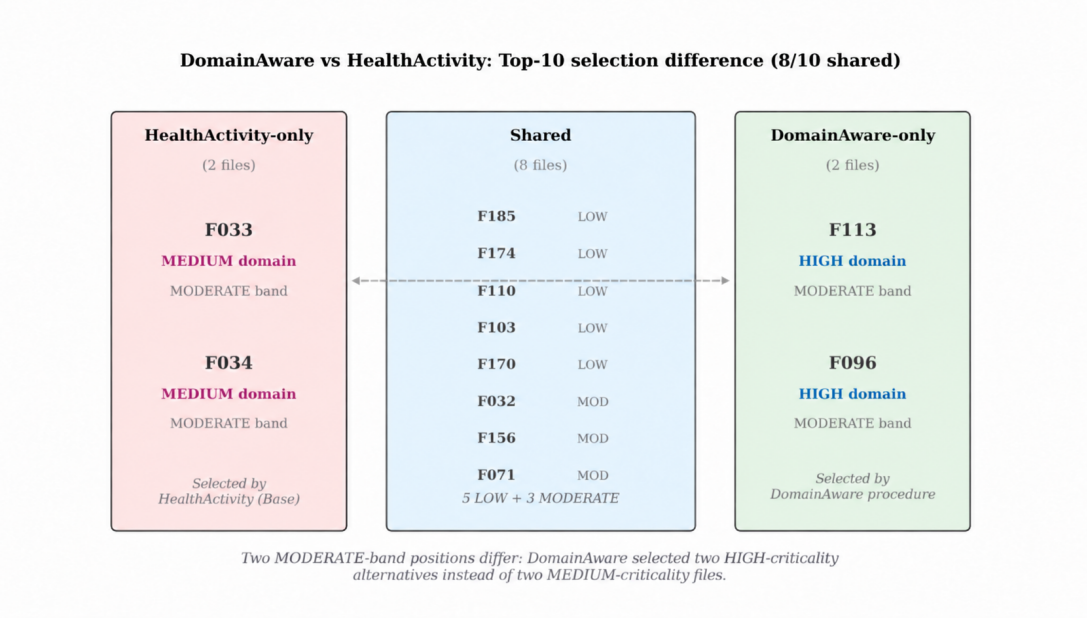


Figure 4.2: DomainAware and HealthActivity differences in the April endpoint Top-10 selection

The two-file substitution increased Criticality Coverage from 0.9000 for HealthActivity to 1.0000 for DomainAware under the frozen researcher mapping.

4.4 Stabilize-First Results

At the April endpoint, the percentile procedure defined in Section 3.6.6 produced the zero-based index $r = 33$. The corresponding 34th ordered Activity value was 2 commits, giving an April $P25$ Activity threshold of 2.

Under the Stabilize-First rule, a scoreable file entered the preparation queue only when its inherited researcher domain label was HIGH, and its raw Activity value was less than or equal to this threshold. In this analysis, Activity ≤ 2 therefore means that the file was changed by at most two non-merge commits during the period from 21 October 2025 to 21 April 2026.

Applying these conditions to the 136 scoreable files identified 28 Stabilize-First candidates, corresponding to approximately 20.6% of the scoreable population.

These files were retained as part of the planning output but removed from immediate DomainAware Refactor-Now eligibility. The rule therefore separates two types of technically relevant HIGH-criticality candidates: files that remain eligible for immediate ranking and files that are routed to a preparation/review queue because they combine HIGH criticality with relatively low recent development activity.

The baseline strategies did not apply this exclusion. In the April endpoint Top-10 selections, HealthOnly selected one file that also belonged to the final 28-file Stabilize-First set. ActivityOnly and HealthActivity did not select a Stabilize-First file in their respective Top-10 lists.

The numerical result itself does not determine what action should be taken on each of the 28 files. The practitioner review reported in Section 4.7 provides separate evidence about whether engineers considered the resulting preparation queue reasonable.

The complete April and January Stabilize-First memberships are reported in Appendix A.6.

4.5 January Endpoint Results

The second application of the artifact used the January endpoint defined in Section 3.5.3, with development activity measured from 21 October 2025 to 21 January 2026 and Code Health measured for the January point-in-time software state.

Table 4.4 summarizes the resulting January endpoint population and state-specific Activity characteristics.

Table 4.4: January endpoint analysis population

Property	**January state**
Master-mapped identities	200
Numeric Code Health available	192
LOW	26
MODERATE	107
OUT_OF_SCOPE	59
Present files without numeric Code Health	4
Not present at endpoint	4
Scoreable population	133
Maximum Activity	27
P25	1
Stabilize-First candidates	33

Table 4.5 presents the quantitative planning metrics obtained after applying the same artifact definitions to this January endpoint.

Table 4.5: January endpoint quantitative planning metrics

Strategy	**Criticality Coverage**	**Debt Severity**	**Activity Alignment**	**Risk Separation**
DomainAware	1.0000	0.7510	0.8008	0
HealthOnly	0.9500	0.7883	0.6610	2
ActivityOnly	1.0000	0.7111	0.8347	0
HealthActivity	0.9500	0.7582	0.7968	0

The January endpoint metric profile was similar in structure to the April endpoint profile. DomainAware and ActivityOnly both produced Criticality Coverage of 1.0000; HealthOnly had the highest Debt Severity (0.7883), while ActivityOnly had the highest Activity Alignment (0.8347). DomainAware produced Debt Severity 0.7510 and Activity Alignment 0.8008, compared with 0.7582 and 0.7968 for HealthActivity.

HealthOnly overlapped with two January Stabilize-First candidates, whereas the other three strategies had Risk Separation equal to zero, as DomainAware's value was zero by construction. The January scoreable population again produced $r = 33$, with an Activity threshold of one commit. Section 4.6 compares the resulting January selections with the April endpoint, and supporting evidence is reported in Appendices A.5-A.7.

4.6 Temporal Sensitivity Across the January and April Endpoints

Across the two evaluated endpoints, DomainAware produced complete Top-10 set overlap: 10 of 10 files were common, corresponding to Jaccard = 1.0000.

ActivityOnly shared 9 of 10 files between the two endpoints and had Jaccard 0.8182. HealthOnly shared 8 of 10 files, with Jaccard 0.6667. HealthActivity shared 7 of 10 files, with Jaccard 0.5385.

DomainAware therefore had the largest set similarity in this two-state comparison, followed by ActivityOnly, HealthOnly, and HealthActivity.

The result is intentionally bounded to these two evaluated endpoints. It does not establish that DomainAware will always produce an unchanged Top-10 list over time. It establishes only that, under the January and April endpoint-specific inputs used in this study, the DomainAware set membership was unchanged while the other three strategy memberships changed to different degrees.

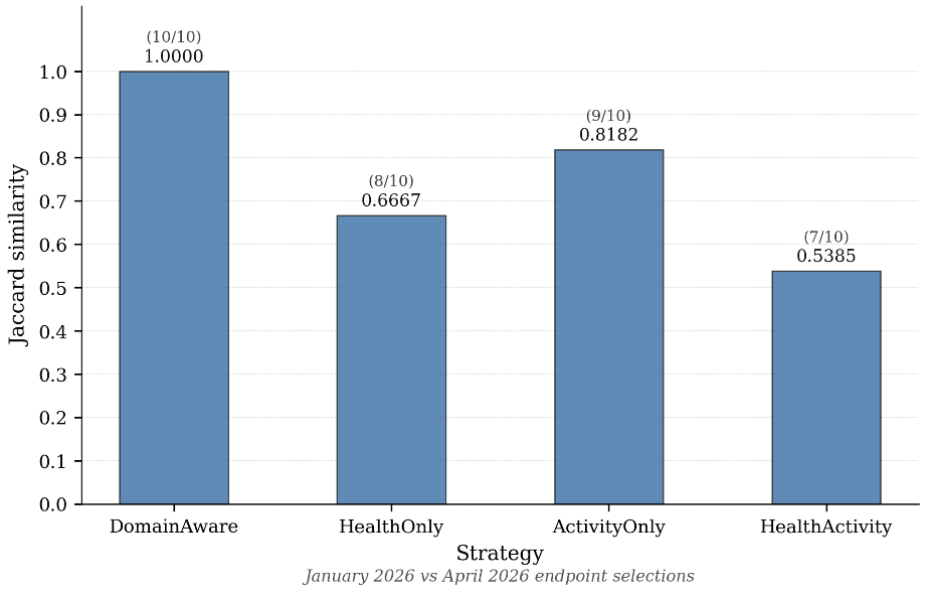


Figure 4.3: Top-10 similarity across the two evaluated endpoints. Bars show Jaccard similarity between the January and April selections; annotations show the number of common files.

The complete January-April Top-10 overlap results are provided in Appendix A.7.

4.7 Practitioner Evaluation Results

Four Ericsson 5G baseband software practitioners evaluated the finalized April endpoint artifact. During the evaluation, the algorithms were presented as Strategies A-D. The hidden mapping was:

- Strategy A = DomainAware;
- Strategy B = HealthOnly;
- Strategy C = ActivityOnly; and
- Strategy D = HealthActivity.

The practitioners reviewed the finalized ten-file lists and the complete 28-file Stabilize-First output. Internal file paths remained visible, but Code Health, Activity, domain labels, normalized values, scores, and the algorithm identities were hidden.

The individual anonymized practitioner rankings and ratings are reported in Appendix A.8.

4.7.1 Strategy Ranking

Each practitioner assigned a unique rank from 1 (most suitable) to 4 (least suitable) to the four anonymized strategies.

Table 4.6 summarizes the resulting rank distribution and mean rank for each strategy.

Table 4.6: Practitioner strategy rankings

Strategy	**Rank-1 votes**	**Mean rank**
DomainAware	2	1.50
HealthOnly	2	1.75
HealthActivity	0	2.75
ActivityOnly	0	4.00

Because rank 1 denotes the most suitable strategy, lower mean rank is more favourable. DomainAware had the lowest mean rank (1.50), followed by HealthOnly (1.75), while each received two first-place votes. HealthActivity had mean rank 2.75 with no first-place votes, and all four practitioners ranked ActivityOnly last, giving it a mean rank of 4.00. The ranking therefore shows a split between DomainAware and HealthOnly at the top rather than unanimous preference for one strategy.

4.7.2 Strategy Ratings

Figure 4.4 summarizes the mean practitioner ratings across the five evaluation dimensions and the overall strategy mean.

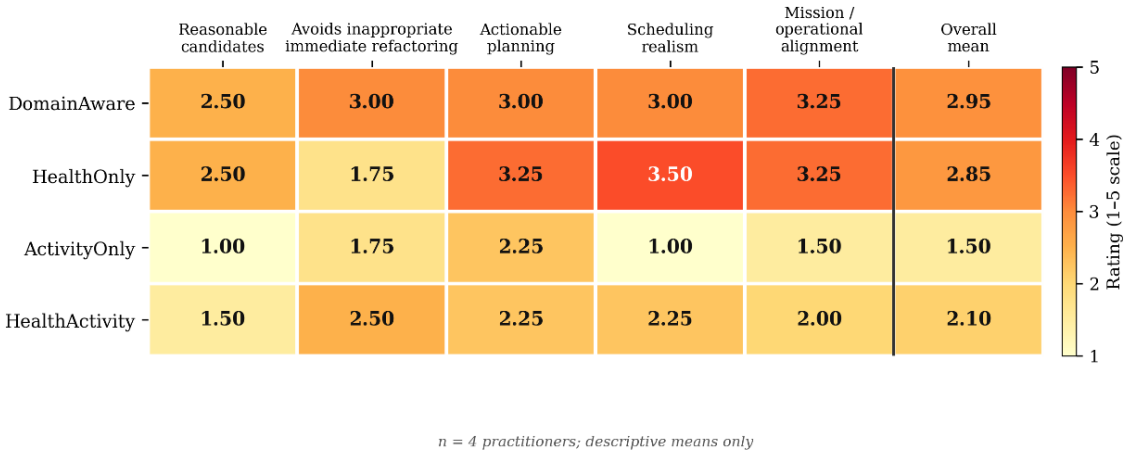


Figure 4.4: Mean practitioner ratings across the five evaluation dimensions and overall strategy mean. Ratings use a 1-5 scale and are based on four practitioners.

Practitioners also rated each strategy on a five-point agreement scale, where 1 denotes strongly disagree and 5 denotes strongly agree, across five evaluation dimensions: reasonable candidate selection, avoidance of inappropriate immediate refactoring, actionable planning, scheduling realism, and mission/operational alignment.

In Figure 4.4, each strategy-dimension value is the mean of the four practitioner ratings for that dimension. The overall mean summarizes the five dimension means for each strategy. Higher values therefore indicate more favourable practitioner assessments on the corresponding dimension. The overall mean is used only as a descriptive summary across the five dimensions and is not interpreted as a separate validated measurement scale.

DomainAware had the highest overall mean rating, 2.95, followed closely by HealthOnly at 2.85. HealthActivity had an overall mean of 2.10 and ActivityOnly 1.50.

The dimensional ratings show that the two highest-ranked strategies had different strengths. DomainAware had the highest mean score for avoiding candidates considered inappropriate for immediate refactoring, at 3.00. HealthOnly scored highest on actionable planning, at 3.25, and scheduling realism, at 3.50. DomainAware and HealthOnly were tied on mission/operational alignment at 3.25, and both had a mean of 2.50 for reasonable candidates.

ActivityOnly's April endpoint Activity Alignment was the highest of the four strategies, yet its practitioner evaluation was the weakest, with all four practitioners ranking it last and an overall mean rating of 1.50. This is a direct contrast between one quantitative planning characteristic and the contextual practitioner assessment. The broader meaning of that contrast is discussed in Chapter 5.

4.7.3 Qualitative Observations and Stabilize-First Review

In their qualitative comments, practitioners repeatedly referred to key performance indicator (KPI) impact when explaining which candidates they would prefer to address first. In this context, KPI refers to operational or service-performance measures

used by engineers when considering the potential impact of changes. The practitioner workbook recorded these concerns qualitatively. The thesis did not collect or measure KPI values for the selected files.

The comments did not treat higher operational importance as an automatic reason to refactor a file immediately. Instead, practitioners distinguished between technically relevant candidates that appeared suitable for near-term work and operationally sensitive candidates that merited additional caution.

Carrier Aggregation-related files were repeatedly identified as operationally sensitive. Two practitioners also explicitly noted that the CoMP functionality represented in the reviewed material had not been released to customers at the time of the study. This observation is consistent with the operational context that had previously triggered the domain-classification consistency review during artifact development, but the ratings and rankings reported here belong to the final evaluation performed after the researcher mapping had been frozen.

The Stabilize-First review showed broad support for a separate preparation queue but not unanimous agreement at file level. Two practitioners did not identify a direct-refactoring exception. One practitioner explicitly stated in the narrative review that F195 and F015 could proceed directly to refactoring. However, the same practitioner's separate movement-suggestion field recorded no Stabilize-First-to-Refactor-Now movement. We therefore retain this as a qualitative inconsistency rather than converting it into an aggregate movement count.

Another practitioner explicitly proposed moving F195 from Stabilize-First to Refactor-Now. The same practitioner also described F078 as having comparatively lower KPI impact and fewer dependencies, although the response did not explicitly request moving F078 between queues.

Practitioners also suggested moving some Carrier Aggregation- and CoMP-related Refactor-Now candidates in the opposite direction, toward Stabilize-First. The practitioner responses therefore included both proposed Stabilize-First-to-Refactor-Now movements and suggestions to move selected Refactor-Now candidates toward Stabilize-First. The implications of these observations for the planning mechanism are discussed in Section 5.4.

4.8 Independent Domain-Criticality Validation

The independent domain-criticality assessment evaluated the researcher-created module classifications separately from practitioner preference for the generated strategies.

Both experts received the same 31-module instrument. The instrument did not display researcher-assigned domain labels, numerical domain weights, Code Health, Activity, Base, DomainAware scores, Refactor-Now selections, baseline outputs, or practitioner-evaluation results.

The complete anonymized researcher and expert module classifications, agreement statistics, and disagreement evidence are provided in Appendix A.9.

4.8.1 Expert Completion and Label Distributions

Both experts provided an ordinal HIGH, MEDIUM, or LOW domain-criticality classification for all 31 represented modules. Neither expert used `INSUFFICIENT_INFORMATION` in the completed assessment.

The frozen researcher mapping contained 11 HIGH, 12 MEDIUM, and 8 LOW represented modules. Expert 1 classified 11 modules as HIGH, 13 as MEDIUM, and 7 as LOW. Expert 2 classified 10 modules as HIGH, 15 as MEDIUM, and 6 as LOW.

Table 4.7 summarizes the three label distributions.

Table 4.7: Domain-criticality label distributions

Assessment	**HIGH**	**MEDIUM**	**LOW**	**Total**
Frozen researcher mapping	11	12	8	31
Expert 1	11	13	7	31
Expert 2	10	15	6	31

Because both experts classified all 31 modules using the ordinal categories, all represented modules could be included directly in the expert-expert and researcher-expert agreement calculations reported below.

4.8.2 Agreement Between the Two Experts

The two experts assigned the same HIGH, MEDIUM, or LOW label to 28 of the 31 represented modules. Their exact raw agreement was therefore:

$$\frac{28}{31} \times 100 = 90.32\%.$$

The linearly weighted Cohen's kappa across all 31 modules was:

$$\kappa_w \approx 0.8347.$$

The experts disagreed on three modules, shown in Table 4.8.

Table 4.8: Modules on which the two experts assigned different domain-criticality labels

Module	**Expert 1**	**Expert 2**
M13	HIGH	MEDIUM
M14	HIGH	MEDIUM
M40	LOW	HIGH

For M13 and M14, the disagreement was between adjacent HIGH and MEDIUM categories. M40 showed the largest disagreement, with Expert 1 assigning LOW and Expert 2 assigning HIGH. Thus, despite the strong aggregate agreement, individual modules can still receive substantially different interpretations.

4.8.3 Agreement With the Frozen Researcher Mapping

We next compared each expert independently with the researcher-created domain-criticality mapping that had been frozen before the experts conducted their assessments. Because both experts supplied an ordinal label for all 31 represented modules, each comparison uses the complete represented-module population.

Table 4.9 summarizes the resulting agreement statistics.

Table 4.9: Agreement between the frozen researcher mapping and the two expert assessments

Comparison	**Exact agreement**	**Raw agreement**	**Weighted κ**
Researcher - Expert 1	24/31	77.42%	0.7264
Researcher - Expert 2	26/31	83.87%	0.7984

For Expert 1, the frozen researcher mapping agreed on 24 of the 31 modules. The seven disagreements involved M06, M07, M13, M14, M30, M31, and M40. The corresponding linearly weighted Cohen's kappa was:

$$\kappa_w \approx 0.7264.$$

Table 4.10 shows the complete comparison.

Table 4.10: Frozen researcher labels compared with Expert 1

Frozen researcher label	**Expert 1 LOW**	**Expert 1 MEDIUM**	**Expert 1 HIGH**	**Total**
LOW	6	2	0	8
MEDIUM	1	9	2	12
HIGH	0	2	9	11
Total	7	13	11	31

Two researcher-HIGH modules, M06 and M07, were classified MEDIUM by Expert 1. Two researcher-MEDIUM modules, M13 and M14, were classified HIGH. Two researcher-LOW modules, M30 and M31, were classified MEDIUM. M40 was classified MEDIUM by the researchers and LOW by Expert 1.

For Expert 2, the frozen researcher mapping agreed on 26 of the 31 modules. The five disagreements involved M06, M07, M30, M31, and M40. The corresponding linearly weighted Cohen's kappa was:

$$\kappa_w \approx 0.7984.$$

Table 4.11 shows the complete comparison.

For Expert 2, M06 and M07 were classified MEDIUM rather than the researcher HIGH classification. M30 and M31 were classified MEDIUM rather than LOW, while M40 was classified HIGH rather than the researcher MEDIUM classification.

Table 4.11: Frozen researcher labels compared with Expert 2

Frozen label	Expert 2 LOW	Expert 2 MEDIUM	Expert 2 HIGH	Total
LOW	6	2	0	8
MEDIUM	0	11	1	12
HIGH	0	2	9	11
Total	6	15	10	31

The researcher mapping agreed with Expert 1 on 24 of 31 modules and with Expert 2 on 26 of 31 modules. These assessments support the consistency of much of the frozen mapping while identifying specific modules for further review.

Agreement After Propagation to Files

The independent experts assessed domain criticality at module level rather than classifying individual files. Because the artifact propagates each module-level label to all files associated with that module, we additionally examined the consequence of the module-level classifications for the 200-file master population.

This file-level comparison is a derived analysis based on the established module-to-file mapping. It should therefore not be interpreted as 200 independent file-level expert judgments.

Table 4.12 summarizes the module-level and corresponding propagated file-level agreement.

Table 4.12: Module-level and propagated file-level domain-classification agreement

Comparison	Module agreement	Propagated file agreement
Expert 1 - Expert 2	28/31 (90.32%)	176/200 (88.00%)
Researcher - Expert 1	24/31 (77.42%)	170/200 (85.00%)
Researcher - Expert 2	26/31 (83.87%)	172/200 (86.00%)
All three classifications	24/31 (77.42%)	170/200 (85.00%)

All three assessments assigned the same domain-criticality label to 24 of the 31 represented modules. Through the established module-to-file mapping, these commonly classified modules account for 170 of the 200 files in the master population. Thus, 85.00% of the master-population files inherit the same domain-criticality label under the frozen researcher mapping and both independent expert assessments.

The propagated file-level percentages do not correspond directly to the module-level percentages because the represented modules contain different numbers of files. A disagreement concerning a module with many mapped files therefore affects more inherited file labels than a disagreement concerning a module represented by only one or a few files. For example, M40 contains 22 files in the 200-file master population, so the disagreement between the experts for M40 has a comparatively large effect on propagated file-level agreement.

These values therefore characterize the consequence of module-level classification differences for the file-level artifact. They do not constitute an independent file-level expert-validation exercise.

4.8.4 Disagreement Patterns and Module-Level Observations

The aggregate agreement statistics conceal several informative module-level patterns. Table 4.13 presents representative cases.

Table 4.13: Selected domain-classification comparisons

Module	Frozen researcher label	Expert 1	Expert 2
M06	HIGH	MEDIUM	MEDIUM
M07	HIGH	MEDIUM	MEDIUM
M13	MEDIUM	HIGH	MEDIUM
M30	LOW	MEDIUM	MEDIUM
M40	MEDIUM	LOW	HIGH

M06 and M07 illustrate a common researcher-expert disagreement. Both experts classified these modules MEDIUM, whereas the frozen researcher mapping classified them HIGH. The two independent assessments therefore placed these modules one ordinal level below the researcher classification.

M30 and M31 show a common disagreement in the opposite direction. Both experts classified these modules MEDIUM, whereas the frozen researcher mapping classified them LOW. The expert assessments therefore assigned greater domain criticality to these modules than the researcher mapping.

M13 and M14 illustrate disagreements involving only one expert. The frozen researcher mapping and Expert 2 classified these modules MEDIUM, whereas Expert 1 classified them HIGH.

M40 produced the largest disagreement among the independent assessments. The frozen researcher mapping classified M40 MEDIUM, Expert 1 classified it LOW, and Expert 2 classified it HIGH. The three classifications therefore span the complete ordinal range for this module. Because M40 also contains multiple mapped files, this disagreement has a comparatively large effect on the propagated file-level agreement.

The modules explicitly reconsidered during the classification-consistency refinement also provide a useful observation. M04 was classified HIGH in the frozen researcher mapping and by both experts. M10 and M18 were classified MEDIUM in the frozen researcher mapping and by both experts. The independent assessments therefore agreed with the final frozen classifications of these three modules.

The qualitative expert material also identified a limitation of module-level propagation. Expert 2 explicitly indicated that a single domain label did not adequately characterize all files belonging to M01 and suggested distinguishing files according to the feature they implement. This observation directly concerns the study's decision to assign criticality at module level and then propagate the label uniformly to files.

The two expert assessments were retained as independent evaluations of the frozen researcher mapping. No post-assessment consensus classification was pro-

duced. Accordingly, the expert-validation analysis concludes with the agreement statistics, researcher-expert confusion matrices, propagated file-level agreement, and module-level observations reported above, while the primary DomainAware results continue to use the frozen researcher mapping evaluated by the experts.

4.9 Summary of Results

The factor-feasibility assessment retained Code Health, development activity, and module-level domain criticality as the three artifact inputs. At the April endpoint, DomainAware and HealthActivity shared eight of ten selections, with the full DomainAware procedure producing two different MODERATE-band selections. DomainAware retained identical Top-10 membership across the January and April endpoints, while the three baseline selections changed to different degrees.

Practitioner evidence was favourable to DomainAware on average but not unanimous, with HealthOnly receiving the same number of first-place votes. The independent domain assessment showed 28/31 expert-expert agreement (90.32%, $\kappa_w \approx 0.8347$), while the remaining disagreements highlighted uncertainty at specific domain-category boundaries and within heterogeneous modules. Chapter 5 interprets these findings in relation to the research questions, prior work, practical implications, and validity limitations.

Chapter 5

Discussion

5.1 Chapter Overview

This chapter interprets the findings presented in Chapter 4 and relates them to the research questions, the methodological choices of the study, and the body of work reviewed in Chapter 2. Whereas Chapter 4 reports what was observed, the purpose here is to examine what those observations mean for domain-aware refactoring prioritization in the studied telecom context.

The discussion first synthesizes the findings against the two research questions. It then examines the Stabilize-First mechanism, temporal sensitivity, and the domain-criticality construct before relating the findings to prior work, alternative explanations, practical transferability, and wider ethical and sustainability considerations.

The study is intentionally positioned as a pre-refactoring decision-support study. Its contribution is to make the selection of near-term refactoring candidates more structured, transparent, and auditable before implementation work begins. The artifact therefore provides evidence about which files receive priority under the defined technical, activity, and domain criteria and how practitioners assess the resulting plans.

Because the study ends at the prioritization stage, downstream effects such as defect reduction, post-refactoring maintainability, engineering productivity, network reliability, or refactoring safety require separate outcome evaluation. This boundary defines the level of evidence provided by the thesis without reducing the artifact's role as a planning aid.

5.2 RQ1: Relevant and Feasible Prioritization Factors

RQ1 asked:

> *Which prioritization factors are relevant and feasible for file-level refactoring-target selection in the selected telecom software context?*

In this discussion, a factor is considered feasible when the required evidence is available with sufficient coverage for the intended analysis, can be connected to the candidate files, and can be applied consistently enough to support the deterministic artifact. Feasible therefore refers to practical usability of the factor in this study, not to whether the factor is theoretically important.

Within the selected UPC uplink scope, the study identified Code Health, development activity, and domain criticality as the factors that were both relevant to the prioritization problem and feasible to operationalize with sufficient population-wide evidence.

Code Health addresses the maintainability side of the problem. This is consistent with the maintainability and refactoring literature discussed in Section 2.4. Structural quality matters because refactoring is fundamentally concerned with improving the internal structure and evolvability of software. The study could also operationalize this factor consistently through file-level CodeScene measurements for nearly the complete master population.

However, the Results reinforce why Code Health alone does not completely represent the planning problem. HealthOnly maximized Debt Severity, but this came with substantially lower Activity Alignment than the other strategies. Thus, the strongest maintainability concerns were not necessarily the files most aligned with recent development interaction.

Development activity contributed a different perspective. The use of change history follows the reasoning behind change metrics and Behavioural Code Analysis reviewed in Section 2.5. Software evolution history can reveal areas that repeatedly attract engineering attention. ActivityOnly achieved the highest Activity Alignment in the six-month analysis, confirming that the signal behaved as designed.

At the same time, its practitioner performance was poor, with all four practitioners ranking it last. This distinction is important. Development activity was relevant and feasible, but activity alone did not correspond to practitioner perceptions of suitable refactoring planning.

Domain criticality added information that Code Health and development activity could not provide. Two files can have similar maintainability condition and similar development activity while belonging to modules with very different operational roles. The domain classification allowed the artifact to represent this difference by considering service impact, operational importance, and available fallback or workaround behaviour.

This is consistent with the architecture and technical-debt literature discussed in Section 2.6, where engineering decisions are influenced by consequences and context in addition to technical condition.

Domain criticality was nevertheless different from the other two factors because it was not obtained directly from a repository or analysis tool. The researchers interpreted architecture and HLD evidence to classify the represented modules and then propagated those classifications to files through the established module-to-file mapping.

The independent expert assessment provides additional support for the feasibility of this construct. Both experts were able to classify all 31 represented modules, and the resulting assessments showed high agreement with each other and substantial agreement with the frozen researcher mapping. At the same time, specific module-level disagreements remained, and one expert explicitly identified within-module heterogeneity as a limitation of assigning one criticality label uniformly to all files in a module.

Domain criticality was therefore feasible to operationalize and independently review, but it remains more dependent on contextual engineering judgement than Code

Health or development activity. The expert evidence strengthens confidence that the construct captured substantial shared domain understanding without establishing any individual classification as objective ground truth.

Other candidate factors may also be relevant. Change or temporal coupling, incident information, and consistent file-level test coverage were not excluded because the study showed them to be unimportant. They were excluded because the available evidence was not sufficiently complete and consistent for use across the intended candidate scope.

RQ1 should therefore be understood as identifying the factors that this study could both justify and operationalize, rather than identifying every factor that could matter in refactoring planning.

RQ1 can consequently be answered as follows:

> *Code Health, development activity, and module-level domain criticality mapped to files were the three prioritization factors that were both relevant to the decision problem and feasible to use in the selected telecom software context. Code Health represented maintainability concern, development activity represented recent development interaction, and domain criticality represented operational context.*

Other factors may also be relevant, but the available evidence did not support their consistent inclusion in this study.

A broader implication is that industrial prioritization methods must consider data availability as well as conceptual relevance. In this study, using a smaller set of well-defined and auditable inputs supported a reproducible decision artifact without requiring incomplete factors to be estimated or selectively applied.

5.3 RQ2: Usefulness of the Domain-Aware Artifact

RQ2 asked:

> *How useful is the resulting domain-aware refactoring-prioritization method compared with domain-agnostic baselines from quantitative and practitioner perspectives?*

The combined evidence indicates that DomainAware can support pre-refactoring planning within the studied context, but it does not support the stronger claim that DomainAware is unambiguously or universally superior to all simpler strategies.

This distinction is important because different parts of the evaluation provide different kinds of evidence. The quantitative comparison shows how the generated selections differ on predefined planning dimensions. The practitioner evaluation shows how four practitioners with relevant local knowledge assessed the resulting plans. The independent expert study, meanwhile, evaluates a key construct on which DomainAware depends rather than directly evaluating planning usefulness.

The independent expert evidence strengthens the construct-level basis for interpreting the DomainAware results because the two independent assessments showed substantial agreement with the frozen researcher mapping. However, this evidence

remains conceptually separate from practitioner usefulness. Agreement about module criticality does not demonstrate that DomainAware is the best planning strategy, just as practitioner preference does not validate the underlying domain labels. The two forms of evidence therefore complement rather than substitute for one another.

5.3.1 Quantitative Comparison with the Baselines

The most informative quantitative comparison is between DomainAware and HealthActivity. HealthActivity uses the same technical Base as DomainAware but does not incorporate domain criticality or apply the DomainAware planning separation. It therefore provides the most informative domain-agnostic technical baseline for examining how the final DomainAware selections differ.

At the April endpoint, DomainAware and HealthActivity selected eight of the same ten files (Table 4.2 and Figure 4.2). Their five LOW Code Health selections were identical, and their first three MODERATE Code Health selections were also shared.

The difference was confined to the final two MODERATE-band positions: HealthActivity selected two files mapped to MEDIUM-criticality modules, whereas DomainAware selected two files mapped to HIGH-criticality modules. The full DomainAware procedure therefore changed two selections at the boundary while retaining the other eight files in the technical-composite shortlist.

The resulting selection increased Criticality Coverage from 0.9000 to 1.0000. At the same time, Debt Severity changed only from 0.7661 to 0.7583, while Activity Alignment changed from 0.8146 to 0.8191. Therefore, in this dataset, the full DomainAware procedure produced a selected set with stronger alignment to the frozen researcher criticality mapping while retaining very similar Debt Severity and Activity Alignment values to HealthActivity.

These metric differences should be interpreted as trade-offs among planning dimensions rather than as a single ordering of strategy quality. HealthOnly emphasizes maintainability concern by design, ActivityOnly emphasizes recent development interaction, and DomainAware introduces operational context while retaining the technical Base. A strategy can therefore score highest on the dimension it directly prioritizes while scoring lower on another dimension.

The observed pattern is consistent with the design rationale for separating the technical Base from the domain adjustment. The DomainAware selections remained largely aligned with the technical-composite baseline while differing at a small number of positions near the selection boundary. This behaviour is consistent with the artifact's design objective that domain information should modify an existing technical case rather than make HIGH criticality sufficient, by itself, for immediate refactoring selection. However, because DomainAware also applies the Stabilize-First eligibility rule, this comparison does not isolate the effect of the domain multiplier alone.

Criticality Coverage must nevertheless be interpreted cautiously. The metric uses the same researcher-created domain construct that DomainAware uses for scoring. A higher value therefore indicates stronger alignment with that mapping, not independently verified operational superiority.

The ActivityOnly result illustrates this limitation particularly clearly. ActivityOnly also achieved Criticality Coverage of 1.0000 despite not using domain infor-

mation at all. This occurred because all ten ActivityOnly candidates happened to belong to HIGH-labelled modules in the frozen mapping. Consequently, a high Criticality Coverage score alone cannot establish that a strategy reasons appropriately about domain importance.

The quantitative evaluation therefore supports a narrower conclusion: the full DomainAware procedure produced a selection more strongly aligned with the researcher-defined domain construct while retaining similar maintainability and activity characteristics to HealthActivity. This comparison therefore describes the behaviour of the complete DomainAware procedure rather than the effect of any single design component. The broader usefulness of the resulting selection requires the practitioner and construct-validity evidence considered below.

5.3.2 Practitioner Assessment of Planning Usefulness

The practitioner evaluation provides contextual evidence about the usefulness of the generated plans, while also limiting how strongly the DomainAware result should be interpreted.

As shown in Table 4.6, DomainAware had the most favourable mean rank at 1.50, followed closely by HealthOnly at 1.75. The first-place votes were evenly divided: two practitioners ranked DomainAware first and two ranked HealthOnly first. Figure 4.4 shows a similar pattern in the dimensional ratings. DomainAware had the highest overall mean rating at 2.95, while HealthOnly followed at 2.85. The evaluation therefore favoured DomainAware on average, but it did not show unanimous practitioner preference.

The absolute strategy ratings strengthen this cautious interpretation. On the five-point agreement scale, DomainAware's overall mean of 2.95 lies close to the scale midpoint and therefore should not be interpreted as strong endorsement. The practitioner evidence therefore indicates that DomainAware was the strongest strategy on average, but not that practitioners considered it clearly superior in every dimension.

The practitioner results suggest that the complete DomainAware method can support planning in this setting, but they do not establish that each additional signal independently improves the decisions. HealthOnly remained close in the ratings and was preferred by two of the four practitioners. The dimensional results also reveal different strengths. DomainAware received the highest mean rating for avoiding candidates considered inappropriate for immediate refactoring, while HealthOnly was rated more strongly on actionable planning and scheduling realism. Both strategies received the same mean rating for mission or operational alignment.

ActivityOnly provides the clearest counterexample for treating any individual quantitative metric as sufficient evidence of usefulness. It achieved the highest Activity Alignment and, coincidentally, Criticality Coverage of 1.0000, yet all four practitioners ranked it last and its overall mean rating was 1.50. This finding suggests that repeated development interaction alone does not capture the practitioner decision context represented in this evaluation.

Taken together, the quantitative and practitioner evidence answer RQ2 more defensibly than either source alone:

The evidence from the studied setting indicates that DomainAware can support transparent pre-refactoring planning by introducing operational context while largely preserving the technical candidate set. DomainAware received the most favourable mean practitioner rank and overall mean rating, but practitioner preference was not unanimous, HealthOnly remained competitive, and the absolute practitioner ratings did not indicate strong endorsement. The findings therefore provide context-specific evidence that DomainAware can support pre-refactoring planning rather than establishing universal superiority.

5.4 Interpreting the Stabilize-First Mechanism

The Stabilize-First mechanism addresses a different question from the DomainAware score. Domain amplification answers which technically relevant candidates receive greater priority because of their mapped operational context. Stabilize-First asks whether a particular combination of HIGH domain criticality and relatively low recent activity should trigger additional preparation before immediate refactoring consideration.

The six-month rule identified 28 Stabilize-First candidates. This result should not be interpreted as identifying 28 files that are unsafe to refactor. The threshold is a researcher-designed planning rule, and DomainAware's Risk Separation value of zero is guaranteed by the artifact because Stabilize-First files are removed before Refactor-Now selection. Risk Separation therefore describes the operation of the planning mechanism rather than independently measuring engineering safety.

The practitioner responses nevertheless indicate that separating preparation from immediate ranking has practical relevance, but not as an inflexible rule. As reported in Section 4.7.3, practitioners proposed exceptions in both directions: some Stabilize-First candidates were considered potentially suitable for direct refactoring, while some Refactor-Now candidates were considered better suited to additional preparation. One response also contained an inconsistency between the participant's narrative comment and movement-suggestion field, reinforcing the need to interpret the queue assignments as decision support rather than final action decisions.

These exceptions indicate that practitioners did not always agree with the assigned queues. They show that a deterministic queue rule cannot replace contextual engineering judgement. Hence, Stabilize-First is best understood as a mechanism for making candidates requiring additional preparation or review explicit, rather than as an automatic decision about what must or must not be refactored.

This interpretation is consistent with the broader decision-oriented perspective discussed in Section 2.6. Architectural and technical-debt decisions involve context, consequences, and timing. The contribution of Stabilize-First is to represent one such timing distinction explicitly at planning level: a technically relevant file may warrant attention without necessarily being selected for immediate structural change.

The resulting implication for the artifact design is that Refactor-Now and Stabilize-First should remain decision-support outputs. A practitioner should be able to inspect why a file entered either queue, challenge the underlying information, and override the recommendation when additional local knowledge is available.

5.5 Temporal Sensitivity of the Prioritization

Across the January and April endpoints, DomainAware produced complete Top-10 set overlap: all 10 selected files were common, corresponding to Jaccard similarity of 1.0000. ActivityOnly shared nine files, HealthOnly eight, and HealthActivity seven, as shown in Figure 4.3.

This result is noteworthy because the technical evidence was endpoint-specific. The January analysis used point-in-time January Code Health and Activity measured over the October-to-January interval, whereas the April analysis used the corresponding April Code Health and the longer October-to-April Activity window. The artifact rules and researcher domain mapping remained fixed.

The January activity window is contained within the April activity window, so the two measurements share part of the development history. This shared history may contribute to selection overlap. The temporal comparison therefore describes changes between two related endpoints, not independent development periods. Complete overlap in this comparison does not establish general temporal stability across other periods or software contexts.

Several characteristics of the artifact may have contributed to the observed overlap. The frozen domain mapping remained constant, the selection budget and maintainability-band allocation were fixed, and domain amplification may have changed the relative distance between candidates close to a ranking boundary. Stabilize-First also changes DomainAware's eligible population. These mechanisms could potentially make some selections less sensitive to changes in the time-dependent technical inputs near the selection boundary than the technical Base alone.

With only two evaluated endpoints, the study cannot isolate how much of the observed 10/10 overlap is associated with domain amplification, the fixed 5+5 band allocation, the Stabilize-First eligibility rule, or the endpoint-specific technical inputs. A stronger assessment would require additional temporal endpoints and/or controlled sensitivity analyses in which individual design parameters or inputs are varied while the others are held fixed.

The contrast with HealthActivity is nevertheless notable. HealthActivity shared only seven candidates across the same two endpoints even though it forms the technical Base of DomainAware. The contrast is consistent with the interpretation that domain amplification and the DomainAware planning rules affected which candidates remained in the final set across the two evaluated endpoints. It does not establish that this effect will recur or improve temporal consistency in other periods.

The temporal result therefore establishes a narrow but clear observation: no DomainAware Top-10 membership change occurred between the January and April endpoints. Whether the method is generally robust to temporal changes remains an open empirical question.

5.6 Domain-Criticality Construct and Independent Expert Evidence

The independent expert assessment provides important construct-level evidence for interpreting the DomainAware results. Unlike the practitioner evaluation, which

examined the usefulness of the generated planning outputs, this assessment examined the researcher-created domain-criticality classifications themselves.

Both experts classified all 31 represented modules and agreed on 28 of them (90.32%, $\kappa_w \approx 0.8347$). The frozen researcher mapping agreed with Expert 1 on 24 of 31 modules and with Expert 2 on 26 of 31 modules. These results provide construct-level support for substantial parts of the researcher-created mapping, while the remaining disagreements show that the classification still depends on contextual engineering judgement.

The module-level disagreements reported in Section 4.8 show that uncertainty remained at specific classification boundaries rather than being distributed uniformly across the represented modules. M40 was the clearest case: the researcher mapping classified it MEDIUM, Expert 1 classified it LOW, and Expert 2 classified it HIGH. This disagreement across the complete ordinal range shows that strong aggregate agreement does not eliminate uncertainty for individual modules.

The two experts also agreed with the frozen classifications of M04, M10, and M18. This provides additional support for the final consistency-review outcomes without establishing those classifications as objectively correct.

After propagation through the established module-to-file mapping, all three assessments assigned the same inherited label to 170 of the 200 master-population files. This value must be interpreted as a consequence of module-level judgments rather than as 200 independent file-level expert assessments, because modules contain different numbers of mapped files. For example, M40 contains 22 master-population files and therefore has a comparatively large influence on the propagated agreement.

Module-level propagation also requires attention to population concentration. M01 contains 100 of the 200 master-population files and 69 of the 136 April scoreable files. Eight of the ten April DomainAware selections belong to this module (Appendix A.5 and Table A.8). Expert 2 noted that one criticality label may not adequately represent the different functions implemented by files within M01. Consequently, several selected files inherit the same module-level judgement. This concentration makes the treatment of within-module variation particularly relevant when interpreting the results.

The expert evidence therefore supports a more qualified interpretation of the researcher mapping: substantial agreement was observed overall, while specific modules and abstraction boundaries remained uncertain.

Only two experts participated; two of the expert validators also participated in the practitioner exercise, and the classification still depends on contextual engineering judgement. No post-assessment expert-agreement mapping was created, and the primary DomainAware results were not recomputed using retrospectively modified labels.

Consequently, quantitative results involving DomainWeight, especially Criticality Coverage, remain conditional on the frozen researcher mapping. DomainAware's Criticality Coverage of 1.0000 indicates alignment with that mapping. The expert assessment increases confidence that substantial parts of the mapping reflect shared domain understanding, but it does not transform Criticality Coverage into an independent measurement of operational correctness.

The contribution is therefore not a definitive taxonomy of operational criticality. Rather, the study demonstrates a transparent approach in which domain judgments

are explicitly represented, independently reviewed, propagated to the file-level planning artifact, and retained together with the remaining areas of disagreement. The complete anonymized expert classifications and derived agreement statistics are reported in Appendix A.9.

5.7 Relation to Prior Work and Research Contribution

The results can be positioned against three main strands of related work: technical-debt and refactoring prioritization, behavioural/change-history analysis, and architecture/domain-aware decision making.

Lenarduzzi et al. characterize technical-debt prioritization as a multi-dimensional decision involving different technical and contextual factors [19], while Alfayez et al. show that developers can differ in how they balance value and cost under limited resources [5]. The present findings support this framing at file level: no strategy dominated all planning dimensions, and practitioner preference was split between DomainAware and HealthOnly. The contribution is therefore not to replace existing prioritization criteria, but to examine how operational context can be incorporated into a bounded file-level pre-refactoring planning artifact.

Prior work on change metrics and Behavioural Code Analysis shows that development history can provide information complementary to static structural properties [24, 29]. HealthActivity operationalizes this principle by combining maintainability concern with recent development activity. At the April endpoint, eight of the ten DomainAware candidates were also selected by HealthActivity, showing that the technical composite remained a substantial part of the final DomainAware selection. The full DomainAware procedure produced different candidates only at two MODERATE-band boundary positions. Because the procedure combines domain amplification with the Stabilize-First eligibility rule, this difference cannot be attributed to the domain multiplier alone. In the evaluated configuration, operational context therefore modified a largely shared technical shortlist rather than replacing the technical evidence.

The observed 8/10 overlap is nevertheless conditional on the proof-of-concept budget of $K = 10$ and the 5+5 band allocation. A larger selection budget could expose additional differences deeper in the rankings, but this study did not evaluate larger K values. The effect of selection budget should therefore be treated as a parameter-sensitivity question for future evaluation rather than as an expected result.

The ActivityOnly result should be interpreted narrowly. In this evaluation, ranking files only by normalized non-merge commit activity did not align well with practitioner preferences, as all four practitioners ranked ActivityOnly last. This finding does not evaluate Behavioural Code Analysis as a whole. It shows only that development activity by itself was insufficient as a refactoring-planning criterion in the evaluated artifact and industrial context.

Martini and Bosch's AnaConDebt shows that architectural technical-debt decisions depend on consequences, timing, and stakeholder reasoning [22], while de Almeida et al. demonstrate that business-oriented criteria can influence industrial

technical-debt prioritization [27]. The present study extends this decision-oriented perspective to a different granularity: module-level operational context is mapped to file-level candidates and incorporated into a bounded pre-refactoring planning procedure. The result is therefore an application of context-aware decision reasoning to file-level planning rather than a claim that contextual prioritization itself is new.

The specific research contribution is therefore not the general observation that domain context matters. That principle already appears in architecture and technical-debt decision research. The contribution is the operationalization and industrial evaluation of a lightweight, deterministic combination of file-level maintainability, development activity, module-level domain criticality, and a separate preparation queue.

From a Design Science Research perspective, the artifact also contributes design knowledge about the structure of such a prioritization method. Separating the technical Base from domain amplification supports transparency because the technical case can be inspected independently from the contextual adjustment. Similarly, separating Refactor-Now from Stabilize-First prevents HIGH criticality from automatically implying immediate refactoring.

A further practical aspect of this contribution is auditability. The artifact's input populations, frozen researcher mapping, parameter configuration, endpoint-specific outputs, human-evaluation evidence, and derived validation statistics are linked to the supporting evidence reported in Appendix A. Although proprietary Ericsson source material cannot be reproduced publicly in full, the evidence included in the thesis allows the reported computational and human-evaluation results to be traced through the documented study records. Transparency therefore concerns not only the scoring formula but also the evidence chain supporting the reported findings.

These design choices remain hypotheses for wider reuse rather than universally established principles. Their value outside the present case requires replication.

5.8 Alternative Explanations and Limits of Interpretation

Several findings admit more than one plausible interpretation, and these alternatives are important for avoiding confirmation bias.

One potentially surprising result is that ActivityOnly achieved Criticality Coverage of 1.0000 even though it did not use domain information. HIGH-labelled files were common in the studied population, and the most active selected candidates happened to belong to HIGH-labelled modules. Full Criticality Coverage in this population therefore does not establish that activity generally represents operational importance. Separately, all four practitioners ranked ActivityOnly last, indicating that its candidate lists were less preferred for refactoring planning.

A second finding is the strong performance of HealthOnly in the practitioner evaluation. Two practitioners ranked it first, and its overall mean rating was close to DomainAware's. HealthOnly may have selected files that practitioners already recognized as having maintainability concerns. However, because strategy identities and numerical inputs were hidden, the evaluation does not establish whether practi-

tioners valued the simplicity of HealthOnly's decision logic. This result means that the thesis should not frame simpler strategies as weaker baselines.

A third issue concerns the absolute practitioner ratings. Although DomainAware had the highest overall mean rating, its value of 2.95 lies close to the midpoint of the five-point scale. The relative ranking is therefore more favourable than the absolute level of practitioner agreement.

One possible explanation is that the four generated strategies exclude contextual information that may matter in practical planning, such as dependency relationships, test confidence, current feature plans, ownership, or implementation-specific knowledge. The present evaluation did not measure the contribution of those additional factors.

A fourth issue concerns interpretation of the independent expert agreement. The high expert-expert agreement and substantial researcher-expert agreement are consistent with the interpretation that the domain rubric captured a meaningful body of shared operational knowledge. However, agreement between two experts does not establish objective correctness. The experts worked within the same organizational and technical context and received the same structured module information, so some common interpretation is expected.

The localized disagreements provide an important counterbalance to the aggregate agreement. M40 received LOW, MEDIUM, and HIGH across the three assessments, while M06, M07, M30, and M31 showed systematic differences between the researcher and both experts. These cases demonstrate that strong aggregate agreement can coexist with important uncertainty at individual-module boundaries.

The propagated file-level agreement should also not be interpreted as 200 independent validation observations. It inherits the module-level assessments, and modules with more mapped files contribute more strongly to the resulting percentage. The 85.00% three-way propagated agreement therefore describes the consequence of the module classifications for the file-level artifact rather than a separate file-level validation study.

In addition, two of the four practitioner participants also served as the two independent domain experts. The expert-assessment instrument withheld researcher labels and scoring information, but the use of the same participants across the two tasks did not make the human evidence sources statistically independent.

A fifth issue is parameter dependence. The evaluated configuration fixes the technical weights, domain weights, β, K, band allocation, and Stabilize-First threshold before evaluation. Alternative configurations may change candidate ordering or selection membership. The present results therefore apply to the evaluated configuration, while systematic sensitivity analysis is needed to determine how robust the selections are to parameter variation.

Likewise, Risk Separation cannot be used as symmetric evidence that DomainAware "outperformed" the baselines on safety. DomainAware excludes Stabilize-First candidates by construction, whereas the baseline strategies do not. The metric therefore exposes a difference in policy behaviour rather than an independently measured safety outcome.

Finally, the evidence comes from one bounded Ericsson component, four practitioners, two domain validators, and two evaluated endpoints. This is appropriate for an industrial design study but prevents statistical generalization.

These limitations do not invalidate the artifact. They define what the evidence can support: a transparent proof-of-concept with observed behaviour and context-specific evidence of planning usefulness in one industrial setting, rather than a universally validated prioritization model.

5.9 Practical Implications and Transferability

The main practical implication is that refactoring prioritization can be treated as a staged decision process rather than as a single score.

The first stage narrows attention to files with maintainability concern. The second adds development activity to distinguish code that repeatedly attracts engineering interaction. The third uses domain context to change priority when technical candidates have different operational significance. The final planning stage separates immediate candidates from HIGH-criticality low-activity files that warrant preparation or explicit review. This structure has an advantage for an industrial decision support artifact, as each transition remains explainable. An engineer can inspect a file's Code Health, activity, module classification, domain multiplier, and final planning status.

The practitioner findings indicate that transparency should be paired with human review. DomainAware generates candidate lists and explanations to support planning. Engineers should review these outputs before making refactoring decisions and consider relevant information that the artifact does not contain.

The practical value of maintaining explicit domain information must also be considered together with the effort required to construct and update it. In this study, the full DomainAware procedure differed from HealthActivity at two of ten April positions, while practitioners repeatedly referred to operational or KPI impact when explaining their preferences. This indicates that domain context can influence planning even when the resulting shortlist changes only selectively. However, the mapping requires architecture/HLD interpretation, domain expertise, and periodic review as responsibilities, release status, or fallback behaviour change. Further evaluation is needed to determine whether the planning benefits justify the effort required to create and maintain the domain mapping.

A practical implementation should also retain the ability to record uncertainty or exceptions rather than assuming that one module label necessarily describes every associated file. The M01 observation and the M40 disagreement show why engineers should be able to challenge or refine module-level classifications when more specific information becomes available.

Transfer to another software component would require equivalent evidence rather than identical numerical outcomes. At minimum, another application would need a file-level maintainability indicator, repository change history, a defensible mapping from domain units to files, and domain knowledge sufficient to assess operational consequences.

The current weights and thresholds should not automatically be transferred. A replication should reconsider the appropriate planning budget, maintainability bands, domain labels, domain amplification strength, and Stabilize-First rule in the target context.

The supporting material reported in Appendix A illustrates the type of evidence chain needed to support such traceability: a defined study population, module mapping, frozen domain classification, parameter configuration, strategy outputs, and human-evaluation records. A replication need not use the same numerical values, but it should preserve an equivalent chain from inputs to reported outputs.

5.10 Ethical, Societal, and Sustainability Considerations

The principal ethical concern is responsible interpretation of the prioritization output.

In operationally important software, a numerical ranking can create automation bias if engineers treat it as an objective statement of which code must be changed. This risk is amplified when the ranking includes a domain-criticality value, because a numerical weight can appear more precise than the qualitative judgment from which it originated.

The thesis addresses this concern by keeping the method deterministic and inspectable, explicitly identifying researcher-designed parameters, preserving practitioner review, and distinguishing the Stabilize-First queue from a statement of objective safety. The combination of high expert agreement and specific remaining disagreements further demonstrates why domain labels should remain reviewable rather than authoritative.

Confidentiality is another ethical consideration. The empirical material includes proprietary source identifiers and architecture knowledge. The public thesis therefore uses anonymized file and module identifiers and aggregated evidence, while the Ericsson-controlled research material retains proprietary details that cannot be released publicly. The evidence reported in Appendix A provides traceability while using anonymized identifiers and sanitized descriptions to avoid exposing proprietary source-code names, repository paths, architecture identifiers, or participant identities.

Human-evaluation data were reported in a way that limits unnecessary disclosure of participant identity. Practitioner and expert findings are presented using generic identifiers and aggregated summaries rather than personal names. The practitioner-consent procedure is described in Section 3.9.1.

The societal relevance is indirect but real. Telecom software forms a part of communication infrastructure, so maintenance decisions occur in a context where operational consequences matter. However, this thesis does not measure service outcomes and therefore cannot claim that the artifact improves network reliability, availability, security, or service quality.

The sustainability implications are similarly limited by the study design. A more structured prioritization process could potentially help organizations direct finite maintenance effort toward selected areas, but the thesis does not measure saved engineering effort, economic benefit, energy consumption, or environmental impact. Such benefits should therefore be treated as potential implications rather than empirical outcomes.

From an organizational sustainability perspective, the method's reliance on existing repository history, maintainability measurements, and architecture knowledge may make it easier to reuse than a method requiring extensive new data collection. Yet, the ongoing effort required to maintain accurate domain mappings and review recommendations should also be considered.

The wider responsibility is therefore to use the artifact as transparent decision support under human oversight, not as automated authority.

5.11 Summary of Discussion

The findings support the artifact as transparent, context-specific planning support. The retained factors provide complementary technical and operational evidence, while the two queues distinguish immediate candidates from those requiring preparation. DomainAware largely preserved the technical shortlist and was preferred on average by practitioners, although HealthOnly remained competitive and absolute ratings were modest. Expert agreement supported much of the domain mapping while revealing classification uncertainty. The temporal findings apply only to the two evaluated endpoints. The resulting contribution is an auditable planning procedure whose assumptions and recommendations remain open to engineering review.

Chapter 6

Conclusions and Future Work

This chapter answers the research questions, states the contribution of the thesis, and identifies the main directions for future work.

6.1 Conclusions from the Research Questions

6.1.1 Relevant and Feasible Prioritization Factors

RQ1 asked:

> *Which prioritization factors are relevant and feasible for file-level refactoring-target selection in the selected telecom software context?*

The answer to RQ1 is that Code Health, development activity, and module-level domain criticality mapped to files were the three factors that were both relevant to the prioritization problem and feasible to operationalize in the selected Ericsson UPC uplink scope.

Code Health provided the maintainability-related signal, development activity provided the repository-history signal, and domain criticality provided the operational-context signal.

Change or temporal coupling, incident information, and consistent file-level test coverage were not included because sufficiently complete and reproducible evidence was not available across the intended candidate population.

The domain-criticality construct was additionally assessed by two Ericsson experts. The experts assigned the same HIGH, MEDIUM, or LOW label to 28 of the 31 represented modules. The resulting domain classifications nevertheless remained researcher-created inputs to the primary artifact.

6.1.2 Usefulness of the Domain-Aware Prioritization Artifact

RQ2 asked:

> *How useful is the resulting domain-aware refactoring-prioritization method compared with domain-agnostic baselines from quantitative and practitioner perspectives?*

RQ2 was evaluated using the quantitative baseline comparison, practitioner assessment, Stabilize-First review, and the two-endpoint temporal comparison.

At the April endpoint, DomainAware and HealthActivity shared eight of their ten selected files. The two DomainAware-only files in the MODERATE Code Health band were mapped to HIGH-criticality modules. The two HealthActivity-only files were mapped to MEDIUM-criticality modules.

In the practitioner evaluation, DomainAware received the most favourable mean rank of 1.50 and the highest overall mean rating of 2.95. DomainAware and HealthOnly each received two first-place votes.

The Stabilize-First mechanism produced a separate preparation and review queue, and the practitioner responses included both support for this separation and file-specific exceptions.

Across the January and April endpoints, DomainAware retained the same Top-10 membership. This result applies only to the two endpoints evaluated in this study.

Taken together, these results provide context-specific evidence that DomainAware can support pre-refactoring planning in the studied setting. They do not establish an objectively correct file ranking, universally optimal prioritization, general temporal stability, refactoring safety, or post-refactoring improvement.

6.2 Research and Practical Contribution

The research contribution of this thesis consists of three related elements.

First, the thesis provides a deterministic and auditable file-level pre-refactoring prioritization artifact that integrates three different forms of evidence: file-level maintainability through Code Health, file-level development history through non-merge development activity, and module-level domain criticality mapped to files. The artifact keeps the technical Base separate from the domain adjustment and translates the resulting prioritization into the two planning outputs Refactor-Now and Stabilize-First.

Second, the empirical evaluation provides design knowledge about how operational context can influence a technically grounded prioritization in the studied case. At the April endpoint, DomainAware and HealthActivity shared eight of ten selected files, while the full DomainAware procedure produced two different selections at the MODERATE Code Health boundary, with the DomainAware-only files mapped to HIGH-criticality modules and the HealthActivity-only files mapped to MEDIUM-criticality modules. The corresponding quantitative metrics changed only modestly in Debt Severity and Activity Alignment, while Criticality Coverage increased under the frozen researcher mapping. In this case, the complete domain-aware planning procedure therefore acted as a bounded contextual modification of the technical shortlist rather than replacing the underlying technical case. This observation is specific to the studied artifact configuration and software scope and is not claimed as a universal property of domain-aware prioritization.

Third, the thesis contributes a transparent way of representing and examining a contextual domain construct derived from architecture and HLD evidence. The independent expert assessment showed high overall agreement while also identifying specific classification and abstraction uncertainties. The contribution is therefore not only the use of domain information, but also the explicit treatment of that information as a reviewable engineering judgement rather than objective ground

truth.

The practical contribution is an inspectable planning procedure in which engineers can trace a recommendation from candidate eligibility through technical evidence, module-level context, scoring parameters, ranking rules, and planning-queue assignment. The artifact is therefore intended as transparent decision support under limited engineering capacity rather than as automated authority.

Taken together, the thesis contributes an industrial example of how repository-derived technical evidence and architecture-derived operational context can be integrated while keeping the assumptions, contextual judgements, and resulting planning decisions explicitly reviewable.

6.3 Future Work

The present study leaves four main areas for further research: broader replication, domain-criticality refinement, parameter sensitivity and artifact extension, and post-refactoring outcome evaluation.

6.3.1 Replication Across Software Contexts and Time

Future studies should apply the artifact to additional software components and, where possible, to different industrial contexts. Each replication should establish its own candidate population, domain mapping, and context-appropriate parameter configuration rather than automatically reusing the numerical values from this study.

The practitioner evaluation should also be repeated with a larger and more diverse participant group, including practitioners from additional teams or software scopes. Temporal evaluation should be extended beyond the January and April endpoints. Applying the fixed artifact across additional development periods or releases would provide a larger set of observations for evaluating changes in recommendation membership over time.

6.3.2 Strengthening the Domain-Criticality Construct

Future work should assess domain criticality with a larger group of experts having complementary knowledge of the software scope. Experts should first provide independent classifications. The independent assessments should be preserved, after which a jointly reviewed classification could be produced for modules where sufficient evidence supports agreement. The artifact could then be re-executed separately using the researcher mapping, the individual expert mappings, and any jointly reviewed mapping while keeping the technical inputs and remaining artifact configuration fixed. The resulting Refactor-Now and Stabilize-First memberships could then be compared.

Future studies should also examine whether module-level classification adequately captures the details of heterogeneous modules. Where appropriate architecture or feature-level evidence exists, selected modules could be evaluated using a more detailed classification unit.

6.3.3 Parameter Sensitivity and Artifact Extension

Future work should perform systematic sensitivity analysis for the researcher-designed artifact parameters.

The analysis should vary:

- the Code Health and Activity weights in the technical Base;
- the HIGH, MEDIUM, and LOW domain weights;
- the domain-amplification parameter β;
- the selection budget K;
- the LOW/MODERATE band allocation; and
- the P25 Stabilize-First threshold.

For each configuration, changes in candidate membership, selection boundaries, and the quantitative planning metrics should be recorded.

Additional prioritization signals could also be evaluated when sufficiently complete and traceable evidence becomes available. Candidate signals include dependency relationships, change coupling, test-confidence information, ownership information, and incident history.

If a sufficiently large and reliable history of practitioner prioritization decisions or refactoring outcomes becomes available, the deterministic artifact could also be compared with learned ranking approaches under the same candidate population and evaluation criteria.

6.3.4 Longitudinal Post-Refactoring Evaluation

The principal outcome question left unanswered by this thesis is what happens after prioritized candidates are actually refactored. Future longitudinal studies should implement selected refactorings under normal engineering controls and collect post-refactoring observations over an appropriate period.

Depending on data availability, the evaluation could measure subsequent:

- Code Health;
- maintenance effort;
- review effort;
- development activity;
- defect occurrence; and
- other relevant engineering or operational outcomes.

The study could compare outcomes for files prioritized by DomainAware with files selected using the baseline strategies or normal engineering planning. Stabilize-First could also be evaluated by recording the preparation activities performed before refactoring and the subsequent outcomes of those changes. Such evidence would be required to evaluate post-refactoring maintainability, engineering efficiency, reliability, or refactoring safety.

6.4 Final Conclusion

This thesis addressed the problem of selecting a bounded set of file-level refactoring candidates when technical evidence alone does not represent the operational context in which those files are used. In the selected Ericsson UPC uplink scope, Code Health, development activity, and module-level domain criticality could be operationalized together in a deterministic pre-refactoring planning artifact.

The main design insight from the studied case is that operational context can be introduced as an explicit and reviewable adjustment to technical evidence rather than as a substitute for it. DomainAware retained most of the candidates selected by the HealthActivity technical baseline while changing a limited number of positions toward files associated with HIGH-criticality modules. The practitioner evaluation provided context-specific evidence of planning usefulness, while the independent expert assessment supported the consistency of the domain-criticality construct and highlighted the need to keep contextual judgements open to engineering review.

The strongest conclusion supported by the study is therefore not that DomainAware is a universally superior refactoring strategy. Rather, the thesis demonstrates a transparent and auditable way to combine maintainability, development history, and operational context under limited engineering capacity while preserving the ability of engineers to inspect, challenge, and override the resulting planning recommendations.

References

[1] 3rd Generation Partnership Project (3GPP), "5G; Security architecture and procedures for 5G System," 3rd Generation Partnership Project, Technical Specification 3GPP TS 33.501, Release 18, 2024. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/133500_133599/133501/18.07.00_60/ts_133501v180700p.pdf

[2] ——, "5G; NR; Physical channels and modulation," 3rd Generation Partnership Project, Technical Specification 3GPP TS 38.211, Release 18, 2026. [Online]. Available: https://www.etsi.org/deliver/etsi_TS/138200_138299/138211/18.08.00_60/ts_138211v180800p.pdf

[3] J. Al Dallal, "Identifying refactoring opportunities in object-oriented code: A systematic literature review," *Information and Software Technology*, vol. 58, pp. 231–249, 2015. [Online]. Available: https://doi.org/10.1016/j.infsof.2014.08.002

[4] A. Aldaeej and C. Seaman, "A rule-based decision model to support technical debt decisions: A multiple case study of web and mobile app startups," *Information and Software Technology*, vol. 175, p. 107542, 2024. [Online]. Available: https://doi.org/10.1016/j.infsof.2024.107542

[5] R. Alfayez, R. Winn, W. Alwehaibi, E. Venson, and B. Boehm, "How SonarQube-identified technical debt is prioritized: An exploratory case study," *Information and Software Technology*, vol. 156, p. 107147, 2023. [Online]. Available: https://doi.org/10.1016/j.infsof.2023.107147

[6] N. S. R. Alves, T. S. Mendes, M. G. de Mendonça, R. O. Spínola, F. Shull, and C. Seaman, "Identification and management of technical debt: A systematic mapping study," *Information and Software Technology*, vol. 70, pp. 100–121, 2016. [Online]. Available: https://doi.org/10.1016/j.infsof.2015.10.008

[7] CodeScene, "Code Health – how easy is your code to maintain and evolve?" CodeScene Documentation. Accessed: 2026-02-04. [Online]. Available: https://codescene.io/docs/guides/technical/code-health.html

[8] ——, "CodeScene Terminology," CodeScene Documentation. Accessed: 2026-02-04. [Online]. Available: https://codescene.io/docs/terminology/codescene-terminology.html

[9] ——, "Technical Debt," CodeScene Documentation. Accessed: 2026-10-03. [Online]. Available: https://codescene.io/docs/guides/technical/hotspots.html

[10] J. Cohen, "Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit," *Psychological Bulletin*, vol. 70, no. 4, pp. 213–220, 1968. [Online]. Available: https://doi.org/10.1037/h0026256

[11] S. M. H. Dehaghani and N. Hajrahimi, "Which factors affect software projects maintenance cost more?" *Acta Informatica Medica*, vol. 21, no. 1, pp. 63–66, Mar. 2013. [Online]. Available: https://doi.org/10.5455/aim.2012.21.63-66

[12] European Union Agency for Cybersecurity (ENISA), "Security in 5G specifications," European Union Agency for Cybersecurity, Tech. Rep., 2021, accessed: 2026-02-04. [Online]. Available: https://doi.org/10.2824/30076

[13] D. Fang, Y. Qian, and R. Q. Hu, "Security for 5G mobile wireless networks," *IEEE Access*, vol. 6, pp. 4850–4874, 2018. [Online]. Available: https://doi.org/10.1109/ACCESS.2017.2779146

[14] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley Professional, 2018. [Online]. Available: https://martinfowler.com/books/refactoring.html

[15] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. [Online]. Available: https://doi.org/10.2307/25148625

[16] A. R. Hevner, "A three cycle view of design science research," *Scandinavian Journal of Information Systems*, vol. 19, no. 2, pp. 87–92, 2007. [Online]. Available: https://aisel.aisnet.org/sjis/vol19/iss2/4/

[17] ISO/IEC, "ISO/IEC 25010:2023 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model," International Organization for Standardization / International Electrotechnical Commission, International Standard ISO/IEC 25010:2023, 2023, accessed: 2026-02-04. [Online]. Available: https://www.iso.org/standard/78176.html

[18] R. Khan, P. Kumar, D. N. K. Jayakody, and M. Liyanage, "A survey on security and privacy of 5G technologies: Potential solutions, recent advancements, and future directions," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 196–248, 2020. [Online]. Available: https://doi.org/10.1109/COMST.2019.2933899

[19] V. Lenarduzzi, T. Besker, D. Taibi, A. Martini, and F. Arcelli Fontana, "A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools," *Journal of Systems and Software*, vol. 171, p. 110827, 2021. [Online]. Available: https://doi.org/10.1016/j.jss.2020.110827

[20] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015. [Online]. Available: https://doi.org/10.1016/j.jss.2014.12.027

[21] L. Liu, G. Lin, L. Zhu, Z. Yang, P. Song, X. Wang, and W. Hu, "Revisiting code smell severity prioritization using learning to rank techniques," *Expert Systems with Applications*, vol. 249, p. 123483, 2024. [Online]. Available: https://doi.org/10.1016/j.eswa.2024.123483

[22] A. Martini and J. Bosch, "An empirically developed method to aid decisions on architectural technical debt refactoring: AnaConDebt," in *Proceedings of the*

38th International Conference on Software Engineering Companion, 2016, pp. 31–40. [Online]. Available: https://doi.org/10.1145/2889160.2889224

[23] T. Mens and T. Tourwé, "A survey of software refactoring," *IEEE Transactions on Software Engineering*, vol. 30, no. 2, pp. 126–139, 2004. [Online]. Available: https://doi.org/10.1109/TSE.2004.1265817

[24] R. Moser, W. Pedrycz, and G. Succi, "A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction," in *Proceedings of the 30th International Conference on Software Engineering*, 2008, pp. 181–190. [Online]. Available: https://doi.org/10.1145/1368088.1368114

[25] F. Palomba, G. Bavota, M. Di Penta, F. Fasano, R. Oliveto, and A. De Lucia, "On the diffuseness and the impact on maintainability of code smells: A large scale empirical investigation," *Empirical Software Engineering*, vol. 23, no. 2, pp. 1188–1221, 2018. [Online]. Available: https://doi.org/10.1007/s10664-017-9535-z

[26] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. [Online]. Available: https://doi.org/10.2753/MIS0742-1222240302

[27] R. Rebouças de Almeida, R. do Nascimento Ribeiro, C. Treude, and U. Kulesza, "Business-driven technical debt prioritization: An industrial case study," in *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)*, 2021, pp. 74–83. [Online]. Available: https://doi.org/10.1109/TechDebt52882.2021.00017

[28] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009. [Online]. Available: https://doi.org/10.1007/s10664-008-9102-8

[29] A. Tornhill, *Your Code as a Crime Scene: Use Forensic Techniques to Arrest Defects, Bottlenecks, and Bad Design in Your Programs.* Raleigh, NC, USA: The Pragmatic Programmers, LLC, 2015. [Online]. Available: https://pragprog.com/titles/atcrime/your-code-as-a-crime-scene/

[30] A. Tornhill and M. Borg, "Code red: The business impact of code quality – a quantitative study of 39 proprietary production codebases," in *Proceedings of the International Conference on Technical Debt*, 2022, pp. 11–20. [Online]. Available: https://doi.org/10.1145/3524843.3528091

[31] R. J. Wieringa, *Design Science Methodology for Information Systems and Software Engineering.* Berlin, Germany: Springer, 2014. [Online]. Available: https://doi.org/10.1007/978-3-662-43839-8

Appendix A

Supporting Evidence and Reproducibility

A.1 Evidence, Anonymization, and Study Identifiers

This appendix contains the supporting evidence for the Method and Results. It includes architecture and module context, the frozen researcher domain mapping, artifact configuration, file-level inputs at both endpoints, file-to-module mappings, final selections, quantitative evaluation summaries, individual practitioner rankings and ratings, and complete expert classifications.

Together with the documented calculation and planning rules, the endpoint inputs support reconstruction of the reported computational results. The individual practitioner and expert records support recalculation of the numerical human-evaluation summaries.

To protect proprietary implementation information and participant identities, the thesis uses consistent anonymized identifiers. Source-code files are represented as F001-F200, architecture modules as M01-M50, practitioners as P1-P4, and independent domain experts as E1 and E2. The same identifiers are used consistently across the thesis and this appendix.

Table A.1 summarizes the principal identifiers and study populations.

Table A.1: Anonymized identifiers and study populations

Item	**Representation / count**
Source-code files	F001–F200
Architecture catalogue modules	M01–M50
Represented modules	31
Catalogue-only modules	19
Practitioners	P1–P4
Independent domain experts	E1–E2
April master population	200 files
April scoreable population	136 files
January master-mapped population	200 file identities
January scoreable population	133 files
April Stabilize-First population	28 files
January Stabilize-First population	33 files

The evidence presented in this appendix is intended to support external audit without exposing proprietary source-file names, internal repository paths, source

code, confidential architecture identifiers, or participant identities. More detailed proprietary research material remains within the Ericsson-controlled study environment where it cannot be included in the public thesis.

A.2 Architecture Scope and Anonymized Module Context

The architecture catalogue used in the study contains 50 modules. Thirty-one modules are represented by at least one file in the 200-file master population. The remaining 19 modules are retained as architecture-catalogue context but do not contribute files to the prioritization population because no file from those modules satisfied the study's scope, change-window, and artifact-type candidate criteria.

The absence of these 19 modules from the candidate population was therefore a consequence of candidate construction rather than domain criticality or manual researcher preference.

Table A.2 summarizes the represented and catalogue-only portions of the architecture catalogue.

Table A.2: Architecture catalogue coverage in the study population

Status	**Module identifiers**	**Count**
Represented	M01–M23, M25–M31, M40	31
Catalogue-only	M24, M32–M39, M41–M50	19
Total architecture catalogue	M01–M50	**50**

Domain criticality was constructed from module-level architecture and high-level design evidence rather than from file-level technical metrics. The available material described module responsibility, runtime role, possible service or functional consequences of failure, and documented fallback or workaround behaviour.

Table A.3 presents a sanitized summary of this context for the 31 represented modules. The table deliberately does not include the researcher-assigned HIGH, MEDIUM, or LOW labels. The operational evidence and the researcher classification are presented separately so that the evidence used to understand each module can be distinguished from the researchers' subsequent domain-criticality judgement.

The original proprietary architecture and HLD documents are not reproduced in the public thesis. The descriptions below are anonymized summaries retained for traceability to the domain-classification procedure. In the sanitized module descriptions, UE denotes user equipment, HARQ denotes Hybrid Automatic Repeat Request, and MIMO denotes Multiple-Input Multiple-Output.

Table A.3: Anonymized architecture/HLD-derived context for the 31 represented modules

Module ID	Anonymized architecture/HLD-derived context
M01	**Responsibility and runtime role:** Per-UE uplink scheduling and UE-state handling on the active per-slot scheduling path. It processes scheduling triggers, HARQ state, buffer state, configuration, and UE lifecycle information. **Failure or service consequence:** Loss or corruption can prevent affected UEs from receiving uplink grants, stall uplink data, disrupt HARQ behaviour, and potentially lead to connection timeouts. **Fallback / workaround:** No documented workaround information was identified.
M02	**Responsibility and runtime role:** Per-cell processing of interference measurements, timer events, energy-saving state, power-imbalance information, and sounding-reference-signal utilization. **Failure or service consequence:** Incorrect or unavailable cell-level interference information can degrade scheduling decisions, link adaptation, block-error performance, and cell throughput. **Fallback / workaround:** No documented workaround information was identified.
M03	**Responsibility and runtime role:** Per-cell Random Access processing for initial access, preamble handling, message scheduling, overload protection, retransmission, and transfer of successfully established UEs into normal scheduling. **Failure or service consequence:** Failure can prevent new UE attachment, handover-related access, and beam failure recovery, although already-connected UEs may continue operating. **Fallback / workaround:** No documented workaround information was identified.
M04	**Responsibility and runtime role:** Secondary-cell uplink scheduling support for carrier aggregation, including resource allocation, scheduling requests, HARQ handling, configuration, and timing synchronization. **Failure or service consequence:** Loss of the secondary-cell path removes carrier-aggregation uplink benefit and reduces affected UE throughput, while primary-cell scheduling continues. **Fallback / workaround:** Documented fallback to primary-cell-only uplink scheduling; basic connectivity is maintained.

Table A.3 – continued from previous page

Module ID	Anonymized architecture/HLD-derived context
M05	**Responsibility and runtime role:** Serial cell setup, reconfiguration, release, and related configuration-state management used to establish runtime state for other modules. **Failure or service consequence:** Failure can prevent cell activation or later reconfiguration. Already-configured active scheduling can continue until further configuration is required. **Fallback / workaround:** No documented workaround information was identified.
M06	**Responsibility and runtime role:** Parallel dispatch and control logic relaying events between shared control logic and per-UE/per-cell actors, including timer events and HARQ garbage-collection triggers. **Failure or service consequence:** Loss can stop timer-based triggers, HARQ cleanup, and trace-related processing. Already-triggered scheduling can continue, while new timer-driven processing may be lost. **Fallback / workaround:** No documented workaround information was identified.
M07	**Responsibility and runtime role:** Scheduling-request construction shared across UE processing and used on the per-slot scheduling path immediately before requests are passed to the scheduler. **Failure or service consequence:** Failure prevents construction of scheduling requests and can therefore prevent uplink grants for affected UEs. **Fallback / workaround:** No documented workaround information was identified.
M08	**Responsibility and runtime role:** Authoritative per-UE runtime-session state containing scheduling, buffer, HARQ, DRX, bearer, sounding-reference-signal, carrier-aggregation, and configured-grant information. **Failure or service consequence:** Corrupted or inconsistent state can cause incorrect scheduling decisions, ranging from degraded scheduling to UE disconnection depending on the affected state. **Fallback / workaround:** No documented workaround information was identified.
M09	**Responsibility and runtime role:** Cell-wide performance-measurement counter handling used during active operation for monitoring and network-performance analysis. **Failure or service consequence:** Loss or corruption affects performance visibility and reporting but does not directly prevent scheduling. **Fallback / workaround:** Scheduling continues without the affected performance visibility; alternative monitoring may partially compensate.

Table A.3 – continued from previous page

Module ID	Anonymized architecture/HLD-derived context
M10	**Responsibility and runtime role:** Shared utilities used by several scheduling and configuration paths. The module contains heterogeneous responsibilities ranging from per-slot computations to configuration-time helpers. **Failure or service consequence:** Impact depends on the affected utility. Failure of critical-path calculations can disrupt scheduling, while failures of supporting utilities can have more localized effects. **Fallback / workaround:** No single module-wide workaround was documented.
M11	**Responsibility and runtime role:** Shared cell-configuration and runtime state written during setup or reconfiguration and read by scheduling logic during active operation. **Failure or service consequence:** Incorrect state can cause cell-wide use of wrong limits, stale configuration, or incorrect feature parameters across multiple scheduling decisions. **Fallback / workaround:** No documented workaround information was identified.
M12	**Responsibility and runtime role:** Cell-level runtime data structures containing relations, measurement state, timer configuration, and identity-related information used during active processing and configuration. **Failure or service consequence:** Corrupted cell-level state can affect all UEs associated with the cell and can degrade scheduling or cause broader cell-level failures. **Fallback / workaround:** No documented workaround information was identified.
M13	**Responsibility and runtime role:** Shared HARQ-process pool allocation, release, occupancy tracking, and process handle management on the scheduling path. **Failure or service consequence:** Pool exhaustion or leakage can eventually prevent allocation of HARQ processes and therefore block new uplink scheduling. **Fallback / workaround:** HARQ garbage collection can detect and release leaked processes, recovering pool capacity.
M14	**Responsibility and runtime role:** HARQ feedback handling that determines retransmission or process release after ACK, NACK, or DTX feedback. **Failure or service consequence:** Incorrect decisions can cause unnecessary retransmissions or failure to retransmit lost data. **Fallback / workaround:** Higher-layer retransmission mechanisms can provide recovery, with increased latency.

Table A.3 – continued from previous page

Module ID	Anonymized architecture/HLD-derived context
M15	**Responsibility and runtime role:** Parsing of MAC control elements used to update buffer status, power information, identifiers, and other inputs required by scheduling logic. **Failure or service consequence:** Incorrect parsing can produce wrong buffer tracking, over- or under-scheduling, or incorrect power-control input. **Fallback / workaround:** No documented workaround information was identified.
M16	**Responsibility and runtime role:** Configured-grant activation, deactivation, modification, and interaction with dynamic uplink scheduling. **Failure or service consequence:** Incorrect configured-grant state can lead to missed grants or conflicts with dynamic scheduling, increasing latency or packet loss for affected traffic. **Fallback / workaround:** Dynamic scheduling can continue to serve UEs, with additional control-channel usage and scheduling latency.
M17	**Responsibility and runtime role:** Configured-grant state-machine transitions driven by configuration commands, timer events, and HARQ feedback. **Failure or service consequence:** Incorrect state transitions can leave configured grants incorrectly active, inactive, or suspended, affecting resource use and per-UE behaviour. **Fallback / workaround:** Dynamic scheduling remains available; configured-grant efficiency benefits are lost.
M18	**Responsibility and runtime role:** Uplink coordinated multipoint processing, including cooperating-cell selection, measurement filtering, and calculations supporting joint reception decisions. **Failure or service consequence:** Incorrect selection or stale measurements can remove coordinated-reception gains and degrade cell-edge UE throughput. **Fallback / workaround:** Scheduling continues without the coordinated-multipoint benefit; baseline uplink operation is retained.
M19	**Responsibility and runtime role:** Interference-plus-noise processing that filters and stores measurements used for modulation, coding, and interference-aware scheduling decisions. **Failure or service consequence:** Stale or incorrect values can cause overly conservative or overly aggressive link adaptation and reduce scheduling efficiency. **Fallback / workaround:** Scheduling continues, although modulation and coding selection may be suboptimal.

Table A.3 – continued from previous page

Module ID	Anonymized architecture/HLD-derived context
M20	**Responsibility and runtime role:** Closed-loop uplink power-control computation based on received link quality, target values, and power-headroom information. **Failure or service consequence:** Incorrect control can cause excessive transmit power, insufficient power, or unstable link quality. **Fallback / workaround:** Open-loop power control remains functional, while closed-loop refinement is lost.
M21	**Responsibility and runtime role:** Validation of system-constant configuration values against allowed ranges and cross-parameter consistency constraints during setup and reconfiguration. **Failure or service consequence:** Failure can allow invalid configuration values to enter runtime processing. **Fallback / workaround:** No documented workaround information was identified.
M22	**Responsibility and runtime role:** Trace-data recording of scheduling decisions, state changes, and measurements for offline troubleshooting. **Failure or service consequence:** Loss affects post-event diagnostic information but does not directly affect scheduling correctness. **Fallback / workaround:** Scheduling continues without trace recording; other troubleshooting methods may partially compensate.
M23	**Responsibility and runtime role:** Timer-wheel handling for delayed UE processing such as scheduling retries, DRX-related timing, and postponed triggers. **Failure or service consequence:** Failure can prevent delayed triggers from being revisited, while currently active triggers can continue. **Fallback / workaround:** New events can continue to trigger processing, and some UEs can recover when new runtime events occur.
M25	**Responsibility and runtime role:** Uplink-gain estimation used as an input to scheduling-weight calculations. **Failure or service consequence:** Incorrect estimates can produce suboptimal scheduling-priority decisions. **Fallback / workaround:** Scheduling remains functional with potentially suboptimal priority ordering.
M26	**Responsibility and runtime role:** Evaluation of UE suitability for multi-user MIMO pairing during scheduling. **Failure or service consequence:** Incorrect evaluation can cause missed pairing opportunities or unsuitable pairing decisions. **Fallback / workaround:** Scheduling can continue in single-user mode; multi-user MIMO throughput gains are lost.

Table A.3 – continued from previous page

Module ID	Anonymized architecture/HLD-derived context
M27	**Responsibility and runtime role:** External carrier-aggregation client-side state used during secondary-cell configuration and coordination. **Failure or service consequence:** Incorrect or missing state can prevent carrier-aggregation scheduling toward the partner component. **Fallback / workaround:** Primary-cell scheduling continues; the affected secondary-carrier benefit is lost.
M28	**Responsibility and runtime role:** External carrier-aggregation server-side state used to process incoming carrier-aggregation configuration and coordination. **Failure or service consequence:** Incorrect or missing state can prevent incoming carrier-aggregation requests from being processed correctly. **Fallback / workaround:** Primary-cell scheduling continues; the affected secondary-carrier benefit is lost.
M29	**Responsibility and runtime role:** Rank-estimation adjustment for UEs operating in dual-active connectivity scenarios. **Failure or service consequence:** Incorrect adjustment can lead to over- or under-scheduling of affected UEs. **Fallback / workaround:** Scheduling continues with potentially suboptimal rank; UEs outside the dual-active case are unaffected.
M30	**Responsibility and runtime role:** Maximum UE transmit-power calculation used by power-control and resource-allocation decisions. **Failure or service consequence:** Incorrect values can distort power-headroom estimation and associated resource-allocation decisions. **Fallback / workaround:** Conservative scheduling can maintain link operation at the cost of throughput.
M31	**Responsibility and runtime role:** Control-channel aggregation-level estimation based on channel-quality information during scheduling-request generation. **Failure or service consequence:** Incorrect aggregation estimates can increase decoding failure or waste control resources. **Fallback / workaround:** Control information remains transmittable; insufficient aggregation can lead to subsequent retry through the scheduling-request path.

Table A.3 – continued from previous page

Module ID	Anonymized architecture/HLD-derived context
M40	**Responsibility and runtime role:** Shared radio-resource-management computation library containing multiple scheduling-related calculations. Individual functions range from per-slot critical processing to less frequent supporting computations. **Failure or service consequence:** The consequence depends on the affected function. Critical calculations can directly degrade scheduling decisions, whereas supporting functions have more localized effects. **Fallback / workaround:** Many functions support optimization; degraded values can result in suboptimal but still functional scheduling.

The sanitized evidence in Table A.3 provides the module-level context used in the domain-classification procedure. It should not itself be interpreted as a HIGH, MEDIUM, or LOW classification. The corresponding frozen researcher classifications are reported separately in Section A.3.

A.3 Frozen Researcher Domain-Criticality Mapping

Table A.4 records the complete frozen module-level mapping used in the primary DomainAware analysis.

Table A.4: Frozen researcher domain-criticality mapping

Module ID	Researcher label	Domain weight
M01	HIGH	1.0
M02	HIGH	1.0
M03	HIGH	1.0
M04	HIGH	1.0
M05	HIGH	1.0
M06	HIGH	1.0
M07	HIGH	1.0
M08	HIGH	1.0
M09	HIGH	1.0
M10	MEDIUM	0.5
M11	HIGH	1.0
M12	HIGH	1.0
M13	MEDIUM	0.5
M14	MEDIUM	0.5
M15	MEDIUM	0.5
M16	MEDIUM	0.5
M17	MEDIUM	0.5

Table A.4 – continued from previous page

Module ID	Researcher label	Domain weight
M18	MEDIUM	0.5
M19	MEDIUM	0.5
M20	MEDIUM	0.5
M21	MEDIUM	0.5
M22	LOW	0.2
M23	MEDIUM	0.5
M25	LOW	0.2
M26	LOW	0.2
M27	LOW	0.2
M28	LOW	0.2
M29	LOW	0.2
M30	LOW	0.2
M31	LOW	0.2
M40	MEDIUM	0.5

The frozen researcher mapping contains 11 HIGH, 12 MEDIUM, and 8 LOW represented modules. This is the mapping used in the reported January and April DomainAware analyses and retained as the fixed reference for the independent expert comparison.

A.4 Frozen Artifact Configuration

Table A.5 records the frozen configuration used in the reported analyses. The definitions, formulas, rationale, and calculation procedure are provided in Sections 3.6.2–3.6.6; they are not repeated here.

Table A.5: Frozen artifact configuration used in the final analyses

Configuration item	**Frozen value / rule**
LOW Code Health band	$CH < 4$
MODERATE Code Health band	$4 \leq CH < 9$
OUT_OF_SCOPE	$CH \geq 9$
NEEDS_REVIEW	No valid numeric Code Health
Development Activity	Non-merge file-changing commit count
Technical Base weights	HealthRisk = 0.5; ActivityNorm = 0.5
Domain weights	HIGH = 1.0; MEDIUM = 0.5; LOW = 0.2
Domain amplification	$\beta = 0.5$
Resulting domain multipliers	HIGH = 1.50; MEDIUM = 1.25; LOW = 1.10
Refactor-Now budget	$K = 10$
Band allocation	5 LOW + 5 MODERATE
Stabilize-First threshold	Endpoint-specific floor-based P25 of scoreable Activity
Tie handling	Ascending anonymized File_ID for an exact selection-boundary tie
Numerical convention	Four decimals for stored derived values; full-precision activity-normalization denominator

A.5 Endpoint Population and Eligibility Results

Table A.6 summarizes the population, eligibility, Activity, and Stabilize-First results for both endpoints. The complete file-level Code Health and Activity inputs, together with their module mappings, are reported below.

Table A.6: Population and eligibility results at the April and January endpoints

Result	**April**	**January**
Master file identities	200	200
Numeric Code Health available	196	192
LOW Code Health files	26	26
MODERATE Code Health files	110	107
OUT_OF_SCOPE files	60	59
Present files without numeric Code Health	4	4
Files not present at endpoint	0	4
Scoreable LOW + MODERATE files	136	133
Maximum Activity	48	27
P25 Activity threshold	2	1
Stabilize-First candidates	28	33

Table A.7: Domain-label distribution in the April scoreable population

Inherited domain label	**Scoreable files**
HIGH	109
MEDIUM	24
LOW	3
Total	**136**

Complete file-level endpoint inputs and file-to-module mapping. The tables below report the file-level inputs and module mappings. Each entry is identified by `File_ID/Module_ID`. April (A) and January (J) values are written as Code Health/Activity, where Activity is the raw number of non-merge file-changing commits in the corresponding observation window. At the January endpoint, F001-F004 were present but had no numeric Code Health. F047, F060, F087, and F199 were not present. A dash denotes a present file without numeric Code Health, while `NP` denotes a file absent at the endpoint. Neither group was eligible for scoring.

ID/Module	A (CH/Act)	J (CH/Act)	ID/Module	A (CH/Act)	J (CH/Act)
F001/M02	–/2	–/2	F026/M15	4.21/4	4.80/0
F002/M01	–/2	–/1	F027/M09	3.00/19	3.19/14
F003/M06	–/1	–/1	F028/M20	7.95/5	8.00/2
F004/M05	–/1	–/0	F029/M07	6.00/12	6.01/9
F005/M02	10.00/2	10.00/1	F030/M22	5.38/1	5.38/1
F006/M02	5.29/6	5.53/3	F031/M23	7.95/2	7.95/2
F007/M02	6.22/1	6.22/1	F032/M08	4.23/25	4.24/14
F008/M02	10.00/2	10.00/1	F033/M18	4.48/10	3.06/5
F009/M02	6.72/6	6.85/4	F034/M10	4.55/10	4.57/6
F010/M02	10.00/2	10.00/1	F035/M21	10.00/30	10.00/14
F011/M02	5.68/8	5.67/5	F036/M11	8.19/6	8.45/4
F012/M02	3.69/14	3.69/8	F037/M03	10.00/1	10.00/1
F013/M29	9.28/3	9.38/1	F038/M03	9.38/1	9.38/0
F014/M30	7.61/1	7.62/0	F039/M03	4.44/2	4.44/2
F015/M12	4.70/2	4.70/1	F040/M03	8.82/1	8.82/0
F016/M16	4.98/11	4.94/6	F041/M03	7.00/1	7.04/0
F017/M17	2.07/5	3.07/2	F042/M03	3.32/5	3.18/1
F018/M31	9.68/1	9.68/0	F043/M03	9.68/2	9.68/1
F019/M25	9.06/4	9.10/3	F044/M03	8.57/9	8.52/8
F020/M26	8.87/2	9.24/1	F045/M40	9.38/1	9.38/1
F021/M27	9.68/1	9.68/1	F046/M40	9.31/2	9.68/1
F022/M28	9.68/1	9.68/1	F047/M40	10.00/1	NP/0
F023/M14	8.34/3	8.22/1	F048/M40	3.79/10	4.29/6
F024/M13	6.21/1	6.21/1	F049/M40	7.49/1	7.49/1
F025/M19	7.78/3	7.78/2	F050/M40	10.00/1	10.00/1

ID/Module	A (CH/Act)	J (CH/Act)	ID/Module	A (CH/Act)	J (CH/Act)
F051/M40	5.23/7	5.41/4	F076/M04	4.19/7	4.23/4
F052/M40	6.21/1	6.21/1	F077/M04	4.94/5	4.98/3
F053/M40	5.55/7	5.54/4	F078/M04	7.06/1	7.11/0
F054/M40	6.45/2	6.45/1	F079/M04	5.98/3	5.98/3
F055/M40	7.33/4	8.53/1	F080/M04	7.45/4	7.47/2
F056/M40	9.01/4	9.38/3	F081/M04	9.24/1	9.24/0
F057/M40	9.31/1	9.31/0	F082/M04	9.59/1	9.59/1
F058/M40	6.52/1	6.52/1	F083/M04	5.81/12	5.84/10
F059/M40	9.23/1	9.23/0	F084/M04	2.41/1	2.41/1
F060/M40	9.68/1	NP/0	F085/M04	7.05/7	7.05/6
F061/M40	7.55/4	7.55/4	F086/M04	3.59/14	3.64/7
F062/M40	2.98/3	3.04/1	F087/M04	9.01/1	NP/0
F063/M40	6.96/4	6.96/4	F088/M01	7.01/2	7.01/2
F064/M40	7.23/4	8.13/2	F089/M01	9.92/1	9.92/1
F065/M40	4.57/7	8.11/3	F090/M01	6.28/17	6.29/12
F066/M40	8.45/1	8.45/1	F091/M01	8.66/2	8.67/1
F067/M04	9.38/2	9.38/1	F092/M01	9.24/2	9.25/1
F068/M04	7.00/9	7.06/5	F093/M01	9.68/2	9.68/0
F069/M04	8.10/2	8.12/1	F094/M01	8.46/5	8.47/4
F070/M04	5.11/3	5.37/2	F095/M01	7.64/7	7.64/5
F071/M04	4.82/12	4.94/7	F096/M01	5.55/15	5.67/10
F072/M04	8.51/2	8.51/2	F097/M01	6.52/13	7.00/8
F073/M04	2.57/12	2.59/6	F098/M01	8.72/3	8.72/2
F074/M04	8.93/2	8.93/2	F099/M01	9.11/1	9.11/0
F075/M04	5.42/3	5.42/3	F100/M01	6.36/13	7.61/9

ID/Module	A (CH/Act)	J (CH/Act)	ID/Module	A (CH/Act)	J (CH/Act)
F101/M01	6.08/13	6.12/6	F126/M01	5.80/7	5.80/6
F102/M01	9.14/1	9.14/0	F127/M01	2.85/25	2.91/16
F103/M01	1.53/29	1.53/16	F128/M01	9.24/2	9.24/2
F104/M01	3.44/16	3.45/10	F129/M01	9.19/8	9.20/7
F105/M01	8.22/9	6.51/5	F130/M01	8.80/8	8.81/7
F106/M01	8.44/6	8.73/2	F131/M01	9.31/2	9.31/2
F107/M01	7.34/6	7.37/3	F132/M01	6.46/8	6.47/4
F108/M01	7.30/7	7.34/5	F133/M01	9.08/2	9.23/1
F109/M01	3.10/21	3.46/14	F134/M01	8.47/4	8.53/3
F110/M01	1.97/43	1.88/27	F135/M01	8.97/3	8.98/1
F111/M01	9.68/5	9.68/4	F136/M01	10.00/1	10.00/1
F112/M01	9.08/6	9.08/5	F137/M01	10.00/1	10.00/1
F113/M01	4.18/8	4.34/6	F138/M01	8.60/3	8.65/0
F114/M01	7.08/5	7.37/3	F139/M01	8.90/2	8.90/0
F115/M01	8.56/3	8.56/2	F140/M01	8.83/3	8.83/0
F116/M01	9.57/2	9.57/2	F141/M01	8.72/2	8.72/0
F117/M01	5.49/7	5.53/2	F142/M01	9.43/3	9.44/0
F118/M01	9.53/1	9.53/1	F143/M01	8.91/2	8.91/0
F119/M01	8.71/2	8.71/1	F144/M01	9.38/1	9.40/0
F120/M01	9.09/1	9.09/1	F145/M01	9.68/2	10.00/1
F121/M01	8.39/3	8.39/2	F146/M01	6.45/4	6.45/3
F122/M01	3.23/15	3.45/10	F147/M01	7.06/9	7.09/4
F123/M01	8.97/6	8.98/4	F148/M01	9.25/3	9.25/3
F124/M01	8.35/5	8.36/3	F149/M01	10.00/2	10.00/1
F125/M01	5.87/10	5.97/6	F150/M01	7.39/2	7.40/0

ID/Module	A (CH/Act)	J (CH/Act)	ID/Module	A (CH/Act)	J (CH/Act)
F151/M01	10.00/1	10.00/1	F176/M01	6.50/1	6.50/1
F152/M01	10.00/1	10.00/1	F177/M01	2.10/2	2.10/1
F153/M01	5.38/7	5.59/4	F178/M01	1.19/3	1.19/1
F154/M01	7.12/5	7.13/4	F179/M01	4.31/1	4.31/1
F155/M01	3.50/4	3.49/2	F180/M01	9.84/1	9.84/1
F156/M01	4.33/18	4.60/11	F181/M01	5.86/13	6.29/9
F157/M01	6.41/4	6.54/0	F182/M01	1.85/20	2.06/9
F158/M01	7.22/2	7.22/2	F183/M01	10.00/1	10.00/1
F159/M01	9.68/2	9.68/2	F184/M01	8.09/3	8.10/2
F160/M01	3.18/19	3.22/12	F185/M01	1.27/48	1.29/24
F161/M01	9.68/1	9.68/1	F186/M01	7.38/4	9.29/3
F162/M01	8.34/3	8.63/0	F187/M06	10.00/1	10.00/1
F163/M01	7.59/3	9.68/0	F188/M06	6.64/4	6.67/2
F164/M01	7.89/5	8.72/1	F189/M05	10.00/1	10.00/1
F165/M01	6.11/2	6.11/0	F190/M05	5.48/2	5.48/0
F166/M01	8.73/1	8.73/0	F191/M05	5.10/10	5.19/5
F167/M01	9.68/1	9.68/1	F192/M05	9.42/1	9.42/1
F168/M01	5.51/2	5.55/1	F193/M05	3.97/23	3.94/13
F169/M01	10.00/1	10.00/1	F194/M05	6.84/2	6.91/0
F170/M01	2.33/25	2.17/13	F195/M05	8.22/2	8.22/1
F171/M01	2.06/15	2.08/8	F196/M05	9.92/1	9.92/1
F172/M01	9.38/2	9.38/0	F197/M05	4.66/3	4.66/2
F173/M01	5.75/3	6.16/1	F198/M05	9.40/1	9.40/1
F174/M01	1.54/44	1.75/20	F199/M05	10.00/2	NP/0
F175/M01	3.59/1	3.59/1	F200/M05	7.31/3	7.10/0

A.6 Prioritization Output Evidence

The tables in this section record the anonymized Refactor-Now selections and Stabilize-First memberships produced at the two evaluated endpoints.

Worked example. At the April endpoint, F185 had Code Health 1.27, Activity 48, and HIGH domain criticality. Applying the documented calculations gives HealthRisk 0.9700, ActivityNorm 1.0000, Base 0.9850, and DomainAwareScore 1.4775. Its Activity exceeds the April P25 threshold of 2, and it was selected in the LOW-band Refactor-Now list. In contrast, F179 had Code Health 4.31, Activity 1, and

HIGH domain criticality. It therefore met the Stabilize-First condition and was excluded from immediate DomainAware selection. These examples use the inputs in Appendix A.5 and the outputs in Tables A.8 and A.10.

Table A.8: April endpoint Refactor-Now Top-10 selections

Strategy	**LOW-band selections**	**MODERATE-band selections**
DomainAware	F185, F174, F110, F103, F170	F032, F156, F071, F113, F096
HealthOnly	F178, F185, F103, F174, F182	F113, F076, F026, F032, F179
ActivityOnly	F185, F174, F110, F103, F127	F032, F156, F090, F096, F097
HealthActivity	F185, F174, F110, F103, F170	F032, F156, F071, F033, F034

Table A.9: January endpoint Refactor-Now Top-10 selections

Strategy	**LOW-band selections**	**MODERATE-band selections**
DomainAware	F185, F110, F174, F103, F170	F032, F156, F113, F096, F071
HealthOnly	F178, F185, F103, F174, F110	F076, F032, F048, F179, F113
ActivityOnly	F110, F185, F174, F103, F127	F032, F090, F156, F083, F096
HealthActivity	F185, F110, F174, F103, F170	F032, F156, F048, F113, F096

The Stabilize-First File_IDs in Table A.10 are displayed in ascending identifier order for readability; the displayed order does not represent a ranking.

Table A.10: Complete anonymized Stabilize-First membership at the two endpoints

Endpoint	**Stabilize-First File_IDs**
April (28 files)	F007, F015, F039, F040, F041, F069, F072, F074, F078, F084, F088, F091, F119, F139, F141, F143, F150, F158, F165, F166, F168, F175, F176, F177, F179, F190, F194, F195
January (33 files)	F007, F015, F040, F041, F042, F069, F078, F084, F091, F119, F135, F138, F139, F140, F141, F143, F150, F157, F162, F164, F165, F166, F168, F173, F175, F176, F177, F178, F179, F190, F194, F195, F200

A.7 Quantitative Evaluation Evidence

This section consolidates the quantitative strategy metrics and selection-overlap results used in the evaluation.

Table A.11: Quantitative planning metrics at the April and January endpoints

Endpoint	**Strategy**	**Criticality Coverage**	**Debt Severity**	**Activity Alignment**	**Risk Separation**
April	DomainAware	1.0000	0.7583	0.8191	0
	HealthOnly	0.9500	0.7944	0.6518	1
	ActivityOnly	1.0000	0.7103	0.8389	0
	HealthActivity	0.9000	0.7661	0.8146	0
January	DomainAware	1.0000	0.7510	0.8008	0
	HealthOnly	0.9500	0.7883	0.6610	2
	ActivityOnly	1.0000	0.7111	0.8347	0
	HealthActivity	0.9500	0.7582	0.7968	0

Table A.12: Pairwise overlap among April endpoint Top-10 selections

Strategy pair	**Common files**	**Jaccard**
DomainAware – HealthActivity	8	0.6667
DomainAware – ActivityOnly	7	0.5385
DomainAware – HealthOnly	5	0.3333
HealthActivity – ActivityOnly	6	0.4286
HealthActivity – HealthOnly	4	0.2500
ActivityOnly – HealthOnly	4	0.2500

Table A.13: Top-10 overlap between the January and April endpoints

Strategy	**Common files**	**Jaccard**
DomainAware	10/10	1.0000
HealthOnly	8/10	0.6667
ActivityOnly	9/10	0.8182
HealthActivity	7/10	0.5385

A.8 Practitioner Evaluation Evidence

The following tables report the anonymized practitioner rankings and ratings underlying the numerical summaries in Section 4.7. Qualitative observations and Stabilize-First feedback are reported in Section 4.7.3.

Table A.14: Practitioner strategy-ranking evidence

Strategy	Rank-1 votes	Mean rank
DomainAware	2	1.50
HealthOnly	2	1.75
HealthActivity	0	2.75
ActivityOnly	0	4.00

Table A.15: Individual practitioner rankings and ratings

Pract.	Strat.	Rank	RC	AV	AP	SR	OA
P1	A	1	3	3	4	4	4
P1	B	2	2	2	3	3	3
P1	C	4	1	1	3	1	1
P1	D	3	1	2	3	2	2
P2	A	2	2	3	2	2	3
P2	B	1	3	2	3	4	4
P2	C	4	1	3	1	1	2
P2	D	3	1	3	1	2	2
P3	A	2	2	3	2	2	2
P3	B	1	3	1	4	4	4
P3	C	4	1	2	2	1	2
P3	D	3	2	3	1	3	1
P4	A	1	3	3	4	4	4
P4	B	3	2	2	3	3	2
P4	C	4	1	1	3	1	1
P4	D	2	2	2	4	2	3

Note. Rank uses 1 = most suitable and 4 = least suitable. The five ratings use a 1–5 agreement scale. RC = reasonable candidates; AV = avoids inappropriate immediate refactoring; AP = actionable planning; SR = scheduling realism; OA = mission/operational alignment. Strategy A = DomainAware, B = HealthOnly, C = ActivityOnly, and D = HealthActivity.

A.9 Independent Expert Validation Evidence

Both experts provided a HIGH, MEDIUM, or LOW classification for all 31 represented modules. Table A.16 records the complete anonymized comparison between the frozen researcher mapping and the two independent expert assessments.

Table A.16: Complete comparison of researcher and expert domain classifications

Module ID	Researcher	Expert 1	Expert 2
M01	HIGH	HIGH	HIGH
M02	HIGH	HIGH	HIGH
M03	HIGH	HIGH	HIGH
M04	HIGH	HIGH	HIGH
M05	HIGH	HIGH	HIGH
M06	HIGH	MEDIUM	MEDIUM
M07	HIGH	MEDIUM	MEDIUM
M08	HIGH	HIGH	HIGH
M09	HIGH	HIGH	HIGH
M10	MEDIUM	MEDIUM	MEDIUM
M11	HIGH	HIGH	HIGH
M12	HIGH	HIGH	HIGH
M13	MEDIUM	HIGH	MEDIUM
M14	MEDIUM	HIGH	MEDIUM
M15	MEDIUM	MEDIUM	MEDIUM
M16	MEDIUM	MEDIUM	MEDIUM
M17	MEDIUM	MEDIUM	MEDIUM
M18	MEDIUM	MEDIUM	MEDIUM
M19	MEDIUM	MEDIUM	MEDIUM
M20	MEDIUM	MEDIUM	MEDIUM
M21	MEDIUM	MEDIUM	MEDIUM
M22	LOW	LOW	LOW
M23	MEDIUM	MEDIUM	MEDIUM
M25	LOW	LOW	LOW
M26	LOW	LOW	LOW
M27	LOW	LOW	LOW
M28	LOW	LOW	LOW
M29	LOW	LOW	LOW
M30	LOW	MEDIUM	MEDIUM
M31	LOW	MEDIUM	MEDIUM
M40	MEDIUM	LOW	HIGH

One additional qualitative observation from the completed expert material concerned M01: Expert 2 indicated that one module-level classification may not adequately represent all files mapped to that module because individual files can implement different functionality.

Table A.17: Domain-label distributions across the three assessments

Assessment	HIGH	MEDIUM	LOW	Total
Frozen researcher mapping	11	12	8	31
Expert 1	11	13	7	31
Expert 2	10	15	6	31

Table A.18: Module-level and propagated file-level agreement

Comparison	Module agreement	Raw agreement	Weighted κ	Propagated file agreement
Expert 1 - Expert 2	28/31	90.32%	0.8347	176/200 (88.00%)
Researcher - Expert 1	24/31	77.42%	0.7264	170/200 (85.00%)
Researcher - Expert 2	26/31	83.87%	0.7984	172/200 (86.00%)
All three classifications	24/31	77.42%	–	170/200 (85.00%)

Table A.19: Modules showing disagreement in at least one comparison

Module ID	Researcher	Expert 1	Expert 2
M06	HIGH	MEDIUM	MEDIUM
M07	HIGH	MEDIUM	MEDIUM
M13	MEDIUM	HIGH	MEDIUM
M14	MEDIUM	HIGH	MEDIUM
M30	LOW	MEDIUM	MEDIUM
M31	LOW	MEDIUM	MEDIUM
M40	MEDIUM	LOW	HIGH

Appendix B

Generative AI Disclosure

AI-assisted and language-support tools were used during the preparation of this thesis for supporting activities related to literature discovery, literature review, language editing, and visual presentation.

Scopus AI helped identify potentially relevant publications and literature, and NotebookLM helped summarize selected reference material during the literature-review process. The authors read the original publications and checked the claims and citations against them. AI search results and summaries were not cited as scientific sources.

Grammarly was used to proofread author-written text. ChatGPT (GPT-5.6 Sol, OpenAI) was used to suggest improvements to the grammar, clarity, and flow of author-written passages, and to assist with LaTeX formatting. These suggestions were reviewed before inclusion and were modified, rejected, or accepted based on the intended meaning of the text.

Kiro AI assisted with selected illustrations and diagrams. The authors determined their scientific content and reviewed the generated visuals before including them in the thesis.

The AI-assisted tools were not treated as scientific sources and were not used as substitutes for the empirical evidence collected in the study. The research problem, research questions, study design, data collection, artifact configuration, empirical datasets, reported calculations, evaluation procedures, interpretation of the study evidence, and conclusions remain the sole responsibility of the authors.

B.1 Example: Academic Language and Clarity Revision

The following representative example illustrates the use of ChatGPT for language revision and the authors' review of its suggestion.

Author-written draft

> High domain criticality can affect the result in two ways within the final planning procedure. If a file is technically relevant, it has more priority for Refactor-Now. But if the file is HIGH criticality and has low recent activity, it is assigned to Stabilize-First so it can be prepared and reviewed before refactoring.

Example prompt to ChatGPT

> Hey. Can you improve the following paragraph for grammar, clarity, and academic tone? Do not change the technical meaning. Do not introduce any new methodological claims, assumptions, or results. Preserve the terms used here, which are HIGH criticality, Refactor-Now, Stabilize-First, and recent activity.
>
> "High domain criticality can affect the result in two ways within the final planning procedure. If a file is technically relevant, it has more priority for Refactor-Now. But if the file is HIGH criticality and has low recent activity, it is assigned to Stabilize-First so it can be prepared and reviewed before refactoring."

Example AI-assisted suggestion

> Within the final planning procedure, HIGH domain criticality can affect candidates in two ways: it raises the priority of technically relevant files that remain eligible for Refactor-Now, while HIGH-criticality files with relatively low recent activity are instead directed to Stabilize-First for further preparation and review.

Final author-reviewed version used in the thesis

> Within the final planning procedure, HIGH domain criticality can have two effects: it increases the priority of technically relevant files that remain eligible for Refactor-Now, while HIGH-criticality files with relatively low recent activity are instead routed to Stabilize-First for additional preparation and review.

The final version was selected and edited by the authors to keep the explanation concise and consistent with the terminology used in the thesis.



Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden